

3 Mesh networking

This section is structured as in the layered architecture that is described in Section 2.1. In addition, there are sections on mesh security and mesh network management.

3.1 Conventions

The following conventions apply to this specification.

3.1.1 Endianness and field ordering

For the network layer, lower transport layer, upper transport layer, mesh beacons, and Provisioning, all multiple-octet numeric values shall be sent in big-endian, as described in Section 3.1.1.1.

For the access layer and Foundation Models, all multiple-octet numeric values shall be little-endian as described in Section 3.1.1.2.

Where network data structures are made of multiple fields, the fields are listed in the tables from top to bottom and they appear in the corresponding figures from left to right (i.e., the top row of the table corresponds to the left of the figure). Table 3.1 shows an example data structure made up of multiple fields.

Field	Size (bits)	Field Content Description
Field 0	4	The start of this field is in Octet 0 (left most octet in corresponding figure)
Field 1	12	The start of this field is in Octet 0 and ends in Octet 1
Field 2	16	The start of this field is in Octet 2 and ends in Octet 3

Table 3.1: Field ordering example

In order to convert the data structure defined in a table into a series of octets in the layer that uses big-endian the following procedure is used. The binary number with N unassigned bits is created. The number of bits N in the number is equal to the sum of the number of bits of every field in the table. The most significant bits (MSBs) of the number are set to the value of Field 0 (first row of the table), then the number's unassigned MSBs are set to the value of Field 1. This procedure is continued for consecutive fields of the table and ends when least significant bits (LSBs) of the number are set to value of last field of the table. As a final step the number is transmitted in big-endian format (i.e., most significant octet first). This is illustrated in Figure 3.1.

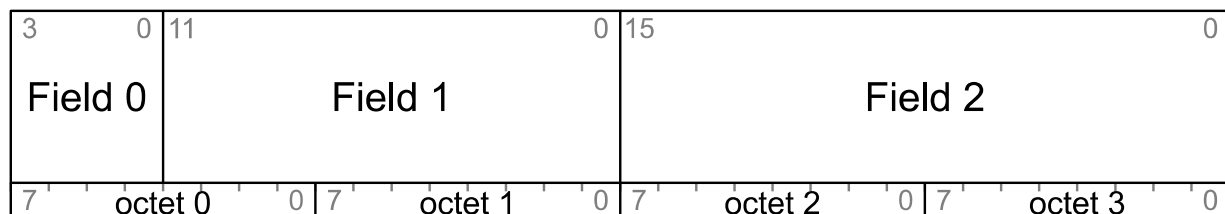


Figure 3.1: Field ordering example: big-endian

For example, the field 0 is 4 bits wide and has value of 0x6, field 1 is 12 bits wide and has value 0x987, and field 2 is 16 bits wide and has value of 0x1234. The value of the binary number is 0x69871234 and shall be transmitted as 0x69, 0x87, 0x12, 0x34.

In order to convert the data structure defined in a table into a series of octets in the layer that uses little-endian the following procedure is used. The binary number with N unassigned bits is created. The number of bits N in the number is equal to the sum of the number of bits of every field in the table. The LSBs of the number are set to the value of Field 0 (first row of the table), then the number's unassigned LSBs are set to the value of Field 1. This procedure is continued for consecutive fields of the table and ends when MSBs of the number are set to the value of last field of the table. As a final step the number is transmitted in little-endian format (i.e., least significant octet first). This is illustrated in [Figure 3.2](#).

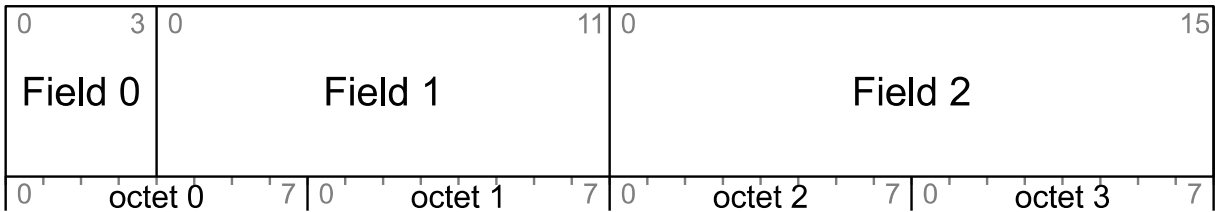


Figure 3.2: Field ordering example: little-endian

For example, the field 0 is 4 bits wide and has a value of 0x6, field 1 is 12 bits wide and has a value of 0x987, and field 2 is 16 bits wide and has a value of 0x1234. The value of the binary number is 0x12349876 and shall be transmitted as 0x76, 0x98, 0x34, 0x12.

3.1.1.1 Big-endian

When multiple-octet values are defined as sent in big-endian (also known as “network byte order”), the conventions in this section apply. For example, the value 0x123456 shall be transmitted as 0x12, 0x34, and 0x56 (most significant octet first).

3.1.1.2 Little-endian

When multiple-octet values are defined as sent in little-endian, the conventions in this section apply. For example, the value 0x123456 shall be transmitted as 0x56, 0x34, and 0x12 (least significant octet first).

3.1.2 One-shot timers

This specification establishes definitions for terminology related to one-shot timers. A one-shot timer counts down from an initial value and when the counter reaches zero, the timer expires. The timer value is defined as the current value of the counter.

[Table 3.2](#) establishes the definitions for actions that can be performed on a timer.

Action	Description
Start from the initial value	The timer value is set to the initial value and the timer countdown is immediately started.
Start from the current value	The timer value is not changed and the timer countdown is immediately started.
Stop	The timer countdown is immediately stopped and the timer value is stored.

Table 3.2: Definitions for actions that can be performed on a timer

When the timer value reaches zero, the timer countdown stops and the timer expires. The timer expiration is an event that may have a behavior defined in this specification.

Table 3.3 establishes the definitions for checking timer status.

Timer Status	Description
Is running	The status beginning when the countdown timer starts until it stops.
Is not running	The status when the timer countdown is stopped.

Table 3.3: Definitions for checking timer status

3.2 Features

This specification defines four optional features:

- Relay feature (see Section 3.4.6.1)
- Proxy feature (see Section 3.4.6.2)
- Friend feature (see Section 3.6.6.3)
- Low Power feature (see Section 3.6.6.4)

3.3 Bearers

This specification defines two mesh bearers over which mesh messages may be transported:

- An advertising bearer (see Section 3.3.1)
- A GATT bearer (see Section 3.3.2)

A node shall support the advertising bearer or the GATT bearer or both.

Bearers are identified by 2-octet indexes as described in Section 4.3.1.4.

3.3.1 Advertising bearer

When using the advertising bearer, a mesh packet shall be sent in the ADV_NONCONN_IND PDU (see [1]) using the Mesh Message AD type identified by «Mesh Message» as defined in [4].



The format of the Mesh Message AD type is defined in [Table 3.4](#).

Field	Size (octets)	Description	Req.
Length	1	Length of the AD Type and Network PDU fields	M
AD Type	1	«Mesh Message»	M
Network PDU	variable	Network PDU	M

Table 3.4: Mesh Message AD type

ADV_NONCONN_IND PDUs are used because there is no need to include the Flags AD Type in the advertising packets, thereby enabling two additional octets to be allocated to the Network PDU (see [\[5\]](#)).

To lower the probability of packet collisions on all advertising channels, the gap between consecutive packets within an Advertising Event (see [\[1\]](#)) and the sequence of the used primary advertising channels within an Advertising Event (see [\[24\]](#)) should be randomized.

A device supporting only the advertising bearer should perform passive scanning with a duty cycle as close to 100 percent as possible in order to avoid missing any incoming mesh messages or Provisioning PDUs.

All devices shall support both the Observer and Broadcaster roles, which are defined in the Generic Access Profile (GAP) [\[1\]](#).

3.3.2 GATT bearer

The GATT bearer is provided to enable devices that are not capable of supporting the advertising bearer to participate in a mesh network. The GATT bearer uses the Proxy protocol (see [Section 6](#)) to transmit and receive Proxy PDUs between two devices over a GATT connection.

The GATT bearer uses a characteristic to write to and receive notifications of mesh messages using the attribute protocol.

The GATT bearer defines two roles: a GATT Bearer Client and a GATT Bearer Server.

The GATT Bearer Client shall be a GATT Client and shall support the Proxy Client role (see [Section 6.2.1](#)). The GATT Bearer Server shall be a GATT Server and shall support the Proxy Server role (see [Section 6.2.1](#)).

The GATT Bearer Server shall instantiate one and only one Mesh Proxy Service, as defined in [Section 7.2](#).

The GATT Bearer Client shall support the Mesh Proxy Service.

The GATT Bearer Client shall perform primary service discovery using either the GATT Discover All Primary Services sub-procedure or the GATT Discover Primary Services by Service UUID sub-procedure to discover the Mesh Proxy Service.

As required by the GATT profile defined in Volume 3, Part G of the Core Specification [\[1\]](#), the GATT Bearer Client must be tolerant of additional optional characteristics in the service records of services used with the GATT profile.

The GATT Bearer Client shall use either the GATT Discover All Characteristics of a Service sub-procedure or the GATT Discover Characteristics by UUID sub-procedure to discover the characteristics of the service.

The GATT Bearer Client shall use the GATT Discover All Characteristic Descriptors sub-procedure to discover the characteristic descriptors, which are described in the following sections.

The GATT Bearer Client shall discover the Mesh Proxy Data In characteristic, Mesh Proxy Data Out characteristic and its Client Characteristic Configuration descriptor. Once the Client Characteristic Configuration descriptor has been discovered, it shall enable notifications using this characteristic.

To send a Proxy PDU (see Section 6.3), the GATT Bearer Client shall use the Write Without Response sub-procedure to write the Proxy PDU to the GATT Bearer Server by writing to the Mesh Proxy Data In characteristic.

To receive a Proxy PDU, the GATT Bearer Client shall be able to receive multiple notifications of the Mesh Proxy Data Out characteristic. Each notification contains a single Proxy PDU.

[Figure 3.3](#) illustrates the GATT Bearer Server and network layer (see Section 3.4) interactions.

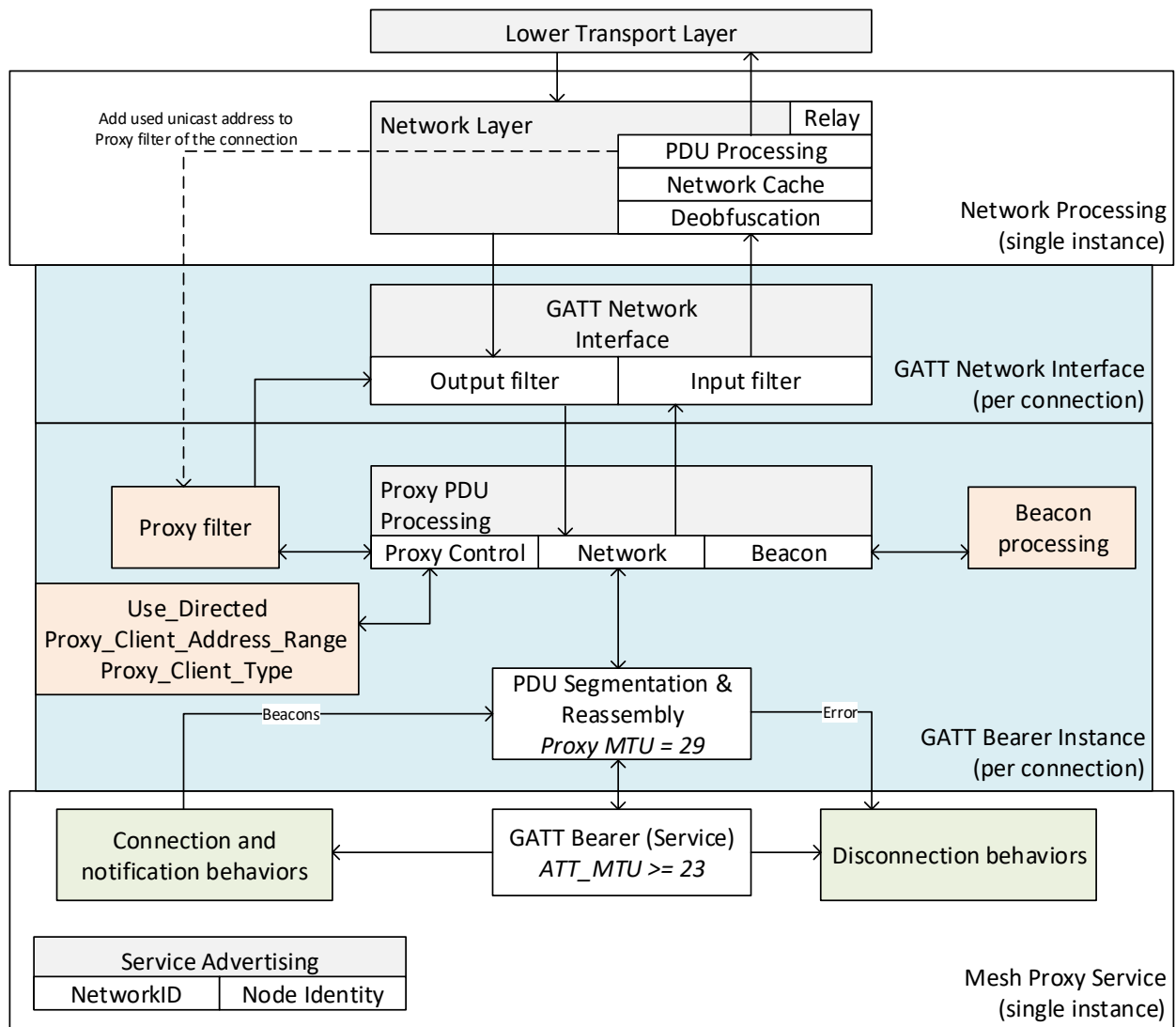


Figure 3.3: GATT Bearer Server and network layer interactions

3.4 Network layer

The network layer defines the Network PDU format that allows Lower Transport PDUs to be transported by the bearer layer. It decrypts and authenticates and forwards incoming Network PDUs received on input interfaces to upper layers and/or output interfaces and encrypts and authenticates and forwards outgoing Network PDUs delivering them to output network interfaces.

3.4.1 Endianness

All multiple-octet numeric values in this layer shall be sent in big-endian, as described in Section 3.1.1.1.

3.4.2 Addresses

The network layer defines four basic types of addresses: unassigned, unicast, virtual, and group.

Addresses are 16 bits in length and are encoded as defined in [Table 3.5](#) below.

Values	Address Type
0b0000000000000000	Unassigned Address
0b0xxxxxxxxxxxxxxx (excluding 0b0000000000000000)	Unicast Address
0b10xxxxxxxxxxxxxxx	Virtual Address
0b11xxxxxxxxxxxxxxx	Group Address

Table 3.5: 16-bit address allocations

3.4.2.1 Unassigned address

An unassigned address is an address in which the element of a node has not been configured yet or no address has been allocated. The unassigned address shall have the value 0x0000 as shown in [Figure 3.4](#) below. This may be used, for example, to disable message publishing of a model by setting the publish address of a model to the unassigned address.

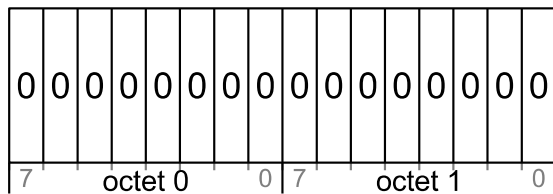


Figure 3.4: Unassigned address format

An unassigned address shall not be used in the SRC field or the DST field of a Network PDU.

3.4.2.2 Unicast address

A unicast address is a unique address allocated to each element. A unicast address has bit 15 set to 0. The unicast address shall not have the value 0x0000, and therefore can have any value from 0x0001 to 0x7FFF inclusive as shown in [Figure 3.5](#) below.

A unicast address is allocated to each element of a node for a term of the node on the network. The address allocation for the initial term is performed by a Provisioner during provisioning as described in [Section 5.4.2.5](#). The address may be changed by executing the Node Address Refresh procedure (see [Section 3.11.8.5](#)) or the Node Composition Refresh procedure (see [Section 3.11.8.6](#)). The address may be unallocated by a Provisioner to allow the address to be reused using the procedure defined in [Section 3.11.7](#).

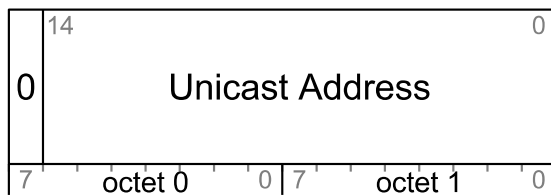


Figure 3.5: Unicast address format

A unicast address shall be used in the SRC field of a Network PDU and may be used in the DST field of a Network PDU. A Network PDU sent to a unicast address shall be processed by at most one element.

3.4.2.2.1 Unicast address range format

A range of unicast addresses may be used in a message, for example, to communicate all the unicast addresses used by a node. In this case, the unicast address range format can be used to efficiently pack the range of unicast addresses. In the unicast address range format, the 15 least significant bits of the starting unicast address of the range and the number of addresses in the range are formatted as in [Table 3.6](#).

Field Name	Bits	Description	Req.
LengthPresent	1	Indicates the presence or absence of the RangeLength field.	M
RangeStart	15	15 least significant bits of the starting unicast address	M
RangeLength	8	Number of addresses in the range (0x02 – 0xFF)	C.1

Table 3.6: Unicast address range format

C.1: If the LengthPresent field value is 1, the RangeLength field shall be present; otherwise, the RangeLength field shall not be present.

The LengthPresent field indicates whether the address range contains a single address or multiple addresses. The LengthPresent field value shall be set to 1 when the number of addresses in the range of unicast addresses is greater than 1; otherwise, the LengthPresent field value shall be set to 0.

The RangeStart field shall contain the 15 least significant bits of the starting unicast address of the range of unicast addresses. The RangeStart field value shall not be 0.

If present, the RangeLength field shall contain the number of addresses in the range of unicast addresses. The valid values for the RangeLength field are 0x02 to 0xFF. The values of 0x00 and 0x01 are prohibited.

The sum of the RangeStart field value and the RangeLength field value shall not exceed 0x8000.

3.4.2.2.2 Unicast address range list

A message may contain multiple unicast address range values. For example, a unicast address range list can be used to communicate all the element addresses of Low Power nodes of a Friend node.

The values shall be serialized as shown in [Figure 3.6](#). The length of each unicast address range value is either 2 or 3 octets depending on the LengthPresent field value within each unicast address range value.

1	RangeStart		Range Length	0	RangeStart		0	RangeStart		1	RangeStart		Range Length
	Octet 0	Octet 1	Octet 2		Octet 0	Octet 1		Octet 0	Octet 1		Octet 0	Octet 1	Octet 2
	Address Range 0				Address Range 1			Address Range 2			Address Range 3		

Figure 3.6: Unicast address range list example

3.4.2.2.3 Matching an address to the unicast address range format

To check for the presence of a specific unicast address within the unicast address range format, the node shall convert the RangeStart field to the range start address by setting the 15 least significant bits of the range start address to the RangeStart field value and setting the most significant bit of the range start address to 0. If the LengthPresent field value is 1, the node shall compute the range end address by adding the RangeLength field value to the range start address; otherwise, the node shall increment the range start address by 1 to derive the range end address. A unicast address is matched to the unicast address range format if it is greater than or equal to the range start address and less than the range end address.

3.4.2.2.4 Converting to and from the unicast address range format

This section specifies the derivation of the primary element address and the number of secondary elements (i.e., the secondary elements count of a node) from the unicast address range format and, conversely, the derivation of the unicast address range format from the primary element address and secondary elements count.

Conversion to the unicast address range format

To derive the unicast address range format from the primary element address and the number of secondary elements, the node shall use the following procedure:

- If the node has secondary elements, the LengthPresent field value shall be 1; otherwise, the LengthPresent field value shall be 0.
- The RangeStart field shall be the 15 least significant bits of the primary element address of the node.
- If the node has secondary elements, the RangeLength field value shall be the number of secondary elements plus 1; otherwise, the RangeLength field shall not be present.

For example, if a node has 5 secondary elements and the primary element address is 0x1234, then the value of the binary number of the unicast address range of the elements of the node and the sequence of octets for its transmission depend on the used endianness as follows:

- If big-endian is used (e.g., when sending a Transport Control message), the value of the binary number is 0x923406 and is transmitted as 0x92, 0x34, 0x06.
- If little-endian is used (e.g., when sending an Access message), the value of the binary number is 0x062469 and is transmitted as 0x69, 0x24, 0x06.

Conversion from the unicast address range format

To derive the primary element address and the number of secondary elements from the unicast address range format, the node shall use the following procedure:

- If the LengthPresent field value is 0, the number of secondary elements is 0; otherwise, the number of secondary elements is equal to RangeLength – 1.
- The 15 least significant bits of the primary element address of the node are equal to the RangeStart field. The most significant bit of the unicast address is always 0.

For example, if the unicast address range of a node is 0x1234, the node's primary element address is 0x1234 and there are no secondary elements on the node.

3.4.2.3 Virtual address

A virtual address represents a set of destination addresses. Each virtual address logically represents a Label UUID, which is a 128-bit value that does not have to be managed centrally. One or more elements may be programmed to publish or subscribe to a Label UUID. The Label UUID is not transmitted and shall be used as the Additional Data field of the message integrity check value in the upper transport layer (see Section 3.9.7.1).

The virtual address is a 16-bit value that has bit 15 set to 1, bit 14 set to 0, and bits 13 to 0 set to the value of a hash. This hash is a derivation of the Label UUID such that each hash represents many Label UUIDs.

$$SALT = s1("vtad")$$

$$hash = AES-CMAC_{SALT}(Label\ UUID) \bmod 2^{14}$$

When an Access message is received to a virtual address that has a matching hash, each corresponding Label UUID is used by the upper transport layer as additional data as part of the authentication of the message until a match is found.

Transport Control messages cannot use virtual addresses.

Label UUIDs may be generated randomly as defined in [6]. A Configuration Manager may assign and track virtual addresses; however, two devices can also create a virtual address using some out-of-band (OOB) mechanism. Unlike group addresses, these could be agreed upon by the devices involved and would not need to be registered in the centralized provisioning database, as they are unlikely to be duplicated.

A disadvantage of virtual addresses is that a multi-segment message is required to transfer a Label UUID to a publishing or subscribing node during configuration.

A virtual address can have any value from 0x8000 to 0xBFFF as shown in Figure 3.7 below.

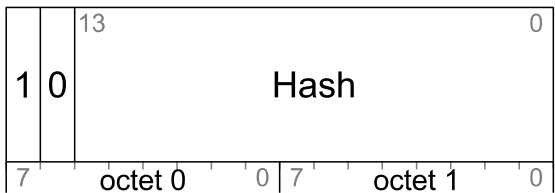


Figure 3.7: Virtual address format

Note: When factoring in a 32-bit MIC and the size of the hash, there is only a $1/2^{46} = 1.42 \times 10^{-14}$ likelihood that two matching virtual addresses using the same application key but different Label UUIDs will collide.

3.4.2.4 Group address

A group address is an address that is programmed into zero or more elements. A group address has bit 15 set to 1 and bit 14 set to 1, as shown in Figure 3.8 below. Group addresses in the range 0xFF00 through 0xFFFF are reserved for Fixed Group addresses (see Table 3.7), and addresses in the range 0xC000 through 0xFEFF are generally available for other usage.

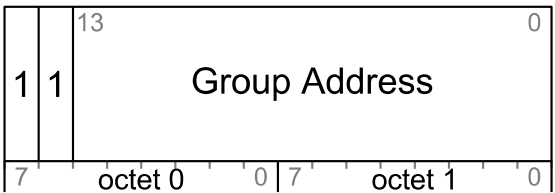


Figure 3.8: Group address format

A group address shall only be used in the DST field of a Network PDU. A Network PDU sent to a group address shall be delivered to all the instances of models that subscribe to this group address.

There are two types of group addresses; those that can be assigned dynamically and those that are fixed. The fixed group addresses are defined in Table 3.7 below.

Values	Fixed Group Address Name
0xFFFF9	all-ipt-border-routers
0xFFFFA	all-ipt-nodes
0xFFFFB	all-directed-forwarding-nodes
0xFFFFC	all-proxies
0xFFFFD	all-friends
0xFFFFE	all-relays
0xFFFFF	all-nodes

Table 3.7: Fixed group addresses

This specification does not define any special behaviors related to values from 0xFF00 to 0xFFFF8, all-ipt-border-routers, and all-ipt-nodes fixed group addresses.

3.4.3 Address validity

Table 3.8 shows which address types are valid for use in the SRC field and the DST field.

Address Type	Valid in SRC Field	Valid in DST Field	
		Segmented and Unsegmented Control Messages (see Section 3.5.2)	Segmented and Unsegmented Access Messages (see Section 3.5.2)
Unassigned Address	No	No	No
Unicast Address	Yes	Yes	Yes
Virtual Address	No	No	Yes
Group Address	No	Yes	Yes

Table 3.8: Address type and message field validity

A fixed group address defined as Reserved for Future Use in Section 3.4.2.4 is a valid group address for the purposes of Table 3.8.

Table 3.9 shows which address types are valid for use with device keys and application keys.

Address Type	Valid with Device Key	Valid with Application Key
Unassigned Address	No	No
Unicast Address	Yes	Yes
Virtual Address	No	Yes
Group Address	No	Yes

Table 3.9: Address type and access layer key type validity

3.4.4 Network PDU

The mesh Network PDU format is defined in Table 3.10 and illustrated in Figure 3.9 below:

Field Name	Bits	Description	Req.
IVI	1	Least significant bit of IV Index	M
NID	7	Value derived from the NetKey used to identify the EncryptionKey and PrivacyKey used to secure this PDU	M
CTL	1	Network Control	M
TTL	7	Time To Live	M
SEQ	24	Sequence number	M
SRC	16	Source address	M
DST	16	Destination address	M

Field Name	Bits	Description	Req.
TransportPDU	8 to 128	Transport Protocol Data Unit	M
NetMIC	32 or 64	Message Integrity Check for Network	M

Table 3.10: Network PDU field definitions

Network PDUs are secured using keys derived from a single network key, as identified by the NID field.

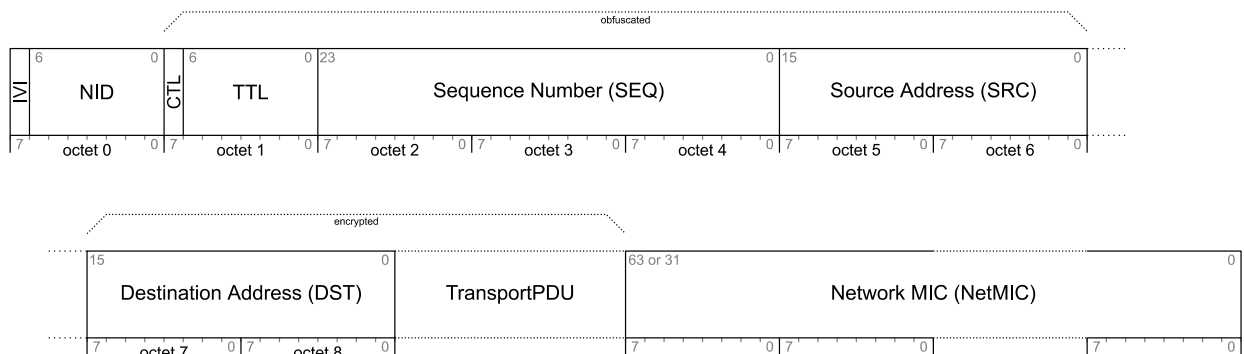


Figure 3.9: Network PDU format

Certain functionalities defined in this specification may require that a Network PDU be tagged with additional metadata. For example, tagging a Network PDU as relay is used to control the number of retransmissions of the Network PDU by the Advertising Bearer Network Interface (see Section 3.4.5.4).

3.4.4.1 IVI

The IVI field contains the least significant bit of the IV Index used in the nonce to authenticate and encrypt this Network PDU (see Section 3.9.2.11).

3.4.4.2 NID

The NID field contains a 7-bit network identifier that allows for an easier lookup of the EncryptionKey and PrivacyKey used to authenticate and encrypt this Network PDU (see Section 3.9.6.3.1).

The NID value is derived from the network key in conjunction with the EncryptionKey and PrivacyKey. It is derived differently for Network PDUs using managed flooding security material, Network PDUs using directed security material, and Network PDUs using friendship security material (see Section 3.9.6.3.1).

3.4.4.3 CTL

The CTL field is a 1-bit value that is used to determine if the Network PDU is part of a Transport Control message or an Access message, as illustrated in Table 3.11.

If the CTL field is set to 0, the NetMIC is a 32-bit field and the Lower Transport PDU contains an Access message.

If the CTL field is set to 1, the NetMIC is a 64-bit field and the Lower Transport PDU contains a Transport Control message.

CTL Field	Message Type	NetMIC Size (bits)
0	Access message	32
1	Transport Control message	64

Table 3.11: CTL field message types and NetMIC sizes

3.4.4.4 TTL

The TTL field is a 7-bit field. The TTL field values are defined in [Table 3.12](#).

Value	Description
0	Network PDU has not been relayed and will not be relayed.
1	Network PDU has been relayed and will not be relayed.
2 - 126	Network PDU has been relayed or Network PDU has not been relayed. Network PDU can be relayed.
127	Network PDU has not been relayed and can be relayed.

Table 3.12: TTL field values

The initial value of this field is set by the transmitting layer (lower transport layer, upper transport layer, access, foundation model, model) or an application and used by the network layer when operating as a Relay node or as a Directed Relay node.

The use of the TTL value of zero allows a node to transmit a Network PDU that it knows will not be relayed, and therefore the receiving node can determine that the sending node is a single radio link away. The use of a TTL value of one or larger cannot be used for such a determination.

3.4.4.5 SEQ

The SEQ field is a 24-bit integer. The combined SEQ field, IV Index field, and SRC field (see [Section 3.4.4.6](#)) shall be a unique value for each new Network PDU that this element originates.

3.4.4.6 SRC

The SRC field is a 16-bit value that identifies the element that originated this Network PDU. This address shall be a unicast address.

The SRC field is set by the originating element and untouched by nodes operating as a Relay node or as a Directed Relay node.

3.4.4.7 DST

The DST field is a 16-bit value that identifies the element or elements that this Network PDU is directed towards. This address shall be a unicast address, a group address, or a virtual address.

The DST field is set by the originating node and is untouched by the network layer in nodes operating as a Relay node or as a Directed Relay node.

3.4.4.8 TransportPDU

The TransportPDU field, from a network layer point of view, is a sequence of octets of data. When the CTL field value is 0, the TransportPDU field shall be a maximum of 128 bits. When the CTL field value is 1, the TransportPDU field shall be a maximum of 96 bits.

The TransportPDU field is set by the originating lower transport layer and shall not be changed by the network layer.

3.4.4.9 NetMIC

The NetMIC field is a 32-bit or 64-bit field (depending on the value of the CTL field) that authenticates that the DST and TransportPDU fields have not been changed.

When the CTL field value is 0, the size of the NetMIC field shall be 32 bits. When the CTL field value is 1, the size of the NetMIC field shall be 64 bits.

The NetMIC field value is set by the network layer at each node that transmits or relays this Network PDU.

3.4.5 Network interfaces

The network layer supports sending and receiving Network PDUs via multiple bearers. Multiple instances of a bearer may be present. Each instance of a bearer is connected to the network layer via a network interface. To allow sending Network PDUs between elements within the same node the local interface is used.

For example, a node may have three interfaces: one used to send and receive Network PDUs via an advertising bearer and two interfaces to a GATT bearer, one for each client connected via a GATT connection.

Interfaces provide input and output filters. Filters may be configured using bearer-specific PDUs or internally by services exposed on a node, such as the Mesh Proxy Service (see Section 7.2).

3.4.5.1 Interface input filter

The interface input filter determines whether an incoming Network PDU is delivered to the network layer for further processing or it is dropped.

A feature or a protocol in this specification may define an input filter. For example, the Proxy Protocol defines filters (see Section 6.4).

The input filter of the interface connected to the GATT bearer shall drop all Network PDUs that have been secured using the friendship security credentials.

3.4.5.2 Interface output filter

The interface output filter determines whether an outgoing Network PDU is delivered to a bearer or it is dropped.

A feature or a protocol in this specification may define an output filter. For example, the Proxy Protocol defines filters (see Section 6.4).

The output filter of the interface connected to advertising or GATT bearers shall drop all Network PDUs with the TTL field value set to 1 unless they contain a Network PDU that is tagged as relay.

The output filter of the interface connected to the GATT bearer shall drop all Network PDUs secured using the friendship security credentials.

3.4.5.3 Local Network Interface

A Local Network Interface allows sending Network PDUs between elements within the same node.

A node shall implement a Local Network Interface.

Upon receiving a Network PDU by a Local Network Interface, the Network PDU shall be delivered to all elements of the node.

3.4.5.4 Advertising Bearer Network Interface

When transmitting a Network PDU that is tagged as friendship, the Advertising Bearer Network Interface shall transmit the Network PDU over the advertising bearer only once.

When transmitting a Network PDU that is not tagged as friendship, the Advertising Bearer Network Interface shall transmit the Network PDU over the advertising bearer using the following configuration:

- If the Network PDU is not using directed security material, then the relay tag is checked:
 - If the Network PDU is not tagged as relay, the number and timing of the transmissions shall be as indicated by the Network Transmit state (see Section 4.2.20).
 - If the Network PDU is tagged as relay, the number and timing of the transmissions shall be as indicated by the Relay Retransmit state (see Section 4.2.21).
- If the Network PDU is using the directed security material, then the combination of relay tag and CTL field is checked:
 - If the Network PDU is not tagged as relay, and the CTL field value is equal to 0, then the number and timing of the transmissions shall be as indicated by the Directed Network Transmit state (see Section 4.2.32.2).
 - If the Network PDU is not tagged as relay, and the CTL field value is equal to 1, then the number and timing of the transmissions shall be as indicated by the Directed Control Network Transmit state (see Section 4.2.39).
 - If the Network PDU is tagged as relay, and the CTL field value is equal to 0, then the number and timing of the transmissions shall be as indicated by the Directed Relay Retransmit state (see Section 4.2.34).
 - If the Network PDU is tagged as relay, and the CTL field value is equal to 1, then the number and timing of the transmissions shall be as indicated by the Directed Control Relay Retransmit state (see Section 4.2.40).

When transmitting a Network PDU that is tagged as relay, the start time of the transmission should be randomized by a small interval to avoid collisions between multiple relays that have received the Network PDU at the same time. If the DST field value of the Network PDU is the all-directed-forwarding-nodes fixed group address (see Section 3.4.2.4), the start time of the transmission should be additionally randomized by an interval from 0 to 100 milliseconds.

3.4.6 Network layer behavior

3.4.6.1 Relay feature

The Relay feature is used to relay/forward Network PDUs received by a node over the advertising bearer. This feature is optional and if supported can be enabled and disabled. If the Proxy feature is supported, then both GATT and advertising bearers shall be supported.

3.4.6.2 Proxy feature

The Proxy feature is used to relay/forward Network PDUs received by a node between GATT and advertising bearers. This feature is optional and if supported can be enabled and disabled. When this feature is supported, the Advertising with Network ID (see Section 7.2.2.2.2) shall be supported and the Mesh Proxy Service (see Section 7.2) shall be contained in the GATT Server on a node; otherwise, the Mesh Proxy service may be contained in the GATT Server.

3.4.6.3 Receiving a Network PDU

A Network PDU is delivered from a bearer to the network layer via a network interface. The interface shall apply filtering rules defined by its input filter (see Section 3.4.5.1). If the Network PDU passes through the input filter, it is delivered to the network layer for further processing.

Upon receiving a Network PDU, the node shall check whether the NID field value matches one or more known NIDs. If the NID field value does not match a known NID, then the Network PDU shall be ignored. If the NID field value matches a known NID, the node shall attempt to authenticate the Network PDU using the security credentials derived from each known network key that matched. If the Network PDU does not authenticate against any known network key, then the Network PDU shall be ignored. If the Network PDU does authenticate against a network key, and the SRC and DST fields are considered valid (see Section 3.4.3), and an entry corresponding to the Network PDU is not in the Network Message Cache (see Section 3.4.6.5), then the Network PDU shall be processed by the lower transport layer.

When the Network PDU is processed by the lower transport layer, and the TTL field has a value of 2 or greater, and the destination is not a unicast address of an element on this node, then the Network PDU shall be evaluated against retransmission conditions for incoming Network PDUs as defined in Table 3.13. For a Network PDU, there may be more than one matching row in Table 3.13. If there is no row that matches the retransmission conditions, then the Network PDU shall not be retransmitted.

For each row in Table 3.13, if the Network PDU is delivered from the bearer identified in the Inbound Bearer column, and is secured using the security material identified in the Inbound Security Material column, and all conditions in the Conditions column are met, then the actions in the Additional Actions column, if present, shall be executed, and the Network PDU shall be retransmitted to network interface(s) identified by the Outbound Bearers column using the security material identified in the Outbound Security Material column. The IV Index used when retransmitting the Network PDU shall be the same IV Index as in the received Network PDU. The TTL field value of the retransmitted Network PDU shall be equal to the

TTL field value of the received Network PDU decremented by 1. The network key used when retransmitting the Network PDU depends on whether the node is a Subnet Bridge node.

If the node is not a Subnet Bridge node, the network key used when retransmitting the Network PDU shall be the same network key used to secure the received Network PDU.

If the node is a Subnet Bridge node, the node shall check all the Bridging Table state entries to determine whether the Network PDU is to be bridged to different subnets.

For a given Bridging Table state entry (see Section 4.2.42), the Network PDU shall be retransmitted using the NetKey identified by NetKeyIndex2 of the entry if all the following conditions are met:

- The SRC field value in the Network PDU matches the address of the node in the first subnet that is indicated in the Address1 field.
- The DST field value in the Network PDU matches the address of the node in the second subnet that is indicated in the Address2 field.
- The received Network PDU was encrypted using the NetKey identified by NetKeyIndex1 of the entry.

The Network PDU shall be retransmitted using the NetKey identified by NetKeyIndex1 of the Bridging Table state entry if all the following conditions are met:

- The SRC field value in the Network PDU matches the address of the node in the second subnet that is indicated in the Address2 field.
- The DST field value in the Network PDU matches the address of the node in the first subnet that is indicated in the Address1 field.
- The received Network PDU was encrypted using the NetKey identified by NetKeyIndex2 of the entry.
- The Directions value of the entry is 0x02.

Inbound Bearer	Inbound Security Material	Conditions	Additional Actions	Outbound Bearers	Outbound Security Material
ADV	flooding	Relay is enabled.	Tag as relay	ADV	flooding
ADV	flooding	Proxy is enabled.	–	GATT	flooding
ADV	flooding	Traffic is to be bridged. AND Directed forwarding is enabled. AND DST is a valid path destination. AND Path from supporting node does not exist.	Create Path Origin State Machine AND Update replay protection list	all bearers	flooding

Inbound Bearer	Inbound Security Material	Conditions	Additional Actions	Outbound Bearers	Outbound Security Material
ADV	flooding	Traffic is to be bridged. AND Directed forwarding is enabled. AND Path from supporting node exists. AND Dependent node is not listed.	Start Directed Forwarding Dependents Update AND Update replay protection list	all bearers	flooding
ADV	flooding	Traffic is to be bridged. AND Directed forwarding is enabled. AND Path from supporting node exists. AND Dependent node is listed.	Update replay protection list	path bearers	directed
ADV	friendship	Friend is enabled. AND Directed friend is disabled.	–	all bearers	flooding
ADV	friendship	Directed friend is enabled. AND DST is a valid path destination. AND Path from supporting node does not exist.	Create Path Origin State Machine	all bearers	flooding
ADV	friendship	Directed friend is enabled. AND Path from supporting node exists. AND Dependent node is not listed.	Start Directed Forwarding Dependents Update	all bearers	flooding
ADV	friendship	Directed friend is enabled. AND Path from supporting node exists. AND Dependent node is listed.	–	path bearers	directed

Inbound Bearer	Inbound Security Material	Conditions	Additional Actions	Outbound Bearers	Outbound Security Material
ADV	directed	Directed relay is enabled. AND Path exists.	Tag as relay	path bearers	directed
ADV	directed	Directed proxy is enabled. AND Path to supporting node exists. AND Dependent node is listed.	–	GATT	flooding
ADV	directed	Traffic is to be bridged. AND Directed forwarding is enabled. AND Path from supporting node does not exist.	Create Path Origin State Machine AND Update replay protection list	all bearers	flooding
ADV	directed	Traffic is to be bridged. AND Directed forwarding is enabled. AND Path from supporting node exists. AND Dependent node is not listed.	Start Directed Forwarding Dependents Update AND Update replay protection list	all bearers	flooding
ADV	directed	Traffic is to be bridged. AND Directed forwarding is enabled. AND Path exists.	Update replay protection list	path bearers	directed
GATT	flooding	Proxy is enabled. AND Directed proxy is disabled.	–	all bearers	flooding
GATT	flooding	Directed proxy is enabled. AND Directed forwarding is not selected.	–	all bearers	flooding

Inbound Bearer	Inbound Security Material	Conditions	Additional Actions	Outbound Bearers	Outbound Security Material
GATT	flooding	Directed forwarding is selected. AND DST is a valid path destination. AND Path from supporting node does not exist.	Create Path Origin State Machine	all bearers	flooding
GATT	flooding	Directed forwarding is selected. AND Path from supporting node exists. AND Dependent node is not listed.	Start Directed Forwarding Dependents Update	all bearers	flooding
GATT	flooding	Directed forwarding is selected. AND Path from supporting node exists. AND Dependent node is listed.	–	path bearers	directed
GATT	directed	Directed relay is enabled. AND Path exists.	–	path bearers	directed
GATT	directed	Traffic is to be bridged. AND Directed forwarding is enabled. AND Path from supporting node does not exist.	Create Path Origin State Machine AND Update replay protection list	all bearers	flooding
GATT	directed	Traffic is to be bridged. AND Directed forwarding is enabled. AND Path from supporting node exists. AND Dependent node is not listed.	Start Directed Forwarding Dependents Update AND Update replay protection list	all bearers	flooding

Inbound Bearer	Inbound Security Material	Conditions	Additional Actions	Outbound Bearers	Outbound Security Material
GATT	directed	Traffic is to be bridged. AND Directed forwarding is enabled. AND Path exists.	Update replay protection list	path bearers	directed

Table 3.13: Network layer Network PDU retransmission requirements

The following paragraphs define the requirements associated with each column of [Table 3.13](#).

Inbound Bearer. The entries in the Inbound Bearer column of [Table 3.13](#) identify the type of bearer from which the Network PDU was delivered:

- ADV: The inbound Network PDU is delivered from the advertising bearer.
- GATT: The inbound Network PDU is delivered from the GATT bearer.

Inbound Security Material. The entries in the Inbound Security Material column of [Table 3.13](#) identify the type of security material used to secure the incoming Network PDU (see [Section 3.9.6.3.1](#)):

- flooding: The inbound Network PDU is secured using managed flooding security material.
- friendship: The inbound Network PDU is secured using friendship security material.
- directed: The inbound Network PDU is secured using directed security material.

Conditions. [Table 3.14](#) defines the requirements for the conditions in the Conditions column of [Table 3.13](#).

Condition	Requirements
Proxy is enabled	The Proxy feature is supported and enabled, or the Private Proxy functionality is supported and enabled or the On-Demand Private GATT Proxy state has a value in the range 0x01 to 0xFF (see Section 4.2.47).
Relay is enabled	The Relay feature is supported and enabled.
Directed proxy is enabled	Directed proxy functionality is enabled in the subnet from which the Network PDU is received.
Directed proxy is disabled	Directed proxy functionality is not supported or is disabled for the subnet from which the Network PDU is received.
Directed forwarding is selected	The Use_Directed connection parameter is Enabled in the subnet from which the Network PDU is received and the received Network PDU is not tagged with immutable-credentials tag.

Condition	Requirements
Directed forwarding is not selected	The Use_Directed connection parameter is Disabled in the subnet from which the Network PDU is received; or the Use_Directed connection parameter is Enabled in the subnet from which the Network PDU is received and the received Network PDU is tagged with immutable-credentials tag.
Traffic is to be bridged	Subnet bridge functionality is enabled, and the Network PDU is checked for replay protection (see Section 3.9.8), and the conditions for retransmitting the Network PDU in bridged subnets are met.
Path from supporting node does not exist	The path for the Network PDU does not exist in the subnet where the Network PDU is to be retransmitted as defined in Section 3.6.8.5.1, using the primary element address of the node processing the Network PDU as the source address, and using the DST field value of the Network PDU as the destination address.
Path from supporting node exists	The path for the Network PDU exists in the subnet where the Network PDU is to be retransmitted, as defined in Section 3.6.8.5.1, by using the primary element address of the node processing the Network PDU as the source address and by using the DST field value of the Network PDU as the destination address.
Path to supporting node exists	The path for the Network PDU exists in the subnet where the Network PDU is to be retransmitted, as defined in Section 3.6.8.5.1, by using the primary element address of the node processing the Network PDU as the destination address and by using the SRC field value of the Network PDU as the source address.
Path exists	The path for the Network PDU exists in the subnet where the Network PDU is to be retransmitted as defined in Section 3.6.8.5.1, using the SRC field value of the Network PDU as the source address and the DST field value of the Network PDU as the destination address.
Directed forwarding is enabled	Directed forwarding functionality is enabled in the subnet where the Network PDU is to be retransmitted.
Directed friend is enabled	Directed friend functionality is enabled in the subnet from which the Network PDU is received.
Directed friend is disabled	Directed friend functionality is not supported or is disabled in the subnet from which the Network PDU is received.
Directed relay is enabled	Directed relay functionality is enabled in the subnet from which the Network PDU is received.
Dependent node is listed	The node is the Path Origin of the path for the Network PDU, and the SRC field value of the Network PDU is present in the Dependent_Origin_List field of the Forwarding Table entry that corresponds to the path; or the node is the Path Target of the path for the Network PDU, and the path is a two-way path, and the SRC field value of the Network PDU is present in the Dependent_Target_List field of the Forwarding Table entry that corresponds to the path; or the DST field value of the Network PDU is the all-directed-forwarding-nodes fixed group address.

Condition	Requirements
Dependent node is not listed	The SRC field value of the Network PDU is absent from the Dependent_Origin_List of the Forwarding Table entry that corresponds to the path for the Network PDU.
DST is a valid path destination	The DST field value of the Network PDU is different from the all-nodes and all-relays fixed group addresses.

Table 3.14: Conditions in the Condition column of Table 3.13

Additional Actions. The entries in the Additional Actions column of Table 3.13 define actions, if any, to be taken in addition to retransmitting the inbound Network PDU:

- **Create Path Origin State Machine:** If a Path Origin State Machine for the DST field value of the Network PDU does not exist, and the DST field value of the Network PDU is not the all-directed-forwarding-nodes fixed group address, the node shall instantiate a Path Origin State Machine in the Initial state for that destination (see Section 4.4.7.2).
- **Start Directed Forwarding Dependents Update:** The node shall start a Directed Forwarding Dependents Update (see Section 3.6.8.2.5).
- **Tag as Relay:** The retransmitted Network PDU shall be tagged as relay.
- **Update replay protection list:** The node shall update the replay protection list for the subnet where the Network PDU is to be retransmitted as defined in Section 3.9.8.

Outbound Bearers. The entries in the Outbound Bearers column of Table 3.13 define the network interfaces to which the Network PDU will be retransmitted:

- **GATT:** The Network PDU shall be retransmitted to all network interfaces connected to the GATT bearer.
- **ADV:** The Network PDU shall be retransmitted to all network interfaces connected to the advertising bearer.
- **all bearers:** The Network PDU shall be retransmitted to all network interfaces.
- **path bearers:** The Network PDU shall be retransmitted to all network interfaces connected to the bearers indicated in all the Forwarding Table entries corresponding to the path (see Section 3.6.8.5.1).

Outbound Security Material. The entries in the Outbound Security Material column of Table 3.13 identify the type of security material used to secure the retransmitted Network PDU:

- **flooding:** The retransmitted Network PDU shall be secured using managed flooding security material.
- **directed:** The retransmitted Network PDU shall be secured using directed security material.

Figure 3.10 illustrates an example of processing steps for an incoming Network PDU.



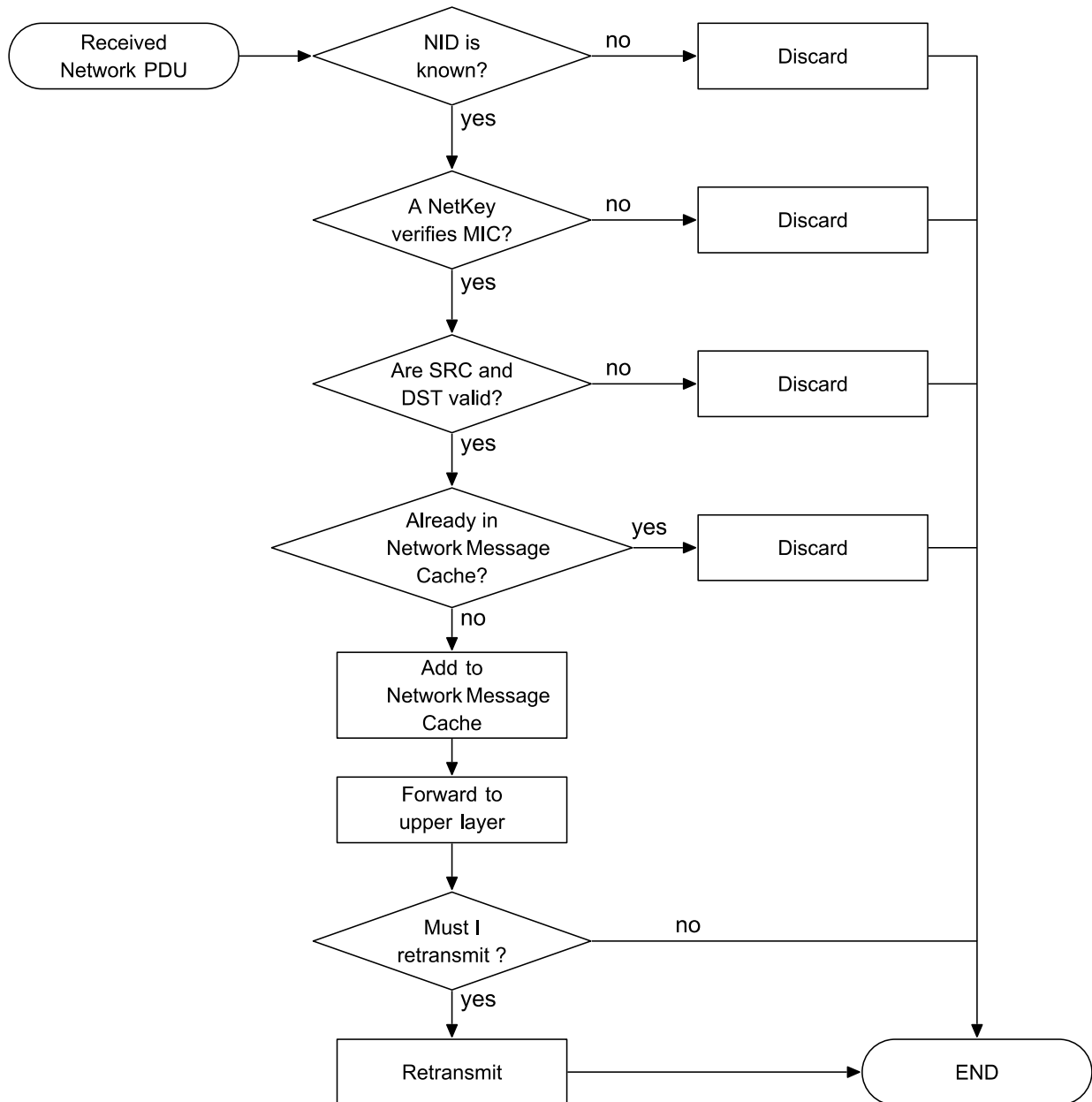


Figure 3.10: Example of Network PDU processing steps

3.4.6.4 Transmitting a Network PDU

Network PDUs are transmitted by an element in the context of a mesh subnet, which is identified by a unique network key.

The IVI field shall be set to the least significant bit of the IV Index value being used to transmit for the mesh subnet.

The NID field shall be set to the NID value associated with the EncryptionKey and PrivacyKey used for encryption and obfuscation.

The CTL field shall be set by a higher layer.

The TTL field shall be set by a higher layer.

The SEQ field shall be set by the network layer to the sequence number of the element. The sequence number shall then be incremented by one for every new Network PDU.

The SRC field shall be set by the network layer to the unicast address of the element that is sending this Network PDU.

The DST field shall be set to a unicast address, a group address, or a virtual address to identify the destination element or elements and shall be set by the lower transport, upper transport, or access layer.

The TransportPDU field shall be set by a higher layer.

The NetMIC field shall be set as defined in Section 3.9.7.2.

If the Network PDU security material is not set by the network layer or any higher layer, the Network PDU shall be secured using the managed flooding security credentials.

If the Network PDU is tagged with the immutable-credentials tag (see Section 3.7.3.1), and the Network PDU is secured using the friendship security credentials, the Network PDU shall be delivered to the advertising bearer network interface.

If the Network PDU is tagged with the immutable-credentials tag (see Section 3.7.3.1), and the Network PDU is not secured using the friendship security credentials, the Network PDU shall be delivered to all network interfaces.

If the Network PDU is not tagged with the immutable-credentials tag, and directed forwarding functionality is supported and enabled in the subnet over which the Network PDU is transmitted, and the DST field is set to a unicast address, and a path from the SRC field value to the DST field value exists (see Section 3.6.8.5.1), then the Network PDU shall use the directed security credentials (see Section 3.9.6.3.1) and shall be delivered to all network interfaces connected to the bearers indicated in all the Forwarding Table entries corresponding to the path (see Section 3.6.8.5.1).

If the Network PDU is not tagged with the immutable-credentials tag, and directed forwarding functionality is supported and enabled in the subnet over which the Network PDU is transmitted, and the DST field is set to a group address or a virtual address, and a path from the SRC field value to the DST field value exists (see Section 3.6.8.5.1), and the Path_Not_Ready field (see Section 4.2.29.2) in the Forwarding Table entry for the path is set to 0, then the Network PDU shall use the directed security credentials (see Section 3.9.6.3.1) and shall be delivered to all network interfaces connected to the bearers indicated in all the Forwarding Table entries corresponding to the path (see Section 3.6.8.5.1).

Each network interface shall apply filtering rules defined by its output filter (see Section 3.4.5.2). If the Network PDU passes through the output filter, it shall be transmitted on a bearer.

3.4.6.5 Network Message Cache

In order to reduce unnecessary security checks and excessive relaying, a node shall include a Network Message Cache that stores details of all recently seen Network PDUs. If a Network PDU is received that corresponds to an entry already in the Network Message Cache, then the Network PDU shall not be processed (i.e., it shall be immediately discarded). If a Network PDU is received and that Network PDU has no corresponding entry in the Network Message Cache, then the Network PDU can be processed (e.g., checked against network security), and if it is a valid Network PDU, a corresponding entry for it shall be added in the Network Message Cache.

The node is not required to store the entire Network PDU contents in a cache entry and may store only a part of it for tracking, such as values for the SRC and SEQ fields, or others. However, this is left to the implementation as long as the condition of not processing the same Network PDU more than once is achieved within the limits of the device capabilities.

Because the TTL field value is decremented when a Network PDU is relayed, a node shall not consider the TTL field value when determining whether the Network PDU already has a corresponding cache entry. Also, because the NetMIC field value is derived using the TTL field value, a node shall not consider the NetMIC field value when determining whether the Network PDU already has a corresponding cache entry.

When the Network Message Cache is full and an entry for an incoming new Network PDU needs to be cached, an entry for the incoming new Network PDU shall replace the entry for the oldest Network PDU that is already in the Network Message Cache.

The Network Message Cache shall be able to store entries for at least two Network PDUs, although it is highly recommended to have a Network Message Cache size appropriate to the anticipated network density. The details of the incoming Network PDU processing procedure are left to the implementation.

3.4.6.6 Private Proxy functionality

The Private Proxy functionality is used to relay/forward Network PDUs received by a node between GATT and advertising bearers. This functionality is optional and if supported can be enabled and disabled. When this functionality is supported, the Proxy feature (see Section 3.4.6.2), the Private Network Identity advertising (see Section 7.2.2.2.4), the Mesh Private Beacon Server model (see Section 4.4.11), and Mesh Private beacons (see Section 3.10.4) shall be supported.

3.4.6.7 Node Identity functionality

The Node Identity functionality is used to exchange Network PDUs between two nodes connected via GATT. This functionality is optional. When this functionality is supported, the Mesh Proxy Service (see Section 7.2) shall be exposed and the Advertising with Node Identity (see Section 7.2.2.2.3) shall be supported.

3.4.6.8 Private Node Identity functionality

The Private Node Identity functionality is used to exchange Network PDUs between two nodes connected via GATT. This functionality is optional. When this functionality is supported, the Node Identity functionality (see Section 3.4.6.7), the Mesh Private Beacon Server model (see Section 4.4.11), and the Advertising with Private Node Identity (see Section 7.2.2.2.5) shall be supported.

3.4.6.9 On-Demand Private Proxy functionality

The On-Demand Private Proxy functionality is used to start Advertising with Private Network Identity (see Section 7.2.2.2.4) using the Solicitation PDU (see Section 6.9.1). This functionality is optional. When this functionality is supported, the Private Proxy functionality (see Section 3.4.6.6), On-Demand Private Proxy Server model (see Section 4.4.13), Solicitation PDU RPL Configuration Server model (see Section 4.4.17), and Solicitation PDU with Identification Type set to Proxy Solicitation with Replay Protection (see Section 6.9.1) shall be supported.

3.5 Lower transport layer

The lower transport layer takes an Upper Transport PDU from the upper transport layer and transmits those messages to a peer lower transport layer. These Upper Transport PDUs may fit into a single Lower Transport PDU or they may be segmented into multiple Lower Transport PDUs. Upon receiving messages, the lower transport layer processes Lower Transport PDUs, reassembling Upper Transport PDUs from possibly multiple PDUs and sending these up to the upper transport layer once reassembly is complete.

3.5.1 Endianness

All multiple-octet numeric values in this layer shall be sent in big-endian, as described in Section 3.1.1.1.

3.5.2 Lower Transport PDU

The Lower Transport PDU is used to transmit Upper Transport PDUs to another node.

The most significant bit of the first octet of the Lower Transport PDU is the SEG field, which is used to determine if the Lower Transport PDU is formatted as a segmented or unsegmented message.

There are four formats used, depending on the value of the CTL field in the Network PDU and the SEG field in the Lower Transport PDU as defined in Table 3.15 below.

CTL Field	SEG Field	Lower Transport PDU Format
0	0	Unsegmented Access message
0	1	Segmented Access message
1	0	Unsegmented Control message
1	1	Segmented Control message

Table 3.15: Lower Transport PDU format types

The SEG field values are defined in the Table 3.16.

Value	Description
0	Unsegmented Message
1	Segmented Message

Table 3.16: SEG field values

3.5.2.1 Unsegmented Access message

The Unsegmented Access message is used to transport an Upper Transport Access PDU that fits into a single Network PDU. [Figure 3.11](#) shows an illustration of an Unsegmented Access message, and [Table 3.17](#) shows the fields for this message.

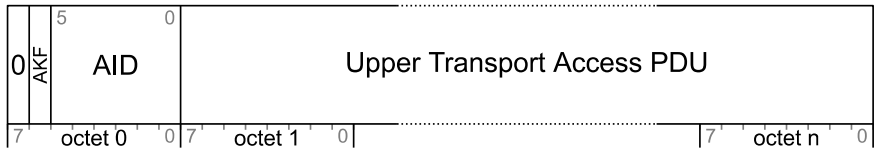


Figure 3.11: Unsegmented Access message

Field	Size (bits)	Description	Req.
SEG	1	Set to Unsegmented Message	M
AKF	1	Application Key Flag	M
AID	6	Application key identifier	M
Upper Transport Access PDU	40 to 120	The Upper Transport Access PDU	M

Table 3.17: Unsegmented Access message format

The SEG field shall be set to Unsegmented Message.

The AKF and AID fields shall be set by the upper transport layer according to the application key or device key used to encrypt the Access message (see [Section 3.6.4.1](#)).

The Upper Transport Access PDU is supplied by the upper transport layer.

This message does not have a SZMIC field. The TransMIC field in the upper transport layer shall be a 32-bit field, as if the SZMIC field has the value 0.

3.5.2.2 Segmented Access message

The Segmented Access message is used to transport a complete Upper Transport Access PDU or a segment of an Upper Transport Access PDU. [Figure 3.12](#) shows an illustration of a Segmented Access message, and [Table 3.18](#) shows the fields for this message.

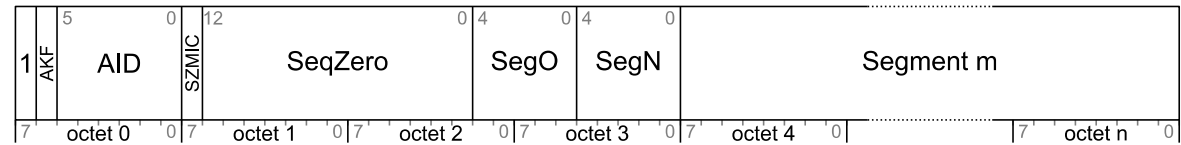


Figure 3.12: Segmented Access message

Field	Size (bits)	Description	Req.
SEG	1	Set to Segmented Message	M
AKF	1	Application Key Flag	M
AID	6	Application key identifier	M
SZMIC	1	Size of the TransMIC field	M
SeqZero	13	Least significant bits of SeqAuth	M
SegO	5	Segment Offset number	M
SegN	5	Last Segment number	M
Segment m	8 to 96	Segment m of the Upper Transport Access PDU	M

Table 3.18: Segmented Access message format

The SEG field shall be set to Segmented Message.

The SZMIC field indicates the size of the TransMIC field in the Upper Transport Access PDU. If the SZMIC field is set to 0, the TransMIC is a 32-bit field. If the SZMIC field is set to 1, the TransMIC is a 64-bit field.

The AKF and AID fields shall be set by the upper transport layer according to the application key or device key used to encrypt the Access message (see Section 3.6.4.1).

The SeqZero field shall be set by the upper transport layer.

The SegO field shall be set to the segment number (zero-based) of the segment m of this Upper Transport PDU.

The SegN field shall be set to the last segment number (zero-based) of this Upper Transport PDU.

The Segment m field, with the segment number m, shall be set to the subset of octets from the Upper Transport Access PDU. For all segments except the last segment, Segment m is octet $12*m$ to $12*m+11$. In the last segment, Segment m is octet $12*m$ through the end of the message.

Every Segmented Access message for the same Upper Transport Access PDU shall have the same values for AKF, AID, SZMIC, SeqZero, and SegN fields.

3.5.2.3 Unsegmented Control message

The Unsegmented Control message is used to transport either a Segment Acknowledgment message or an Upper Transport Control PDU. Figure 3.13 shows an illustration of an Unsegmented Control message, and Table 3.19 shows the fields for this message.

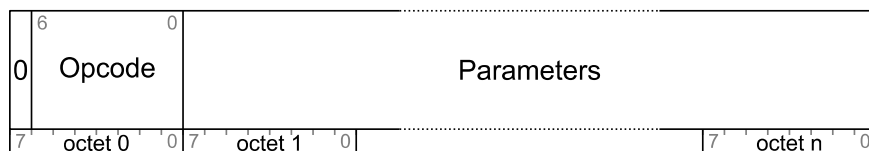


Figure 3.13: Unsegmented Control message

Field	Size (bits)	Description	Req.
SEG	1	Set to Unsegmented Message	M
Opcode	7	Set to Opcode of the Upper Transport Control PDU	M
Parameters	0 to 88	Parameters for the Upper Transport Control PDU	M

Table 3.19: Unsegmented Control message format

The SEG field shall be set to Unsegmented Message.

The Opcode field values are defined in [Table 3.20](#) and shall be set to the appropriate opcode defined in the Assigned Numbers document [4].

Values	Description
0x00	Reserved
0x01 - 0x7F	Opcode of the Upper Transport Control PDU

Table 3.20: Opcode field of the Unsegmented Control message values

The Parameters field is set according to the requirements of the opcode.

3.5.2.3.1 Segment Acknowledgment message

The Segment Acknowledgment message is used by the lower transport layer to acknowledge segments received by a peer lower transport layer. The Segment Acknowledgment message is illustrated in [Figure 3.14](#) and defined in [Table 3.21](#).

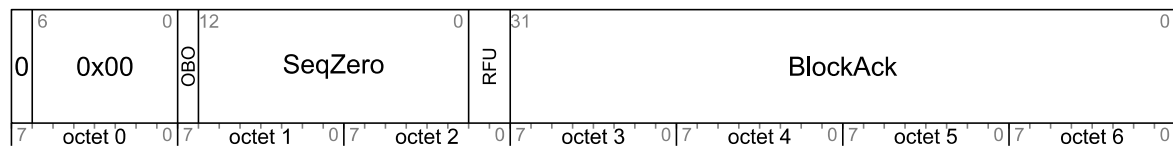


Figure 3.14: Segment Acknowledgment message

Field	Size (bits)	Description	Req.
SEG	1	Set to Unsegmented Message	M
Opcode	7	Set to 0x00	M
OBO	1	Friend on behalf of a Low Power node	M
SeqZero	13	SeqZero of the Upper Transport PDU	M
RFU	2	Reserved for Future Use	M
AckedSegments	32	Acknowledgment for segments	M

Table 3.21: Segment Acknowledgment message format

The SEG field shall be set to Unsegmented Message.

The Opcode field shall be set to 0x00.

The OBO field shall be set to 0 by a node that is directly addressed by the received message and shall be set to 1 by a Friend node that is acknowledging this message on behalf of a Low Power node.

The SeqZero field shall be set to the value of the SeqZero field of the upper transport layer message being acknowledged.

The AckedSegments field shall be set to indicate the segments received. The least significant bit, bit 0, shall represent segment 0; and the most significant bit, bit 31, shall represent segment 31. If bit *n* is set to 1, then segment *n* is being acknowledged. If bit *n* is set to 0, then segment *n* is not being acknowledged. Any bits for segments larger than the SegN field value of the upper transport layer message being acknowledged shall be set to 0 and ignored upon receipt.

If the received segments were sent with the TTL field set to 0, it is recommended that the corresponding Segment Acknowledgment message is sent with the TTL field set to 0.

3.5.2.4 Segmented Control message

The Segmented Control message is used to transport a complete Upper Transport Control PDU or a segment of an Upper Transport Control PDU when the Upper Transport Control PDU will not fit into a single Network PDU. [Figure 3.15](#) shows an illustration of a Segmented Control message, and [Table 3.22](#) shows the fields for this message.

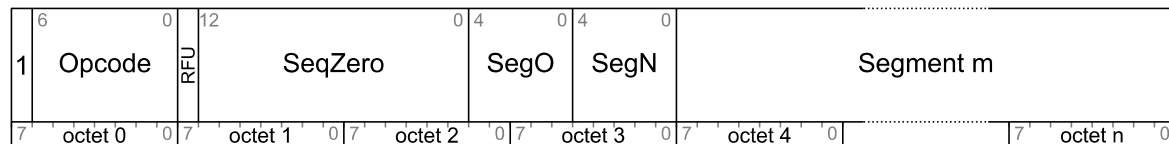


Figure 3.15: Segmented Control message

Field	Size (bits)	Description	Req.
SEG	1	Set to Segmented Message	M
Opcode	7	Opcode of the Upper Transport Control PDU	M
RFU	1	Reserved for Future Use	M
SeqZero	13	Least significant bits of SeqAuth	M
SegO	5	Segment Offset number	M
SegN	5	Last Segment number	M
Segment m	8 to 64	Segment m of the Upper Transport Control PDU	M

Table 3.22: Segmented Control message format

The SEG field shall be set to Segmented Message.

The Opcode field shall be set by the upper transport layer as defined in [Table 3.20](#) to indicate the format of the Parameters field. The value 0x00 is Reserved and shall not be transmitted and ignored upon receipt.

The SeqZero field shall be set by the upper transport layer.

The SegO field shall be set to the segment number (zero-based) of the Upper Transport PDU contained within this message.

The SegN field shall be set to the last segment number (zero-based) of this Upper Transport PDU.

The Segment m field shall be set to the subset of octets from the Upper Transport Control PDU. Segment m is octet $8*m$ to $8*m+7$, except for the last segment where it is octet $8*m$ to the end of the message.

Every Segmented Control message for the same Upper Transport Control PDU shall have the same values for the Opcode, SeqZero, and SegN fields.

3.5.3 Segmentation and reassembly

To transmit Upper Transport Access PDUs larger than 15 octets, or shorter Upper Transport Access PDUs to be sent as single segment, or Upper Transport Control PDUs larger than 11 octets, or shorter Upper Transport Control PDUs to be sent as single segment, the lower transport layer segments and reassembles Upper Transport PDUs. These segments are delivered to the peer lower transport layer using a block acknowledgment scheme to minimize the number of messages that need to be transmitted by the lower transport layer.

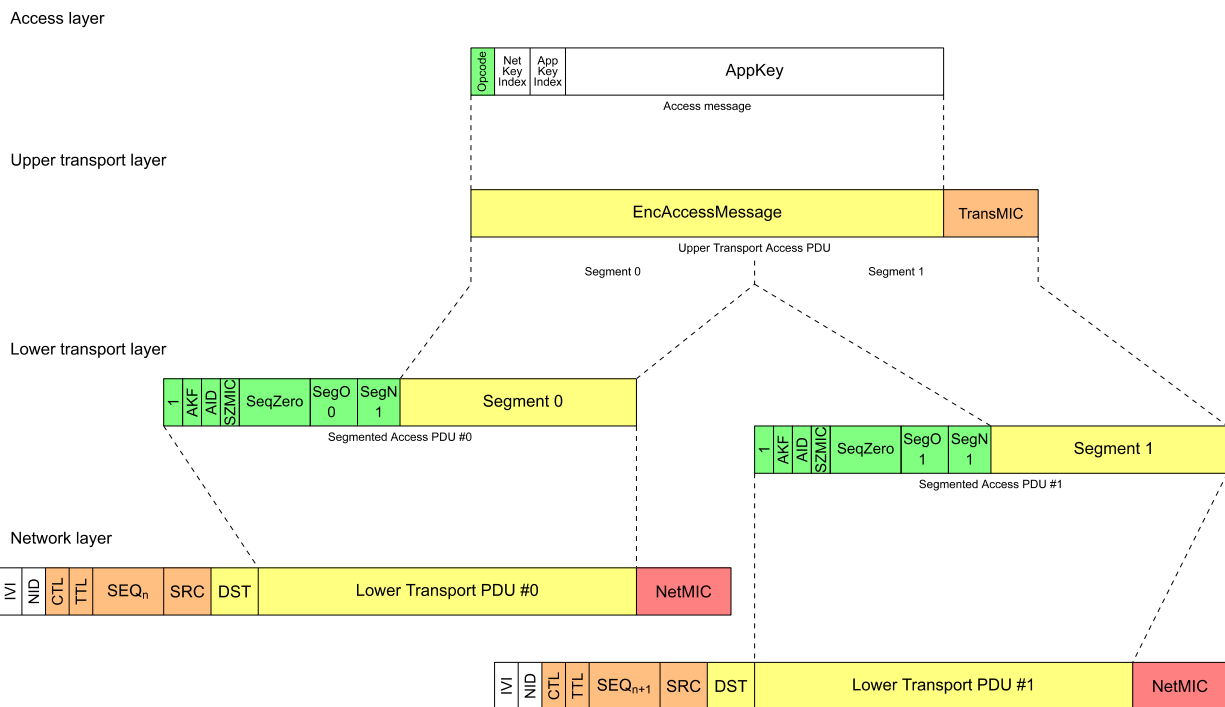


Figure 3.16: Example of segmentation and reassembly for a two-segment PDU

Figure 3.16 illustrates an Access message being sent that has a single-octet opcode, 3 octets for the NetKeyIndexAndAppKeyIndex field, and 16 octets for the AppKey. This means that when encrypted and authenticated with an application key, the Upper Transport Access PDU is 24 octets. This is segmented by the lower transport layer into two segments, Segment 0 and Segment 1. Each segment has a header that identifies the segment number and is then passed to the network layer, where the complete Network PDU is computed. The network layer then encrypts the Network PDU using the sequence number for that

Network PDU and then obfuscates those messages so that only the NID and IVI fields are visible in clear text. Therefore, the single Access message can be delivered securely using two Network PDUs.

The process of segmentation for Upper Transport Access PDUs and Upper Transport Control PDUs is identical, and the description below considers these two PDU types to be identical except where explicitly stated.

Note: The segment sizes are different for Upper Transport Access PDUs and Upper Transport Control PDUs.

3.5.3.1 Segmentation

Segmentation is performed by the lower transport layer of the transmitting node. The lower transport layer checks if an Upper Transport PDU fits into a single Lower Transport PDU. If the Upper Transport PDU fits, it is sent in a single Lower Transport PDU. If the Upper Transport PDU doesn't fit, the lower transport layer divides the Upper Transport PDU into two or more Lower Transport PDUs.

Delivery of a segmented message is acknowledged by the lower transport layer of the receiving node. Delivery of an unsegmented message is not acknowledged. An Upper Transport PDU that fits into one Lower Transport PDU can be sent as a single-segment segmented message when acknowledgment by the lower transport layer is required.

Example: Using a single-segment segmented message can decrease the air traffic, for example, in a situation when a long multi-segment message (e.g., an Upper Transport PDU which was divided into many Lower Transport PDUs) has been transmitted, but the application acknowledgment message sent as a response to this multi-segment message was lost. Sending the application acknowledged message as a single segmented message can improve the reliability of delivery and can remove the risk associated with retransmitting the whole, long multi-segment message.

Each segment of an Upper Transport Access PDU shall be 12 octets long with the exception of the last segment, which may be shorter.

Each segment of an Upper Transport Control PDU shall be 8 octets long with the exception of the last segment, which may be shorter.

Example: When using a 32-bit TransMIC field, if an Upper Transport Access PDU is 42 octets long, then the first 12 octets, octets 0 to 11, are in segment 0; the second set of 12 octets, octets 12 to 23, are in segment 1; the third set of 12 octets, octets 24 to 35, are in segment 2; and the remaining 6 octets, octets 36 to 41, are in segment 3.

Example: If an Upper Transport Control PDU is 42 octets long, then the first 8 octets, octets 0 to 7, are in segment 0; the second set of 8 octets, octets 8 to 15, are in segment 1; the third set of 8 octets, octets 16 to 23, are in segment 2; the fourth set of 8 octets, octets 24 to 31, are in segment 3; the fifth set of 8 octets, octets 32 to 39, are in segment 4; and the remaining 2 octets, octets 40 to 41, are in segment 5.

Each segment of an Upper Transport PDU is identified by the SegO field value. The total number of segments is identified by the SegN field value. The SegO field value of the first segment equals 0. The SegO field value of the last segment equals the SegN field value.

Lower Transport PDUs derived from the same Upper Transport PDU are linked together by the common SeqAuth value. The SeqAuth value is composed of the IV Index and the sequence number of the first segment. The size of the SeqAuth value is 7 octets, where the IV Index is the four most significant octets and the sequence number is the three least significant octets. All Lower Transport PDUs from the same Upper Transport PDU shall be sent using the same IV Index from the common SeqAuth.

The least significant 13 bits of the SeqAuth value constitute the SeqZero field value. The SeqZero field is included in the segmented message and Segment Acknowledgment message to identify the Upper Transport PDU. Upon reassembling a complete Upper Transport PDU from the segments, the SeqAuth value can be retrieved from the IV Index, SeqZero, and SEQ fields included in any of the segments. The common SeqAuth value is the largest SeqAuth value for which the SeqZero field value is between SEQ – 8191 and SEQ inclusive, and the IV Index is the same.

Example: If the SEQ field value of a received message was 0x647262, the IV Index was 0x58437AF2, and the received SeqZero field value was 0x1849, then the SeqAuth value is 0x58437AF2645849.

Example: If the SEQ field value of a received message was 0x647262 and the received SeqZero field value was 0x1263, then the SeqAuth value is 0x58437AF2645263. For an Upper Transport PDU, the SeqAuth value is used to identify it.

Because of the limited size of the SeqZero field, it is not possible to send a segmented message when the SEQ field value is 8192 greater than the SeqAuth value. If a segmented message has not been acknowledged by the time that the SEQ field value is 8192 greater than the SeqAuth value, then the transmission of the Upper Transport PDU shall be canceled. Lower Transport PDUs are delivered to the network layer.

3.5.3.2 Reassembly

Reassembly is performed by the lower layer of the receiving node. When the Low Power node feature is in use, reassembly is performed by a Friend node and the Low Power node does not send any Segment Acknowledgment messages.

Upon receiving a segment from a Segmented message, the lower transport layer retrieves the SeqAuth value from the IV Index, SeqZero, and SEQ fields of this segment to check if the Upper Transport PDU has already been received.

The lower transport layer stores upcoming segments to complete the whole Upper Transport PDU. The value of the SegO field is used to determine the offset of the Lower Transport PDU within the Upper Transport PDU. The maximum size of the complete Upper Transport PDU is determined by the value of the SegN field and the size of the segment.

The lower transport layer sends Segment Acknowledgment messages to either report missing segments or report complete delivery of the Upper Transport PDU.

When all segments of the Upper Transport PDU for a given SeqAuth have been received, the Upper Transport PDU is delivered to the upper transport layer.

3.5.3.3 Segmentation behavior

3.5.3.3.1 *Transmission of segments*

The lower transport layer shall not transmit segmented messages for more than one Upper Transport PDU to the same destination at the same time. The lower transport layer should start to transmit segmented messages for a new Upper Transport PDU for the same destination when the transaction for the last Upper Transport PDU is completed or the message transmission has been canceled.

If the Upper Transport PDU can fit into a single Lower Transport PDU using an Unsegmented Message format and has not been tagged with the send-segmented tag, then the lower transport layer should use an unsegmented message to transmit this Upper Transport PDU.

If the Upper Transport PDU can fit into a single Lower Transport PDU using a Segmented Message format and has not been tagged with the send-segmented tag, then the lower transport layer may use a single segmented message to transmit this Upper Transport PDU.

If the Upper Transport PDU can fit into a single Lower Transport PDU using a Segmented Message format and has been tagged with the send-segmented tag, then the lower transport layer shall use a single segmented message to transmit this Upper Transport PDU.

Otherwise, two or more segmented messages shall be used.

For each transmission to a unicast address, the lower transport layer stores the destination address, the derived SeqAuth of the segmented message, the remaining number of retransmissions value, and the remaining number of retransmissions without progress value. The initial value of the retransmissions for a unicast address is the value indicated by the SAR Unicast Retransmissions Count state (see Section 4.2.48.2). The initial value of the retransmissions without progress is the value indicated by the SAR Unicast Retransmissions Without Progress Count state (see Section 4.2.48.3).

For each transmission to a group address or a virtual address, the lower transport layer stores the destination address, the derived SeqAuth of the segmented message, and the remaining number of retransmissions value. The initial value of the number of retransmissions for a group or a virtual address is the value indicated by the SAR Multicast Retransmissions Count state (see Section 4.2.48.6).

When the lower transport layer starts a new transfer of an Upper Transport PDU, it shall divide the Upper Transport PDU into segments, shall mark all of the segments as unacknowledged, and shall start transmitting the segments. Segments shall be sent in order of SegO field value, starting with the segment with the lowest value in the SegO field.

Transmission of segments shall be separated by the segment transmission interval indicated by the value of the SAR Segment Interval Step state (see Section 4.2.48.1).

When the lower transport layer starts a new transfer of an Upper Transport PDU that is destined to a unicast address, the lower transport layer shall set the remaining number of retransmissions to the initial value and shall set the remaining number of retransmissions without progress to the initial value. The lower transport layer expects a Segment Acknowledgment message from the destination, or from a Friend node on behalf of the destination.

When the lower transport layer starts a new transfer of an Upper Transport PDU that is destined to a group address or a virtual address, the lower transport layer shall set the remaining number of retransmissions to the initial value. Segment Acknowledgment messages are not sent by the destination.

When the last segment marked as unacknowledged is transmitted and the destination is a unicast address, the lower transport layer shall start a SAR Unicast Retransmissions timer. If the value of the TTL field of the message is greater than 0, the initial value of the timer shall be set according to the following formula:

$$[\text{unicast retransmissions interval step} + \text{unicast retransmissions interval increment} * (\text{TTL} - 1)]$$

If the value of the TTL field of the message is 0, the initial value of the timer shall be set to the unicast retransmissions interval step.

The values of the unicast retransmissions interval step and the unicast retransmissions interval increment are indicated by the SAR Unicast Retransmissions Interval Step state (see Section 4.2.48.4) and the SAR Unicast Retransmissions Interval Increment state (see Section 4.2.48.5), respectively.

When the last segment marked as unacknowledged is transmitted, and the destination is a group address or a virtual address, the lower transport layer shall start a SAR Multicast Retransmissions timer with the initial value set to the multicast retransmissions interval. The multicast retransmissions interval is indicated by the SAR Multicast Retransmissions Interval Step state (see Section 4.2.48.7).

3.5.3.3.2 Reception of Segment Acknowledgment messages

When a Segment Acknowledgment message that is a valid acknowledgment (i.e., it meets all conditions in the Table 3.23) for the segmented message is received, then the lower transport layer shall mark as acknowledged the segments that are reported as delivered in the AckedSegments field of the Segment Acknowledgment message (see Section 3.5.2.3.1).

When a valid Segment Acknowledgment message for the segmented message is received (i.e., it meets all conditions in Table 3.23), and the SAR Unicast Retransmissions timer is running, and the Segment Acknowledgment message does not acknowledge all segments of the segmented message, and both the remaining number of retransmissions and the remaining number of retransmissions without progress are greater than 0, then the lower transport layer shall stop the SAR Unicast Retransmissions timer, shall repeat the transmission of all segments that are marked as unacknowledged, shall decrement the remaining number of retransmissions by 1, and shall start the SAR Unicast Retransmissions timer. If at least one segment is newly marked as acknowledged as a result of receiving the Segment Acknowledgment message, the lower transport layer shall set the remaining number of retransmissions without progress to the initial value. If no segment is newly marked as acknowledged, the lower transport layer shall decrement the remaining number of retransmissions without progress by 1.

When a valid Segment Acknowledgment message for the segmented message is received (i.e., it meets all conditions in Table 3.23), and the SAR Unicast Retransmissions timer is running, and the Segment Acknowledgment message acknowledges all segments of the segmented message, the transmission of the Upper Transport PDU is complete. Then the SAR Unicast Retransmissions timer shall be stopped, the number of retransmissions shall be deleted, and the number of retransmissions without progress shall be deleted. The lower transport layer shall remove the destination address and the SeqAuth stored for this segmented message and shall notify the higher layer that the transmission of the Upper Transport PDU has been completed.

When a Segment Acknowledgment message that is a valid acknowledgment for a segmented message with the AckedSegments field set to 0x00000000 is received, then the transmission of the Upper Transport PDU shall be immediately canceled, and the upper transport layer shall be notified that the transmission of the Upper Transport PDU has been canceled. The lower transport layer shall remove the destination address and the SeqAuth stored for this segmented message.

Condition
SeqAuth derived from the SeqZero field of the Segment Acknowledgment message matches the value stored by the lower transport layer
Either the source address of the Segment Acknowledgment message matches the destination address value stored by the lower transport layer, or the value of the OBO field of the Segment Acknowledgment message is 1.
For the SeqAuth derived from the SeqZero field of the message, there is at least one unacknowledged segment that the AckedSegments field of the message reports as delivered
The message was secured using the same NetKey that was used to secure the segmented message

Table 3.23: Conditions to validate a segment acknowledgment message

Note: The reception of a Segment Acknowledgment message with the OBO field set to 1 does not mean that the segmented message has been delivered to the final destination, but only that the segmented message has been delivered to the Friend of that Low Power node. The message is stored in the Friend Queue, but the message can be discarded if other messages are received for that Low Power node or the Friendship is terminated.

3.5.3.3.3 Retransmission of segments

When the SAR Unicast Retransmissions timer expires, if both the remaining number of retransmissions and the remaining number of retransmissions without progress are greater than 0, then the lower transport layer shall repeat the transmission of all segments marked as unacknowledged and shall decrement both the remaining number of retransmissions and the remaining number of retransmissions without progress by 1.

When the last segment marked as unacknowledged is transmitted and the destination is a unicast address, the lower transport layer shall start a SAR Unicast Retransmissions timer. If the value of the TTL field of the message is greater than 0, the initial value of the timer shall be set according to the following formula:

$$[\text{unicast retransmissions interval step} + \text{unicast retransmissions interval increment} * (\text{TTL} - 1)]$$

If the value of the TTL field of the message is 0, the initial value of the timer shall be set to the unicast retransmissions interval step.

The unicast retransmissions interval step value and the unicast retransmissions interval increment value are indicated by the SAR Unicast Retransmissions Interval Step state (see Section 4.2.48.4) and the SAR Unicast Retransmissions Interval Increment state (see Section 4.2.48.5), respectively.

When the SAR Unicast Retransmissions timer expires and either the remaining number of retransmissions or the remaining number of retransmissions without progress is 0, the lower transport layer shall cancel the transmission of the Upper Transport PDU, shall delete the number of

retransmissions value and the number of retransmissions without progress value, shall remove the destination address and the SeqAuth stored for this segmented message, and shall notify the upper transport layer that the transmission of the Upper Transport PDU has timed out.

When the SAR Multicast Retransmissions timer expires and the remaining number of retransmissions value is greater than 0, then the lower transport layer shall repeat the transmission of all the segments of the of the Upper Transport PDU. The lower transport layer shall decrement the remaining number of retransmissions value by 1.

When the last segment marked as unacknowledged is transmitted, and the destination is a multicast address, the lower transport layer shall start a SAR Multicast Retransmissions timer with the initial value set to the multicast retransmissions interval. The multicast retransmissions interval is indicated by the SAR Multicast Retransmissions Interval Step state (see Section 4.2.48.7).

When the SAR Multicast Retransmissions timer expires and the remaining number of retransmissions value is 0, the lower transport layer shall cancel the transmission of the Upper Transport PDU, shall delete the number of retransmissions value and the number of retransmissions without progress value, shall remove the destination address stored for this segmented message, and shall notify the higher layer that the transmission of the Upper Transport PDU has been completed.

If an Upper Transport PDU is tagged with additional metadata (see Sections 3.6.5, 3.6.8.2, and 3.7.3.1) and is segmented, each Lower Transport PDU of the segmented Upper Transport PDU shall be tagged with the same metadata.

3.5.3.4 Reassembly behavior

This section only applies when the Low Power feature is not in use.

The lower transport layer stores one or more pairs of values, consisting of an AckedSegments value, which indicates segments that have already been received for a particular SeqAuth, and a Sequence Authentication value. Each such pair is associated with a source address and a destination address. The initial value for the AckedSegments is a value that indicates that no segments have been received. The initial value for the Sequence Authentication value is 0.

When the lower transport layer receives a segment of a segmented message, it shall process the segment message against the conditions in Table 3.24. Conditions are evaluated one by one starting from the first line in the table. When the condition is met, the processing ends with the Processing Result corresponding to the value in the Condition column.

When the Processing Result is Message Rejected and the message is destined to a unicast address, the lower transport layer shall respond with a Segment Acknowledgment message with the AckedSegments field set to 0x00000000.

When the Processing Result is First Segment and the destination is a group address or a virtual address, the reassembly of a new segmented message is initiated. If another reassembly is pending for the same source address and for the same destination address, the pending reassembly shall be discarded and the SAR Discard timer shall be stopped. The lower transport layer shall set the Sequence Authentication value to the SeqAuth derived from this message. Then, the lower transport layer shall set the AckedSegments value for this SeqAuth to indicate that only this segment has been received and shall

start the SAR Discard timer for this SeqAuth from the initial value. The initial value of the SAR Discard timer is the discard timeout value indicated by the SAR Discard Timeout state (see Section 4.2.49.4).

When the Processing Result is First Segment and the destination is a unicast address, the reception of a new segmented message is initiated. If another reassembly is already pending for the same source address and for the same destination address, the pending reassembly shall be discarded and the SAR Discard timer and SAR Acknowledgment timer shall be stopped. The lower transport layer shall set the Sequence Authentication value to the SeqAuth derived from this message. The lower transport layer shall set the AackedSegments value for this SeqAuth to indicate that only this segment has been received and shall start the SAR Discard timer and SAR Acknowledgment timer for this SeqAuth from the initial values. The initial value of the SAR Discard timer is the discard timeout value indicated by the SAR Discard Timeout state (see Section 4.2.49.4). The initial value of the SAR Acknowledgment timer is calculated using the following formula:

$$[\min(\text{SegN} + 0.5, \text{acknowledgment delay increment}) * \text{segment reception interval}]$$

The acknowledgment delay increment value and the segment reception interval value are indicated by the SAR Acknowledgment Delay Increment state (see Section 4.2.49.2) and the SAR Receiver Segment Interval Step state (see Section 4.2.49.5), respectively.

When the Processing Result is Next Segment and the destination is a group address or a virtual address, the lower transport layer shall store the received segment, shall start the SAR Discard timer from the initial value, and shall set the AackedSegments value for the SeqAuth derived from this message to indicate that the segment identified by the SegO field has been received.

When the Processing Result is Next Segment and the destination is a unicast address, the lower transport layer shall store the received segment, shall start the SAR Acknowledgment timer and the SAR Discard timer from the initial values, and shall set the AackedSegments value for the SeqAuth derived from this message to indicate that the segment identified by the SegO field has been received.

When the Processing Status is SeqAuth Error or Repeated Segment, the lower transport layer shall ignore the message.

When the Processing Result is Most Recent SeqAuth, the lower transport layer shall send a Segment Acknowledgment message with the AackedSegments field set to indicate that all segments have already been delivered for that SeqAuth. The lower transport layer shall not send more than one Segment Acknowledgment message for the same SeqAuth in a period of:

$$[\text{acknowledgment delay increment} * \text{segment reception interval}] \text{ milliseconds}$$

The acknowledgment delay increment and the segment reception interval are indicated by the SAR Acknowledgment Retransmissions Interval state and the SAR Receiver Segment Interval Step state (see Section 4.2.49.5), respectively.

When the Processing Result is Last Segment, the transfer of the segmented message is complete. The lower transport layer shall send a Segment Acknowledgment message with the AackedSegments field set to indicate that all segments have been delivered for the Sequence Authentication value. The Segment Acknowledgment message shall use the same NetKey as the first received segment of the segmented message, and the DST field shall have the same value as the SRC field of the first received segment of the segmented message. The lower transport layer shall stop the SAR Discard timer and the SAR

Acknowledgment timer and shall send the reassembled message to the upper transport layer. The lower transport layer shall store the SeqAuth and the AackedSegments value for complete transactions to be able to acknowledge a transaction when repeated segments are received. The values for the most recent transaction shall be stored, and an implementation may store the values for other recent transactions. The number of additional transactions for which values are stored is an implementation detail.

If the segments are Segmented Access messages, then the reassembled message shall be processed as defined in Section 3.6.4.2.

If the segments are Segmented Control messages, then the reassembled message shall be processed as defined in Section 3.6.5.

Condition	Processing Result
The SeqAuth calculated for the message is less than the Sequence Authentication value and the segmented message has not been received	SeqAuth Error
The SeqAuth calculated for the message is less than the Sequence Authentication value and the whole message has already been received	Repeated Segment
The SeqAuth calculated for the message is equal to the Sequence Authentication value and the whole message has already been received	Most Recent SeqAuth
The lower transport layer cannot accept the segment message because it is currently out of resources	Message Rejected
The SeqAuth value calculated for the segment message is greater than Sequence Authentication value	First Segment
The AackedSegments value for the SeqAuth calculated for the message indicates that the segment has already been received	Repeated Segment
The SAR Discard timer is expired and the reassembly for this SeqAuth is considered failed	Late Segment
The received segment message is not the first, nor the last missing segment for the SeqAuth	Next Segment
The received segment message is the last missing segment for the SeqAuth	Last Segment

Table 3.24: Conditions for segment message processing

When the SAR Acknowledgment timer expires, the lower transport layer shall send a Segment Acknowledgment message with the AackedSegments field set to the AackedSegments value for the identified SeqAuth to indicate the segments have been delivered (see Section 3.5.2.3.1). The Segment Acknowledgment message shall use the same NetKey as the first received segment of the segmented message, and the DST field shall have the same value as the SRC field of the first received segment of the segmented message.

If the number of segments in the transmission indicated by the value of SegN field is greater than the value of the SAR Segments Threshold state (see Section 4.2.49.1), the lower transport layer shall retransmit Segment Acknowledgment messages using the value of the SAR Acknowledgment Retransmissions Count state (see Section 4.2.49.3). Each retransmitted message shall include a new value for the SEQ field. Between retransmissions, the lower transport layer shall introduce a delay indicated by the value of the SAR Receiver Segment Interval Step state (see Section 4.2.49.5).

When the SAR Discard timer expires, the reassembly of the Upper Transport PDU being received is considered failed. For the SeqAuth derived from this message, the lower transport layer shall stop the SAR Acknowledgment timer, stop the SAR Discard timer, remove the AckedSegments value and discard all stored segments.

The Segment Acknowledgment message shall use the same NetKey as the first received segment of a multi-segment message, and the DST field shall have the same value as the SRC field of the first received segment of a multi-segment message.

If the device is acting as a Friend node for a Low Power node, then it shall reassemble segmented messages destined for the Low Power node and act as described, except that it shall set the OBO field to 1 in the Segment Acknowledgment message. Otherwise, the OBO field shall be set to 0.

3.5.3.5 Low Power feature reassembly behavior

This section only applies when the Low Power feature is in use.

For each source address, the lower transport layer stores an AckedSegments value which indicates segments that have already been received for a particular SeqAuth and a Sequence Authentication value. The initial value for the AckedSegments is a value that shall indicate that no segments have been received. The initial value for the Sequence Authentication is 0.

When the lower transport layer receives a segment of a segmented message, it shall process the segment against the conditions in [Table 3.24](#). Conditions are evaluated one by one starting from the first line in the table. When the condition is met, the processing ends with the Processing Result corresponding to the value in the Condition column.

When the Processing Result is First Segment, the reassembly of a new segmented message is initiated. If another reassembly is already pending for the same source address, the pending reassembly shall be discarded. Then, the lower transport layer shall set the Sequence Authentication value to the SeqAuth derived from this message and for this SeqAuth shall set the AckedSegments value to indicate that only this segment has been received.

When the Processing Result is Next Segment, the lower transport layer shall store the received segment and set the AckedSegments value for the SeqAuth derived from this message to indicate that this segment has been received.

When the Processing Status is SeqAuth Error, Repeated Segment, or Most Recent SeqAuth, the lower transport layer shall ignore the message.

When the Processing Result is Last Segment, the transfer of the segmented message is complete. The lower transport layer shall discard the AckedSegments value and shall send the reassembled message to the upper transport layer.

If the segments are Segmented Access messages, then the reassembled message shall be processed as defined in [Section 3.6.4.2](#).

If the segments are Segmented Control messages, then the reassembled message shall be processed as defined in [Section 3.6.5](#).

If the friendship is terminated (see Section 3.6.6.4.2), then any previous partially received multi-segment message shall be cancelled.

3.5.4 Lower transport layer behavior

3.5.4.1 Transmitting a Lower Transport PDU

The Lower Transport PDU shall be delivered to the network layer for processing (see Section 3.4.6.4). TransportPDU field of the Network PDU shall be set to the Lower Transport PDU value.

3.5.4.2 Receiving a Lower Transport PDU

If the Lower Transport PDU is a Segmented Access message, a Segmented Control message or a Segment Acknowledgment message, then it shall be processed as defined in Section 3.5.3.2.

If the Lower Transport PDU is an Unsegmented Access message or an Unsegmented Control message, then the Upper Transport PDU shall be processed as defined in Section 3.6.4.2.

3.5.4.3 Message error procedure

When the lower transport layer receives a Segment Acknowledgment message that is not understood, then the Segment Acknowledgment message shall be ignored. A Segment Acknowledgment message that is not understood includes messages that have incorrect size.

3.5.5 Friend Queue

The Friend node shall have a Friend Queue for each friend Low Power node. The Friend Queue stores Lower Transport PDUs for a Low Power node. No field of the Lower Transport PDU shall be changed due to the message being in the Friend Queue. The CTL, TTL, SEQ, SRC, and DST fields shall be stored with the associated Lower Transport PDU.

When a Friend node receives a message on a subnet that was used for friendship establishment and that is destined for a Low Power node (i.e., the destination of the message is a unicast address of an element of the Low Power node or in the Friend Subscription List), and the TTL field has a value of 2 or greater, then the message shall be processed as follows: If the Friend Queue already contains a message with the same SEQ and SRC field values as in the received message, or if the SRC field of the received message is a unicast address of an element of the Low Power node, then the message shall not be stored in the Friend Queue. Otherwise, the TTL field value shall be decremented by 1, and the message shall be stored in the Friend Queue.

If the message is a Segmented Access message or a Segmented Control message, then the message shall only be stored into the Friend Queue after the complete Upper Transport PDU has been successfully reassembled and the Friend node has acknowledged the reception of all segments.

If the Friend Queue is full and a new message needs to be stored that is not a Friend Update message, the oldest entries other than a Friend Update message shall be discarded to make room for the new message.

Note: An implementation may have to discard multiple messages to fit the new message into the Friend Queue.

If the message that is being stored is a Segment Acknowledgment message and the Friend Queue contains another Segment Acknowledgment message that has the same source and destination addresses, and the same SeqAuth value, but a lower IV Index or sequence number, then the older Segment Acknowledgment message shall be discarded.

When a Friend node becomes aware of a security update, for example by receiving a valid Secure Network beacon or a Mesh Private beacon, or as a result of a change in the Key Refresh Phase state, the Friend node shall add a Friend Update message to the Friend Queue.

When the Low Power node requests a message from the Friend Queue, the oldest entry shall be sent. Once that message has been acknowledged by the Low Power node, that entry shall be discarded.

If the Friend node is polled for a message from a Low Power node using a Friend Poll, and the Friend Queue for that node is empty, then the Friend node shall generate a new Friend Update message and add that message to the Friend Queue before sending the response, so that this Friend Update message can be sent in response to the Friend Poll message.

3.6 Upper transport layer

The upper transport layer takes an Access message from the access layer or an internally generated Transport Control message and transmits the message to a peer upper transport layer.

The Upper Transport Access PDU contains a message from the access layer. The encryption and authentication of the message is performed using an application key or device key. This allows the receiving upper transport layer to authenticate received messages.

The Upper Transport Control PDU contains a message that is internally generated by the upper transport layer. The message is only encrypted and authenticated at the network layer.

The Upper Transport Access PDU and the Upper Transport Control PDU are collectively known as Upper Transport PDUs.

3.6.1 Endianness

All multiple-octet numeric values in this layer shall be sent in big-endian, as described in Section 3.1.1.1.

3.6.2 Upper Transport Access PDU

The Upper Transport PDU is called the Upper Transport Access PDU, when the CTL field in the Network PDU is 0. The Upper Transport Access PDU contains an Access message.

The Access message is encrypted using an application key or device key, and the encrypted Access message and associated message integrity check value are combined into an Upper Transport Access PDU. The Upper Transport Access PDU fields are shown in Table 3.25 and illustrated in Figure 3.17.

Field Name	Octets	Description	Req.
EncAccessMessage	1 to 380	The encrypted Access message	M
TransMIC	4 or 8	The message integrity check value for the Access message	M

Table 3.25: Upper Transport Access PDU fields

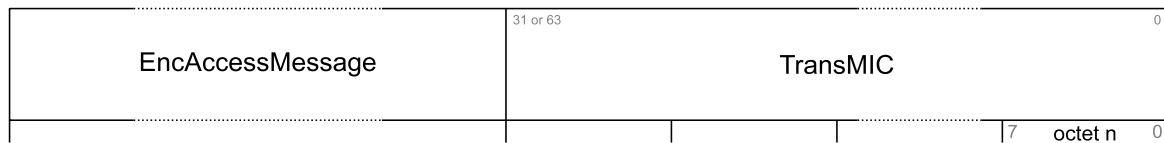


Figure 3.17: Upper Transport Access PDU format

3.6.2.1 EncAccessMessage

The Access message is supplied by the access layer. The EncAccessMessage is an Access message encrypted and authenticated as defined in Section 3.9.7.1. If the TransMIC is a 32-bit field, the Access message can be from a single octet to 380 octets in length. If the TransMIC is a 64-bit field, the Access message can be from a single octet to 376 octets in length. At the upper transport layer, this field is opaque and no information within this field may be used.

3.6.2.2 TransMIC

The Message Integrity Check for Transport (TransMIC) is a 32-bit or 64-bit field that authenticates that the Access message has not been changed. For a Segmented Access message, where the SEG field is set to 1, the size of the TransMIC field is determined by the value of the SZMIC field in the Lower Transport PDU. For Unsegmented Access messages, the TransMIC field is a 32-bit field.

Note: Transport Control messages do not have a TransMIC field.

3.6.3 Upper Transport Control PDU

The Upper Transport PDU is called the Upper Transport Control PDU when the CTL field in the Network PDU is 1. The Upper Transport Control PDU contains a Transport Control message (see Section 3.6.5). A Transport Control message has a 7-bit opcode that determines the format of the parameters. This Opcode field is not included in the parameters field, but is included in the Unsegmented Control message or in each Segmented Control message.

The Upper Transport Control PDU is not authenticated at the upper transport layer and instead relies upon the authentication performed by the network layer. All Upper Transport Control PDUs use a 64-bit NetMIC field.

The lower transport layer may segment messages into smaller PDUs for delivery over the network layer. It is therefore recommended to keep Transport Control PDU payload size as reflected in Table 3.26, where the values represent the maximum useful parameter field sizes depending on the number of packets.

Number of Packets	Transport Control PDU Payload Size
1	11 (Unsegmented)
1	8 (Segmented)
2	16
3	24
n	n*8
32	256

Table 3.26: Maximum Useful Transport Control PDU payload sizes

The maximum size of an Upper Transport Control PDU is 256 octets.

3.6.4 Upper transport layer behavior

3.6.4.1 Transmitting an Upper Transport PDU

All Access messages are sent in the context of an application key or a device key. The Access message shall be encrypted using this application key or device key, and the TransMIC field shall be set to the message integrity check value, as defined in Section 3.9.6. The Upper Transport Access PDU shall then be delivered to the lower transport layer for processing as defined in Section 3.5.3.3.

A sequence number shall be allocated to this message. In the context of a message segmented in the lower transport layer, this sequence number corresponds to the 24 lowest bits of SeqAuth, the sequence number used for authenticating and decrypting the Access message by the receiver, as defined in Section 3.5.3.1.

The AKF and AID fields of the Lower Transport PDU shall be set according to the application key or device key used to encrypt and authenticate the Upper Transport Access PDU. If an application key is used, then the AKF field shall be set to 1 and the AID field shall be set to the AID. If the device key is used, then the AKF field shall be set to 0 and the AID field shall be set to 0b000000.

An Upper Transport Control PDU shall be delivered to the lower transport layer for processing as defined in Section 3.5.3.3.

The upper transport layer shall not transmit a new segmented Upper Transport PDU to a given destination until the previous Upper Transport PDU to that destination has been either completed or cancelled.

3.6.4.2 Receiving an Upper Transport PDU

Upon receiving an Upper Transport Access PDU, the Access message shall be decrypted, and the TransMIC field shall be authenticated against the device key or the Device Key Candidate (see Section 3.11.8.1) or the known application keys for which the AKF and AID fields match. If the Upper Transport Access PDU authenticates and it has been checked for replay attacks (see Section 3.9.8) then it is delivered to the access layer with the contextual information of this message such as the source address, destination addresses, and the keys used for decryption and authentication.

When the Device Key Candidate is available, and an Access message is decrypted using the Device Key Candidate that was delivered to the access layer, then the node shall revoke the device key, the Device Key Candidate shall become the device key, and the Device Key Candidate shall become unavailable.

Upon receiving an Upper Transport Access PDU, this PDU shall be delivered to the Access layer for processing (see Section 3.7.3.2).

Upon receiving an Upper Transport Control PDU, the destination address of the PDU shall be checked. The PDU shall be processed according to the Transport Control opcode (as defined in Sections 3.6.6, 3.6.7, and 3.6.8) if one of the following conditions is met:

- The destination address matches a unicast address of an element of the node
- The destination address matches a fixed group destination address specified in Table 3.27 and the corresponding condition (if any) is satisfied

Destination Address	Condition
all-directed-forwarding-nodes	Directed forwarding functionality is enabled
all-proxies	Proxy functionality is enabled
all-friends	Friend functionality is enabled
all-relays	Relay functionality is enabled
all-nodes	—

Table 3.27: Fixed group destination addresses and conditions

3.6.4.3 Message error procedure

When the upper transport layer receives a message that is not understood, then the message shall be ignored. A message that is not understood includes messages that met one or more conditions listed below:

- The Transport Control message opcode is unknown by the receiving node.
- The Transport Control message size for the Transport Control opcode is incorrect.
- The Transport Control message parameters contain values that are Prohibited.

Messages that are not following segmentation requirements (see Section 3.5.3.1) are also not understood and are ignored.

3.6.5 Transport Control messages

Transport Control messages are transmitted using Upper Transport Control PDUs that can be transmitted either as a single Unsegmented Control message or as a sequence of Segmented Control messages. Unsegmented Control messages or Segmented Control messages have a 7-bit opcode field that determines the format of the parameters field. Each Transport Control message shall be sent using the smallest number of Lower Transport PDUs possible.

Opcode 0x00 is terminated at the lower transport layer as defined in Section 3.5.3 and shall not be sent by the upper transport layer. All other Transport Control messages are terminated at the upper transport layer.

When transmitting a Friend Poll, Friend Update, Friend Request, Friend Offer, Friend Subscription List Add, Friend Subscription List Remove, or Friend Subscription List Confirm message, the message shall be tagged as a friendship PDU.

The list of Transport Control messages, and their opcodes are defined in the Assigned Numbers document [4].

3.6.5.1 Friend Poll

A Friend Poll message is sent by a Low Power node to ask the Friend node to send a message that it has stored for the Low Power node.

The Friend Poll message parameters are defined in Table 3.28.

Field	Size (bits)	Description	Req.
Padding	7	0b0000000. All other values are Prohibited.	M
FSN	1	Friend Sequence Number, used to acknowledge receipt of previous messages from the Friend node to the Low Power node	M

Table 3.28: Friend Poll message parameters

The FSN field indicates the Friend Sequence Number that is used as defined in Section 3.6.6.4.2.

3.6.5.2 Friend Update

A Friend Update message is sent by a Friend node to a Low Power node to inform the Low Power node that the security parameters (see Section 3.6.6.1) for the network have changed or are changing, or that the Friend Queue is empty.

The Friend Update message parameters are defined in Table 3.29.

Field	Size (octets)	Description	Req.
Flags	1	Contains the IV Update Flag and the Key Refresh Flag	M
IV Index	4	The current IV Index value known by the Friend node	M
MD	1	Indicates whether the Friend Queue is empty or not.	M

Table 3.29: Friend Update message parameters

The Flags field format is defined in Table 3.30.

Bit	Definition
0	Key Refresh Flag 0: Not-In-Phase2 1: In-Phase2
1	IV Update Flag 0: Normal Operation 1: IV Update in Progress
2–7	Reserved for Future Use

Table 3.30: Flags field format

The Key Refresh Flag indicates whether the Key Refresh procedure is in progress (see Section 3.11.4).

The IV Update Flag indicates whether the IV Update procedure is in progress (see Section 3.11.5).

The IV Index field contains the current IV Index.

The MD field indicates whether the Friend Queue is empty or not, as defined in Table 3.31.

Value	Description
0	Friend Queue is empty
1	Friend Queue is not empty
2–255	Prohibited

Table 3.31: MD field format

3.6.5.3 Friend Request

A Friend Request message is sent to the all-friends group address by a Low Power node to start to find a friend.

The Friend Request message parameters are defined in Table 3.32.

Field	Size (octets)	Description	Req.
Criteria	1	The criteria that a Friend node should support in order to participate in friendship negotiation	M
ReceiveDelay	1	Receive Delay requested by the Low Power node	M
PollTimeout	3	The initial value of the PollTimeout timer set by the Low Power node	M
PreviousAddress	2	Unicast address of the primary element of the previous friend	M

Field	Size (octets)	Description	Req.
NumElements	1	Number of elements in the Low Power node	M
LPNCounter	2	Number of Friend Request messages that the Low Power node has sent	M

Table 3.32: Friend Request message parameters

The Criteria field format is defined in [Table 3.33](#) and in [Figure 3.18](#).

Field	Size (bits)	Description	Req.
RFU	1	Reserved for Future Use	M
RSSIFactor	2	The contribution of the received signal strength indicator (RSSI) measured by the Friend node used in the Friend Offer Delay calculation	M
ReceiveWindowFactor	2	The contribution of the supported Receive Window used in the Friend Offer Delay calculation	M
MinQueueSizeLog	3	The minimum number of messages that the Friend node can store in its Friend Queue	M

Table 3.33: Criteria field format

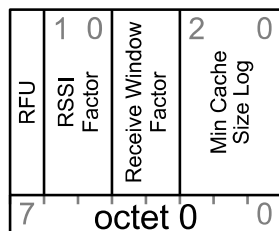


Figure 3.18: Criteria field format

The RSSIFactor field indicates the contribution of the RSSI measured by the Friend node in the Friend Offer Delay calculation (see [Section 3.6.6.3.1](#)).

The RSSIFactor field values are defined in [Table 3.34](#).

Value	RSSI Factor
0b00	1
0b01	1.5
0b10	2
0b11	2.5

Table 3.34: RSSIFactor field values

The ReceiveWindowFactor field indicates the contribution of the supported Receive Window used in the Friend Offer Delay calculation (see Section 3.6.6.3.1).

The ReceiveWindowFactor field values are defined in Table 3.35.

Value	Receive Window Factor
0b00	1
0b01	1.5
0b10	2
0b11	2.5

Table 3.35: ReceiveWindowFactor field values

The MinQueueSizeLog field is defined as $\log_2(N)$, where N is the minimum number of maximum size Lower Transport PDUs that the Friend node can store in its Friend Queue.

The MinQueueSizeLog field values are defined in Table 3.36.

Value	Description
0b000	Prohibited
0b001	$N=2$
0b010	$N=4$
0b011	$N=8$
0b100	$N=16$
0b101	$N=32$
0b110	$N=64$
0b111	$N=128$

Table 3.36: MinQueueSizeLog field values

The ReceiveDelay field indicates the Receive Delay requested by the Low Power node.

The ReceiveDelay field values are defined in Table 3.37.

Value	Description
0x00–0x09	Prohibited
0x0A–0xFF	Receive Delay in units of 1 millisecond

Table 3.37: ReceiveDelay field values

The PollTimeout field indicates the initial value of the Poll Timeout timer set by the Low Power node.

The PollTimeout field values are defined in Table 3.38.

Value	Description
0x000000–0x000009	Prohibited
0x00000A–0x34BBFF	PollTimeout in units of 100 milliseconds
0x34BC00–0xFFFFFFFF	Prohibited

Table 3.38: PollTimeout field values

The PreviousAddress field indicates the previous Friend's unicast address or the unassigned address if no previous friendship was established.

The NumElements field indicates the number of elements of the Low Power node. The value is used by the Friend node to calculate the range of unicast addresses assigned to the Low Power node using the SRC field value of the Network PDU of this message.

The NumElements field values are defined in [Table 3.39](#).

Value	Description
0x00	Prohibited
0x01–0xFF	Number of elements

Table 3.39: NumElements field values

The LPNCounter field indicates the number of Friend Request messages that the Low Power node has sent to date.

3.6.5.4 Friend Offer

A Friend Offer message is sent by a Friend node to a Low Power node to offer a friendship.

The Friend Offer message parameters are defined in [Table 3.40](#).

Field	Size (octets)	Description	Req.
ReceiveWindow	1	Receive Window value supported by the Friend node	M
QueueSize	1	Size of the Friend Queue available on the Friend node	M
SubscriptionListSize	1	Size of the Subscription List that can be supported by a Friend node for a Low Power node	M
RSSI	1	RSSI measured by the Friend node	M
FriendCounter	2	Number of Friend Offer messages that the Friend node has sent	M

Table 3.40: Friend Offer message parameters

The ReceiveWindow field indicates the Receive Window value supported by the Friend node.

The ReceiveWindow field values are defined in [Table 3.41](#).

Value	Description
0x00	Prohibited
0x01–0xFF	Receive Window in units of 1 millisecond

Table 3.41: ReceiveWindow field values

The QueueSize field indicates the number of messages that the Friend node can store for the Low Power node.

The SubscriptionListSize field indicates the number of entries in the Subscription List that the Friend node can support for the Low Power node.

The RSSI field contains a signed 8-bit value, and is interpreted as an indication of received signal strength measured in decibels above 1 milliwatt (dBm). This is measured by the Friend node upon receiving the Friend Request message. The value shall be 0x7F (127 dBm) if the received signal strength indication is not available.

The FriendCounter field represents the number of Friend Offer messages that the Friend node has sent.

3.6.5.5 Friend Clear

A Friend Clear message is an acknowledged message sent to the previous Friend node of a Low Power node to inform the Friend node of the removal of a friendship. This message is sent by the current Friend node of the Low Power node.

The Friend Clear message parameters are defined in [Table 3.42](#).

Field	Size (octets)	Description	Req.
LPNAddress	2	The unicast address of the Low Power node being removed	M
LPNCounter	2	Value of the LPNCounter of the new friendship	M

Table 3.42: Friend Clear message parameters

The LPNAddress field contains the unicast address of the Low Power node that is being removed.

The LPNCounter field contains the LPNCounter value of the latest Friend Request used to establish the relationship.

3.6.5.6 Friend Clear Confirm

A Friend Clear Confirm message is sent by the previous Friend node in response to the Friend Clear message to confirm that the friendship has been terminated.

The Friend Clear Confirm message parameters are defined in [Table 3.43](#).

Field	Size (octets)	Description	Req.
LPNAddress	2	The unicast address of the Low Power node being removed	M
LPNCounter	2	The value of the LPNCounter of the corresponding Friend Clear message	M

Table 3.43: Friend Clear Confirm message parameters

The LPNAddress field contains the unicast address of the Low Power node that was removed.

The LPNCounter field contains the LPNCounter value of the corresponding Friend Clear message.

3.6.5.7 Friend Subscription List Add

A Friend Subscription List Add message is sent by a Low Power node to a Friend node to add group addresses and virtual addresses to the Friend Subscription list (see Section 3.6.6).

The Friend Subscription List Add message parameter is defined in Table 3.44.

Field	Size (octets)	Description	Req.
TransactionNumber	1	The number for identifying a transaction	M
AddressList	2 * N	List of group addresses and virtual addresses where N is the number of group addresses and virtual addresses in this message	M

Table 3.44: Friend Subscription List Add message parameters

The TransactionNumber field is used to distinguish each individual transaction (see Section 3.6.6.4.3).

The AddressList field contains a list of group addresses and virtual addresses to add to the Friend Subscription List.

3.6.5.8 Friend Subscription List Remove

A Friend Subscription List Remove message is sent by a Low Power node to a Friend node to remove group addresses and virtual addresses from the Friend Subscription List (see Section 3.6.6).

The Friend Subscription List Remove message parameters are defined in Table 3.45.

Field	Size (octets)	Description	Req.
TransactionNumber	1	The number for identifying a transaction	M
AddressList	2 * N	List of group addresses and virtual addresses where N is the number of group addresses and virtual addresses in this message	M

Table 3.45: Friend Subscription List Remove message parameters

The TransactionNumber field is used to distinguish each individual transaction (see Section 3.6.6.4.3).

The AddressList field contains a list of group addresses and virtual addresses to remove from the Friend Subscription List.

3.6.5.9 Friend Subscription List Confirm

A Friend Subscription List Confirm message is sent by a Friend node to a Low Power node in response to a Friend Subscription List Add message or a Friend Subscription List Remove message.

The Friend Subscription List Confirm message parameters are defined in Table 3.46.

Field	Size (octets)	Description	Req.
TransactionNumber	1	The number for identifying a transaction	M

Table 3.46: Friend Subscription List Confirm message parameters

The TransactionNumber field is used to distinguish each individual transaction (see Section 3.6.6.4.3).

3.6.5.10 Heartbeat

A Heartbeat message is sent by a node to let other nodes determine topology of a subnet.

The Heartbeat message parameters are defined in Table 3.47.

Field	Size (bits)	Description	Req.
RFU	1	Reserved for Future Use	M
InitTTL	7	Initial TTL used when sending the message	M
Features	16	Bit field of currently active features of the node	M

Table 3.47: Heartbeat message parameters

The InitTTL field contains the initial TTL field value used when sending the message.

The InitTTL field values are defined in Table 3.48.

Value	Description
0x00–0x7F	Initial TTL when sending a message

Table 3.48: InitTTL value definitions

The Features field contains a bit field indicating the features the node currently uses, as defined in Section 3.1.

The Features field format is defined in Table 3.49.

Bit	Feature	Description
0	Relay	Relay feature (see Table 3.50)
1	Proxy	Proxy feature (see Table 3.50)
2	Friend	Friend feature (see Table 3.50)
3	Low Power	Low Power feature (see Table 3.50)
4–15	RFU	Reserved for Future Use

Table 3.49: Features field format

Features field bits values are defined in [Table 3.50](#).

Bit Value	Description
0	The feature indicated by the bit is not in use
1	The feature indicated by the bit is in use

Table 3.50: Features field bit values

3.6.5.11 PATH_REQUEST

A PATH_REQUEST message is sent by a Path Origin or by a Directed Relay node to discover a path to a destination.

The PATH_REQUEST message parameters are defined in [Table 3.51](#).

Field	Size (bits)	Description	Req.
On_Behalf_Of_Dependent_Origin	1	Transmitted on behalf of a dependent node of the Path Origin	M
Path_Origin_Path_Metric_Type	3	Path metric type used to calculate the path metric	M
Path_Origin_Path_Lifetime	2	Path lifetime associated with the path	M
Path_Discovery_Interval	1	Path discovery interval	M
Prohibited	1	Prohibited	M
Path_Origin_Forwarding_Number	8	Forwarding number generated by the Path Origin	M
Path_Origin_Path_Metric	8	Path metric associated with the path	M
Destination	16	Destination address of the path	M
Path_Origin_Unicast_Addr_Range	variable (16 or 24)	Path Origin unicast address range	M

Field	Size (bits)	Description	Req.
Dependent_Origin_Unicast_Addr_Range	variable (16 or 24)	Unicast address range of the dependent node of the Path Origin	C.1

Table 3.51: *PATH_REQUEST* message parameters

C.1: If the `On_Behalf_Of_Dependent_Origin` field is 1, the `Dependent_Origin_Unicast_Addr_Range` field shall be present; otherwise, the `Dependent_Origin_Unicast_Addr_Range` field shall not be present.

The `On_Behalf_Of_Dependent_Origin` field indicates whether or not the `PATH_REQUEST` message is originated by the Path Origin on behalf of a dependent node.

The `Path_Origin_Path_Metric_Type` field contains the value of the Path Metric Type state of the Path Origin that is stored in the Discovery Table (see Section 3.6.8.6.1).

The `Path_Origin_Path_Lifetime` field contains the value of the Path Lifetime state of the Path Origin that is stored in the Discovery Table (see Section 3.6.8.6.1).

The `Path_Discovery_Interval` field contains the value of the Path Discovery Interval state of the Path Origin that is stored in the Discovery Table (see Section 3.6.8.6.1).

The `Path_Origin_Forwarding_Number` field indicates the last forwarding number generated by the Path Origin, as defined in Section 3.6.8.4.

The `Path_Origin_Path_Metric` field indicates the path metric calculated starting from the Path Origin.

The `Destination` field indicates the intended destination. The value of the `Destination` field shall not be the unassigned address nor the all-directed-forwarding-nodes, all-nodes, or all-relays fixed group addresses.

The `Path_Origin_Unicast_Addr_Range` field indicates the unicast address range (see Section 3.4.2.2.1) of the Path Origin.

If present, the `Dependent_Origin_Unicast_Addr_Range` field indicates the unicast address range (see Section 3.4.2.2.1) of the dependent node of the Path Origin.

3.6.5.12 PATH_REPLY

A `PATH_REPLY` message is sent by a Path Target or by a Directed Relay node to establish a path from a Path Origin to a Path Target.

The `PATH_REPLY` message parameters are defined in Table 3.52.

Field	Size (bits)	Description	Req.
Unicast_Destination	1	Flag indicating whether the <code>PATH_REPLY</code> message was transmitted in response to a <code>PATH_REQUEST</code> message with a unicast address in the <code>Destination</code> field	M

Field	Size (bits)	Description	Req.
On_Behalf_Of_Dependent_Target	1	Flag indicating whether the PATH_REPLY message was transmitted on behalf of a dependent node of the Path Target	M
Confirmation_Request	1	Confirmation requested	M
Prohibited	5	Prohibited	M
Path-Origin	16	Primary element address of the Path Origin	M
Path-Origin_Forwarding_Number	8	Forwarding number generated by the Path Origin	M
Path_Target_Unicast_Addr_Range	variable (16 or 24)	Path Target unicast address range	C.1
Dependent_Target_Unicast_Addr_Range	variable (16 or 24)	Unicast address range of the dependent node of the Path Target	C.2

Table 3.52: PATH_REPLY message parameters

C.1: If the Unicast_Destination field is 1, the Path_Target_Unicast_Addr_Range field shall be present; otherwise, the Path_Target_Unicast_Addr_Range field shall not be present.

C.2: If the Unicast_Destination field and the On_Behalf_Of_Dependent_Target field both are 1, the Dependent_Target_Unicast_Addr_Range field shall be present; otherwise, the Dependent_Target_Unicast_Addr_Range field shall not be present.

The Unicast_Destination field indicates whether or not the PATH_REPLY message is originated by a Path Target in response to a PATH_REQUEST message with a unicast address in the Destination field.

The On_Behalf_Of_Dependent_Target field indicates whether or not the PATH_REPLY message is originated by a Path Target on behalf of a dependent node.

The Confirmation_Request field indicates the Two Way Path state of the Path Target (see Section 4.2.31), if the Unicast_Destination field is 1.

The Path-Origin field indicates the primary element address of the Path Origin.

The Path-Origin_Forwarding_Number field indicates the forwarding number generated by the Path Origin, as defined in Section 3.6.8.4.

If present, the Path_Target_Unicast_Addr_Range field indicates the unicast address range (see Section 3.4.2.2.1) of the Path Target.

If present, the Dependent_Target_Unicast_Addr_Range field indicates the unicast address range (see Section 3.4.2.2.1) of the dependent node of the Path Target.

3.6.5.13 PATH_CONFIRMATION

A PATH_CONFIRMATION message is sent by a Path Origin or by a Directed Relay node to confirm that a two-way path has been established from the Path Origin to a Path Target.



The PATH_CONFIRMATION message parameters are defined in [Table 3.53](#).

Field	Size (bits)	Description	Req.
Path_Origin	16	Primary element address of the Path Origin	M
Path_Target	16	Primary element address of the Path Target	M

Table 3.53: PATH_CONFIRMATION message parameters

The Path_Origin field indicates the primary element address of the Path Origin.

The Path_Target field indicates the primary element address of the Path Target.

3.6.5.14 PATH_ECHO_REQUEST

A PATH_ECHO_REQUEST message is sent by a Path Origin to validate a path from the Path Origin to a destination. The response to a PATH_ECHO_REQUEST message is a PATH_ECHO_REPLY message.

The PATH_ECHO_REQUEST message has no parameters.

3.6.5.15 PATH_ECHO_REPLY

A PATH_ECHO_REPLY message is sent by a Path Target in response to a PATH_ECHO_REQUEST message in order to confirm that a path exists from a Path Origin to the destination.

The PATH_ECHO_REPLY message parameters are defined in [Table 3.54](#).

Field	Size (bits)	Description	Req.
Destination	16	Destination address of the path	M

Table 3.54: PATH_ECHO_REPLY message parameters

The Destination field indicates the intended destination. The value of the Destination field shall not be the unassigned address nor the all-directed-forwarding-nodes, all-nodes, or all-relays fixed group addresses.

3.6.5.16 DEPENDENT_NODE_UPDATE

A DEPENDENT_NODE_UPDATE message is sent by a Path Origin or by a Directed Relay node to notify nodes in a subnet that element addresses of a dependent node of the Path Origin are to be added or removed from the Forwarding Table state.

A DEPENDENT_NODE_UPDATE message is also sent by a Path Target or by a Directed Relay node to notify nodes in a subnet that element addresses of a dependent node of the Path Target are to be added or removed from the Forwarding Table state.

The DEPENDENT_NODE_UPDATE message parameters are defined in [Table 3.55](#).

Field	Size (bits)	Description	Req.
Type	1	Type of update	M
Prohibited	7	Prohibited	M
Path_Endpoint	16	Path endpoint, either the Path Origin or the Path Target	M
Dependent_Node_Unicast_Addr_Range	variable (16 or 24)	Unicast address range of the dependent node of the path endpoint	M

Table 3.55: *DEPENDENT_NODE_UPDATE* message parameters

The Type field indicates whether the dependent node address is added or removed. The values of the Type field are defined in [Table 3.56](#).

Value	Description
0	The dependent node address is removed
1	The dependent node address is added

Table 3.56: *Type* field values

The Path_Endpoint indicates the primary element address of the Path Origin or of the Path Target that notified the nodes about the update.

The Dependent_Node_Unicast_Addr_Range field indicates the unicast address range (see [Section 3.4.2.2.1](#)) of the dependent node.

3.6.5.17 PATH_REQUEST_SOLICITATION

A PATH_REQUEST_SOLICITATION message is sent by a Directed Forwarding node or a Configuration Manager to solicit the discovery of paths from other Directed Forwarding nodes.

The PATH_REQUEST_SOLICITATION message parameter is defined in [Table 3.57](#).

Field	Size (octets)	Description	Req.
Addr_List	2 * N	List of N destination addresses	M

Table 3.57: *PATH_REQUEST_SOLICITATION* message parameter

The Addr_List field is a list of unicast addresses, group addresses, and virtual addresses that are destinations for the requested Directed Forwarding Initialization. The all-directed-forwarding-nodes, all-nodes, and all-relays fixed group addresses are Prohibited for the Addr_List field.

3.6.6 Friendship

When a Friend node is in friendship with a Low Power node, it stores Lower Transport PDUs for the Low Power node.



A Friend node may be friends with multiple Low Power nodes. A Low Power node can only be in a friendship with a single Friend node in a given subnet at a time. If a Low Power node is on multiple subnets, it can be in a friendship on each subnet with the same or a different Friend node.

3.6.6.1 Functional overview

In order to optimize the power consumption of a Low Power node, a polling mechanism is used to minimize the Low Power node's receive window. This allows a Low Power node to receive updates from a Friend node by indicating when it is available to receive messages.

Friendship defines timing parameters that are static for the duration of a friend relationship between a Low Power node and a Friend node.

The following timing parameters are used in a friendship:

- ReceiveDelay
- ReceiveWindow
- PollTimeout

The ReceiveDelay is the time between the Low Power node sending a request and listening for a response. This delay allows the Friend node time to prepare the response.

The ReceiveWindow is the time that the Low Power node listens for a response. When the Low Power node receives a message from its Friend node, it can stop listening for additional messages.

The request can be a Friend Poll message, a Friend Subscription List Add message, or a Friend Subscription List Remove message. The response to a Friend Poll message can be a Friend Update message or a stored message. The response to a Friend Subscription List Add message or a Friend Subscription List Remove message is a Friend Subscription List Confirm message.

The timing parameters are illustrated in [Figure 3.19](#).

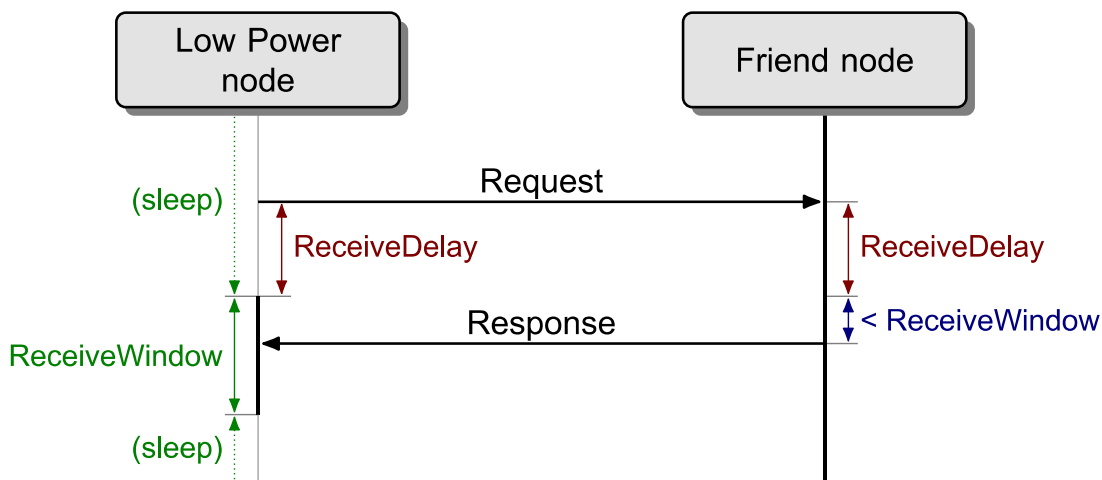


Figure 3.19: Friendship timing parameters

PollTimeout timer is used to measure time between two consecutive requests sent by the Low Power node. If no requests are received by the Friend node before the PollTimeout timer expires, then the friendship is considered terminated. This is illustrated in [Figure 3.20](#).

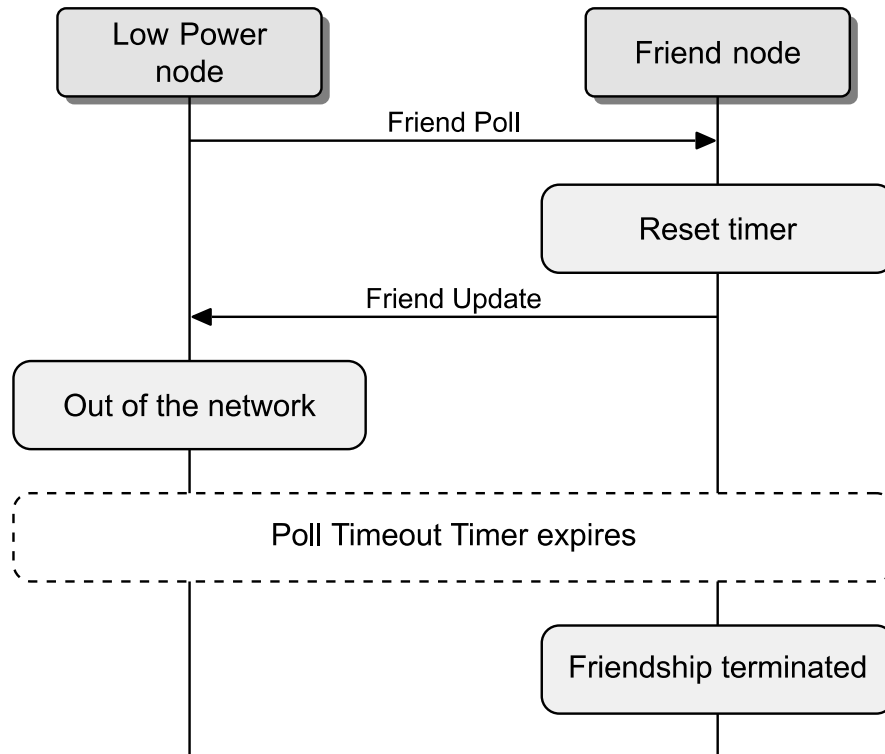


Figure 3.20: Poll Timeout timer illustration

To establish a friendship, a node that supports the Low Power feature sends a Friend Request to the all-friends address. The message is picked up by all nodes within radio range of this node that support the Friend feature.

The Friend Request message includes a number of parameters that define the requirements that this node requires any future Friend node to support. Each of the nodes that support the Friend feature and that can support the requirements of the Friend Request message responds by sending back a Friend Offer message to the requesting node. The offers also include additional information about the capabilities of each of the offering nodes. This allows the Low Power node to determine which of these offers it will accept.

The Low Power node then sends a Friend Poll message to its chosen Friend node, and the Friend node responds with a Friend Update message. At this point, the friendship is established, as illustrated in [Figure 3.21](#).

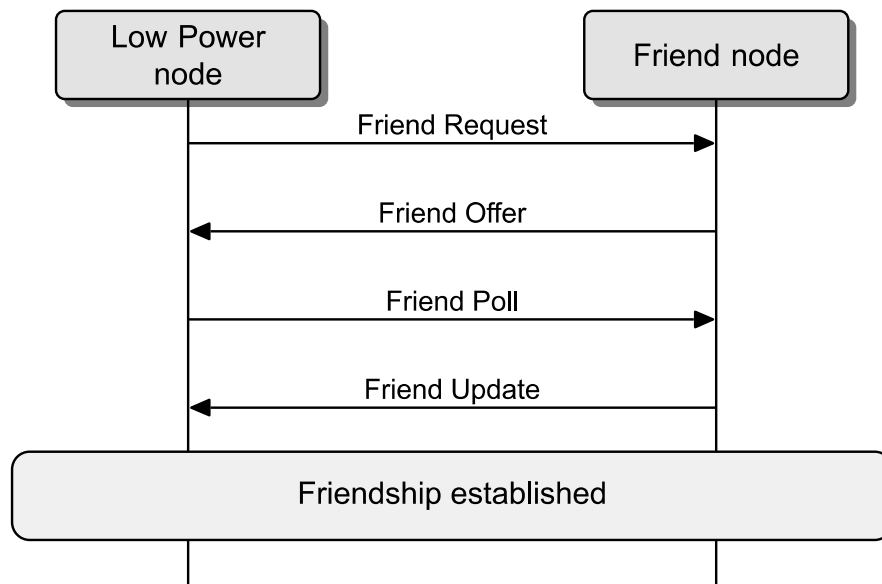


Figure 3.21: Establishment of a friendship

If the Low Power node was previously a friend of another Friend node, then the new Friend node informs the old Friend node that it is now the current friend of this Low Power node (see Section 3.6.6.3.1).

After a friendship is established, the Friend node stores a Friend Subscription List for the Low Power node, which is a collection of group and virtual addresses to which the Low Power node is subscribed. This list allows the Friend node to store the messages that the Low Power node is subscribed to.

The values of the IV Index, IV Update Flag, and Key Refresh Flag for a given NetKey are known as security parameters. Whenever at least one of the values of the security parameters is changed, the new security parameters are known as security updates.

The Friend node stores all messages for the Low Power node, and also the most recent security updates for the Low Power node, in the Friend Queue. Collectively these are known as stored messages.

To obtain stored messages, the Low Power node sends Friend Poll messages and the Friend node replies with stored messages.

Messages stored in the Friend Queue are delivered to the Low Power node with acknowledgment and in order. To enable this, a Boolean Friend Sequence Number is used. This value is stored on the Low Power node and is sent in each Friend Poll message. When the Low Power node receives a message in response to a Friend Poll, and this message has been successfully authenticated using the friendship security credentials, the Low Power node shall change this Friend Sequence Number. The next time this Low Power node polls, the Friend node can send the next message. If there was no response to the Friend Poll message, then the Low Power node does not change the Friend Sequence Number and the Friend node can then determine that the last message it sent was not received and resends it.

3.6.6.2 Friendship security

When a friendship has been established between a Friend node and a Low Power node, all Network PDUs from the Friend node to the Low Power Node are secured using the friendship security material derived from the friendship security credentials (see Section 3.9.6.3.1). The friendship security credentials are exchanged during the Low Power Establishment procedure (see Section 3.6.6.4.1).

The Friend Poll, Friend Update, and Friend Subscription List Add/Remove/Confirm messages, as well as stored messages that the Friend node delivers to the Low Power node, are always secured using the friendship security material. If the Friend node is a Directed Friend node (see Section 2.3.13), friendship security credentials are used for messages that the Low Power node sends to the Friend node to be forwarded along a path (see Section 3.6.8.3). The Friend Clear and Friend Clear Confirm messages are always secured using the managed flooding security material.

Depending on the value of the Publish Friendship Credentials Flag (see Section 4.2.3.4), the Low Power node model publishes messages using either the friendship security credentials or the managed flooding security credentials (see Section 3.9.6.3.1).

All other Network PDUs are sent using the managed flooding security credentials.

Figure 3.22 illustrates messages sent using the friendship security material (dashed lines) and using the managed flooding security material (solid lines). The Low Power node starts by sending the Friend Request message using the managed flooding security credentials. A Friend node responds with a Friend Offer message, again using the managed flooding security credentials. Both the Low Power node and the Friend node are using the managed flooding security credentials as neither device is in a friendship with the other and therefore cannot use the friendship security credentials. The Low Power node accepts the offer of friendship and sends a Friend Poll to confirm this using the friendship security credentials. The Friend node will respond to this using a Friend Update message. The Low Power node can now configure the friend subscription list by using the Friend Subscription List Add message, confirmed using the Friend Subscription List Confirm message from the Friend node. Both of these messages are sent using the friendship security credentials.

Sometime later, the Friend node receives a message (InMsg) from another device that needs to be delivered to the Low Power node, so it will store this message in the Friend Queue. The Low Power node sends a Friend Poll message, secured using the friendship security material, to which the Friend node will reply with the stored InMsg.

The Low Power node then decides to send two messages: OutMsg1 and OutMsg2. OutMsg1 is sent secured using the friend security material and therefore only the Friend node will receive and relay this message. When the Friend node relays OutMsg1, the message will be retransmitted using the managed flooding security credentials. OutMsg2 is sent using the managed flooding security credentials and therefore the Friend node and any other Relay node within range of the Low Power node can relay the message. OutMsg2, when it is relayed, will be retransmitted using the managed flooding security credentials.

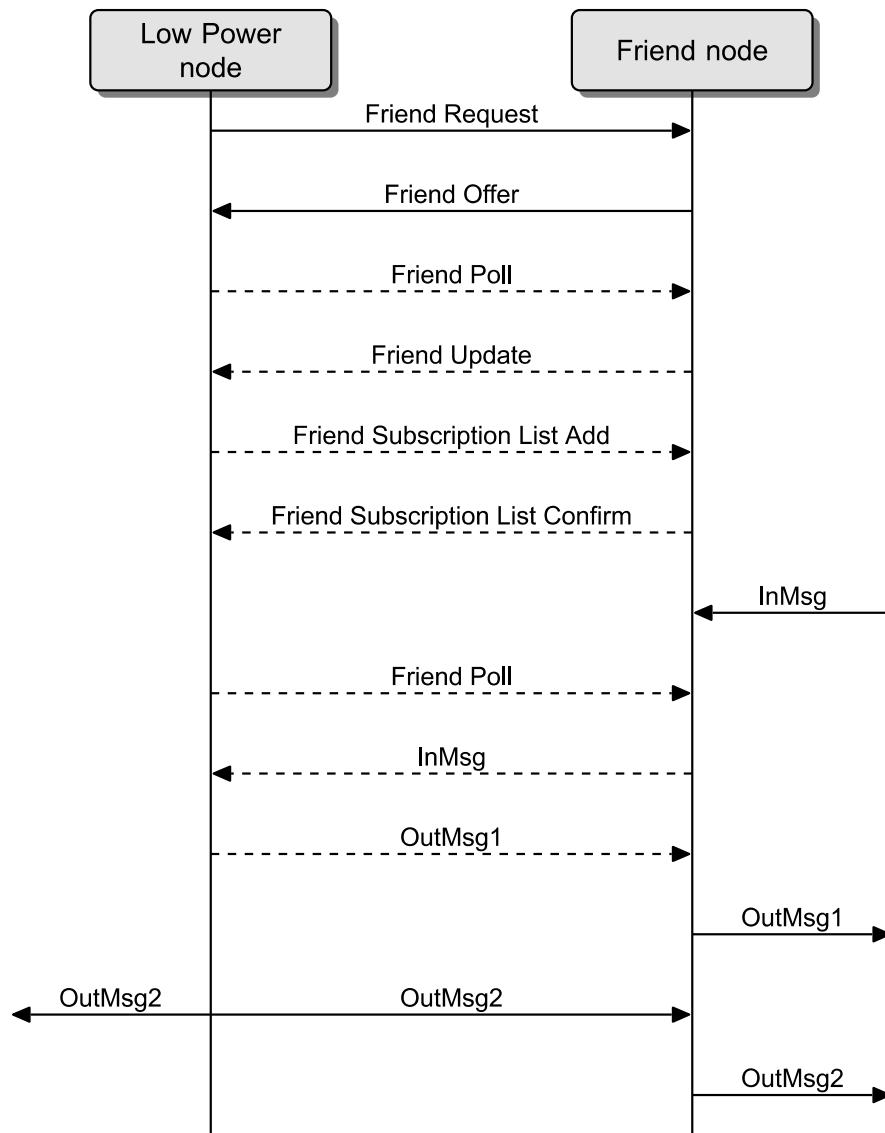


Figure 3.22: Messages secured with friendship and managed flooding security material

3.6.6.3 Friend feature

The Friend feature defines three mandatory procedures: friend establishment, friend messaging, and friend management.

The Friend Update, Friend Offer, Friend Clear, Friend Clear Confirm, and Friend Subscription List Confirm messages originated by a Friend node shall be sent as Unsegmented Control messages with the SRC field set to the unicast address of the primary element of the Friend node.

3.6.6.3.1 Friend establishment

A node that supports the Friend feature and has the feature enabled, and that receives a Friend Request message (see Section 3.6.5.3), that fulfills the minimum requirements specified by the message parameters shall respond with a Friend Offer message (see Section 3.6.5.4).

If the source address of the Friend Request message is an address of a Low Power node that is currently in a friendship with the Friend node on the subnet over which the message was received, then the Friend node shall also consider the existing friendship with that Low Power node on that subnet terminated.

In the Network PDU of a Friend Offer message, the TTL field shall be set to 0 and the DST field shall be set to the SRC value in the Network PDU of the Friend Request message.

The Friend Offer message shall be sent using the managed flooding security credentials and shall be tagged with the immutable-credentials tag.

The node shall keep a Friend node counter (FriendCounter), which is a 2-octet value initialized to 0. This value shall be sent in the Friend Offer message and shall be used to derive the friendship security material if a friendship is established as a result of the Friend Offer message. After each Friend Offer message is sent, the FriendCounter value shall be incremented by 1. The FriendCounter may wrap.

The time between receiving the Friend Request message and sending the Friend Offer message is called the Friend Offer Delay and shall be computed based on the RSSIFactor field and the ReceiveWindowFactor field as defined in the Friend Request message on the supported ReceiveWindow and on the RSSI measured by the Friend node for the Friend Request message.

The Friend Offer Delay allows a Low Power node to receive Friend Offer messages from potential Friend nodes in order to determine how large the offered ReceiveWindow is, and how important the signal quality is. Some Low Power nodes will prefer Friend nodes with a very small ReceiveWindow and therefore set the ReceiveWindowFactor to be more important than the RSSIFactor. Other Low Power nodes will prefer Friends with a very good signal strength and therefore set the RSSIFactor to be more important than the ReceiveWindowFactor. This means that the Low Power node should receive Friend Offers from Friends quicker for those nodes that match the Low Power nodes requirements, reducing the power consumed by the Low Power node when searching for a Friend node.

To optimize power consumption by the LPN, the Friend Node should provide the smallest possible ReceiveWindow.

A Local Delay is computed with the formula:

$$Local\ Delay = ReceiveWindowFactor \times ReceiveWindow - RSSIFactor \times RSSI$$

Where:

ReceiveWindowFactor is a number from the Friend Request message

ReceiveWindow is the value to be sent in the corresponding Friend Offer message

RSSIFactor is a number from the Friend Request message

RSSI is the received signal strength of the received Friend Request message on the Friend node

If the Local Delay value is greater than 100, then the Friend Offer Delay value in milliseconds shall be set to the Local Delay value. Otherwise, the Friend Offer Delay shall be set to 100 milliseconds.

If the node receives a Friend Poll message within 1 second after sending the Friend Offer message, the friendship is established and it shall save the FSN field value from this Friend Poll message; otherwise, the establishment has failed.

The Friend node shall respond with a Friend Update message after a minimum of ReceiveDelay milliseconds and before a maximum of the sum of ReceiveDelay and ReceiveWindow milliseconds, from the reception of the Friend Poll message from the Low Power node.

In the Network PDU of a Friend Update message, the TTL field shall be set to 0.

This Friend Update message shall be sent using the friendship security credentials and shall be tagged with the immutable-credentials tag.

[Figure 3.23](#) illustrates a friendship establishment where multiple nodes with the Friend feature enabled receive the same Friend Request message.

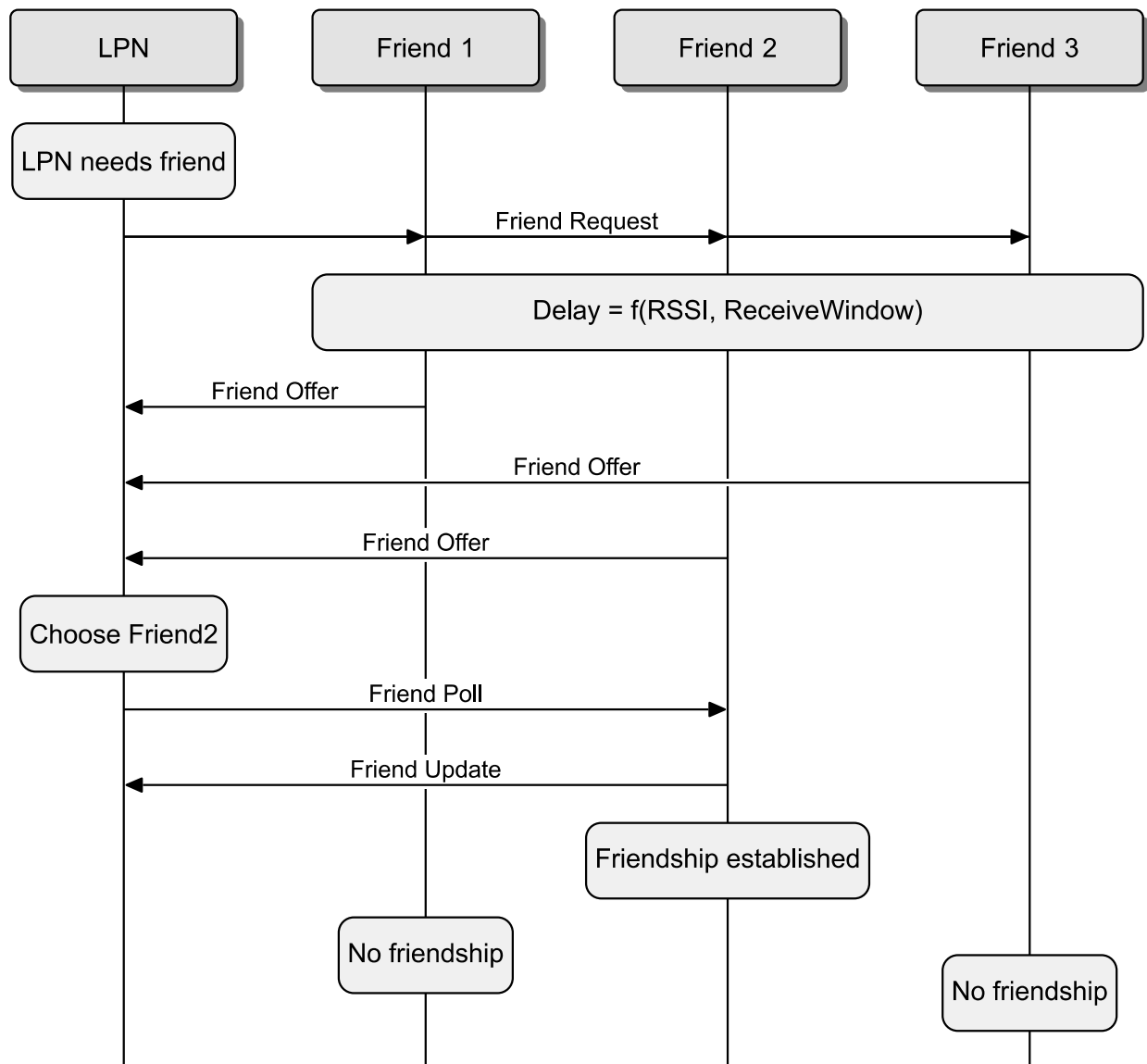


Figure 3.23: Friend establishment example

After a friendship is established, the Friend node shall initialize a Friend Subscription List to a zero-length (empty) list and start storing messages for the Low Power node in the Friend Queue.

After a friendship has been established, if the PreviousAddress field of the Friend Request message contains a valid unicast address that is not the Friend node's own unicast address, then the Friend node

shall begin sending Friend Clear messages (see Section 3.6.5.5) to that unicast address according to the procedure below:

1. In the Network PDU of the Friend Clear message, the TTL field shall be set to 0x7F.
2. The Friend Clear message shall be sent using the managed flooding security credentials and shall be tagged with the immutable-credentials tag.
3. The first Friend Clear message shall be sent as soon as the friendship is established; at the same time, a Friend Clear Repeat timer shall be started with the period set to 1 second, and a Friend Clear Procedure timer shall be started with the period equal to two times the Friend Poll Timeout value.
4. If a Friend Clear Confirm message (see Section 3.6.5.6) is received in response to the Friend Clear message, both timers shall be stopped and the procedure is complete.
5. If the Friend Clear Repeat timer expires, a new Friend Clear message shall be sent and the timer shall be started from the initial value that is double the previous Friend Clear Repeat timer initial value. For example, after the first expiration, the period shall be set to two seconds; on the next expiration, it shall be set to four seconds, and so on.
6. If the Friend Clear Procedure timer expires, then the Friend Clear Repeat timer shall be stopped and the procedure is complete.

An example of this procedure is illustrated in [Figure 3.24](#).

Once friendship has been established the Friend node shall communicate with the Low Power node as defined in friend messaging (see Section 3.6.6.3.2), and may be managed as defined in Friend Management (see Section 3.6.6.3.3).

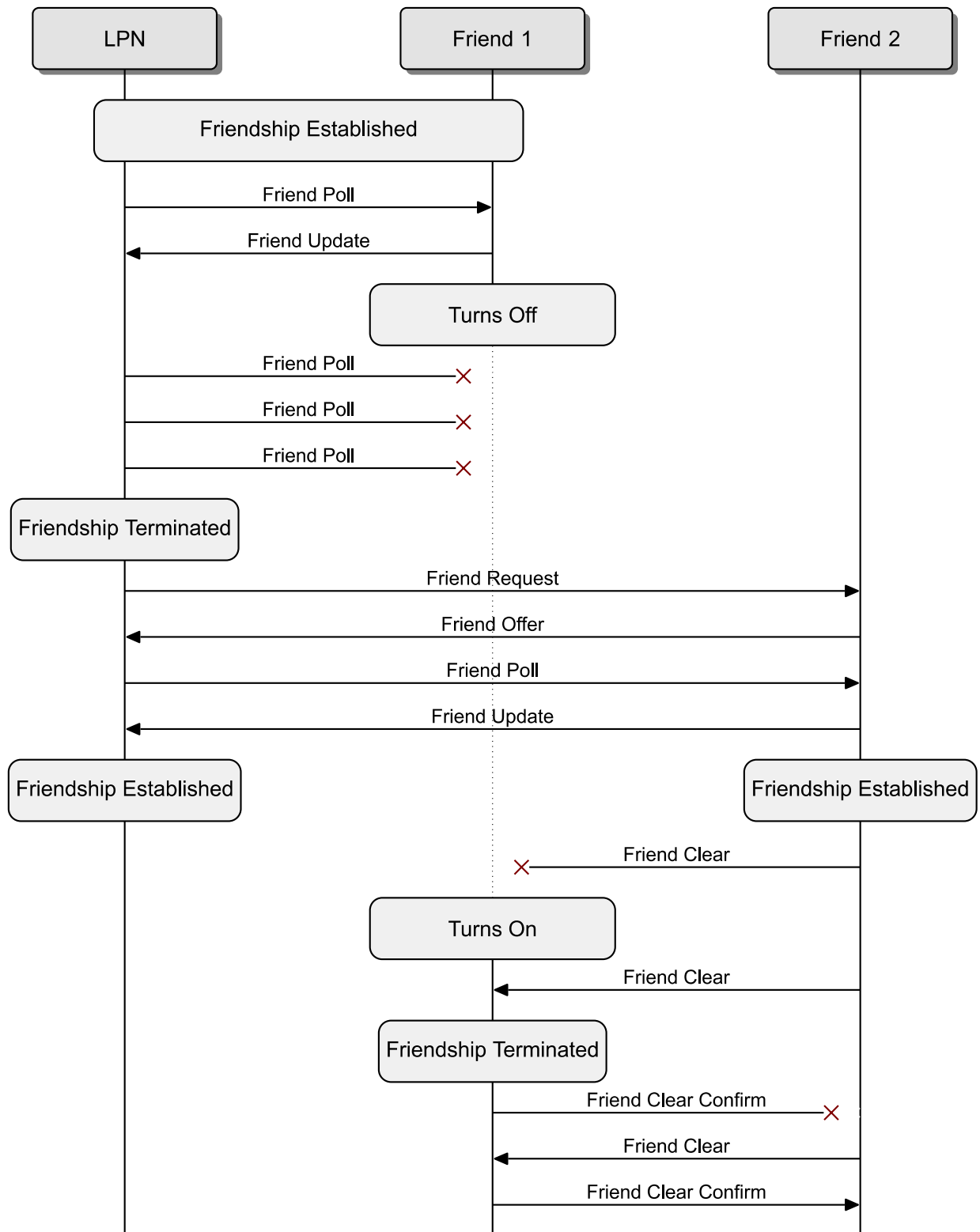


Figure 3.24: Friend Clear procedure example

3.6.6.3.2 *Friend messaging*

When the Friend node receives a Friend Poll message from a friend Low Power node that has the same FSN field value as the last Friend Poll message received from the Low Power node, the Friend node shall respond with exactly the same message it has previously sent, unless that message has been discarded. If the previously sent message has been discarded, then the oldest entry in the Friend Queue shall be sent.

When the Friend node receives a Friend Poll message from a friend Low Power node that has a different FSN field value as the last Friend Poll message from the Low Power node, then it shall send the oldest message from the Friend Queue.

When sending a stored message to the Low Power node, it shall be sent unchanged and shall be tagged as friendship PDU. If the IV Index on the Friend node has changed, for example the node is now transmitting with a new IV Index, the messages sent to the Low Power node shall still be sent in the context of the IV Index that the Friend node received for those messages.

Note: The above requirement implies that a Low Power node should collect all stored messages at least once every 96 hours, otherwise the Friend node may discard the stored messages before the Low Power node can receive them.

Each message shall be sent after a minimum of ReceiveDelay milliseconds and before a maximum of the sum of ReceiveDelay and ReceiveWindow milliseconds, from the reception of the Friend Poll message from a friend Low Power node.

If no Friend Poll, Friend Subscription List Add, or Friend Subscription List Remove messages are received by the Friend node before the PollTimeout timer expires, the friendship is terminated and the Friend node shall discard all entries in the Friend Queue.

The Friend Subscription List Confirm message shall be sent after a minimum of ReceiveDelay milliseconds and before a maximum of the sum of ReceiveDelay and ReceiveWindow milliseconds, from the reception of the Friend Subscription List Add message or the Friend Subscription List Remove message.

In the Network PDU of a Friend Subscription List Confirm message, the TTL field shall be set to 0.

The Friend Subscription List Confirm message shall be sent using the friendship security credentials and shall be tagged with the immutable-credentials tag.

When a Friend node receives a valid Friend Clear message where the LPNAddress field contains the address of a friend Low Power node, and the message is received within the Poll Timeout of the friendship with that Low Power node, the friendship is terminated, and the Friend node shall discard all related entries in the Friend Queue, and the Friend node should respond with a Friend Clear Confirm message.

A Friend Clear message is considered valid if the result of the subtraction of the value of the LPNCounter field of the Friend Request message (the one that initiated the friendship) from the value of the LPNCounter field of the Friend Clear message, modulo 65536, is in the range 0 to 255 inclusive.

If the Friend Clear message was received with a TTL field value of 0, the Network PDU of the Friend Clear Confirm message should be set to 0 as well.

The Friend Clear Confirm message shall be sent using the managed flooding security credentials and shall be tagged with the immutable-credentials tag.

3.6.6.3.3 *Friend management*

If the Friend node receives a Friend Subscription List Add message (see Section 3.6.5.7) from the Low Power node, it shall add the address or list of addresses contained in the message into the Friend Subscription List and shall respond with a Friend Subscription List Confirm message (see Section 3.6.5.9) setting the value of the TransactionNumber field to the same value as in the received Friend Subscription List Add message. If the Friend node is a Directed Friend node, the Friend node initiates a Directed Forwarding Solicitation procedure (see Section 3.6.8.3).

If the Friend node receives a Friend Subscription List Remove message (see Section 3.6.5.8) from the Low Power node, it shall remove the address or list of addresses contained in the message from the Friend Subscription List and shall respond with a Friend Subscription List Confirm message setting the value of the TransactionNumber field to the same value as in the received Friend Subscription List Remove message.

The Friend node shall respond with a Friend Update message after a minimum of ReceiveDelay milliseconds and before a maximum of the sum of ReceiveDelay and ReceiveWindow milliseconds, from the reception of the Friend Poll message from the Low Power node.

3.6.6.4 Low Power feature

A node that supports the Low Power feature shall support the three mandatory procedures: Low Power establishment, Low Power messaging, and Low Power management.

The Friend Poll, Friend Request, Friend Subscription List Add, and Friend Subscription List Remove messages originated by a Low Power node shall be sent as Unsegmented Control messages with the SRC field set to the unicast address of the primary element of the node that supports the Low Power feature.

3.6.6.4.1 *Low Power establishment*

The Low Power establishment procedure is used to establish friendship between a node supporting the Low Power feature and a node supporting the Friend feature.

This procedure is started by sending a Friend Request message.

In the Network PDU of a Friend Request message, the TTL field shall be set to 0 and the DST field shall be set to the all-friends address. The node shall keep a Low Power node counter (LPNCounter), which is a 2-octet value initialized to 0. This value shall be sent in the Friend Request message and used to derive the friendship security material if a friendship is established as a result of the Friend Request message. After each Friend Request message is sent, this value shall be incremented by 1. The LPNCounter may wrap. The Friend Request message shall be sent using the managed flooding security credentials and shall be tagged with the immutable-credentials tag.

After 100 milliseconds have passed from the Friend Request, the node should listen for up to 1 second for the Friend Offer messages sent by potential Friend nodes, and it may select one of the Friend nodes to establish a friendship. The Low Power node may accept a received Friend Offer or continue to listen for other Friend Offer messages for comparison.



If no acceptable Friend Offer message is received, the node may send a new Friend Request message. The time interval between two consecutive Friend Request messages shall be greater than 1.1 seconds.

To establish a friendship with a potential Friend that has sent a Friend Offer message, the node shall set the Friend Sequence Number to zero and shall send a Friend Poll message to the selected Friend node within 1 second after the reception of the Friend Offer message. If a Friend Update message is received in response, the friendship is established, the Low Power feature of the node supporting it is in use and the Friend feature of the node supporting it is in use.

In the Network PDU of a Friend Poll message, the TTL field shall be set to 0.

This Friend Poll message shall be sent using the friendship security credentials and shall be tagged with the immutable-credentials tag.

The node should restart the Low Power Establishment procedure if it does not receive a Friend Update message after several attempts (e.g., 6 attempts).

Multiple failures of the Low Power Establishment procedure may be an indication that the Low Power node no longer has a valid IV Index and it should initiate the IV Index Recovery procedure (see Section 3.11.6).

Once friendship has been established the Low Power node shall communicate with the Friend node as defined in Low Power messaging (see Section 3.6.6.4.2) and may manage the friendship as defined in Low Power management (see Section 3.6.6.4.3).

If the Low Power node supports directed forwarding functionality when the friendship is established in a subnet, the Low Power node shall store the current value of the Directed Forwarding state and shall set the state to 0x00 (see Section 4.2.26.1) for that subnet. When that friendship is terminated, the Low Power node shall set the Directed Forwarding state to the stored value.

3.6.6.4.2 *Low Power messaging*

The Low Power messaging procedure is executed by a Low Power node to receive stored messages and security updates from the Friend node.

The procedure consists of asynchronous requests from the Low Power node to the Friend node and timed responses from the Friend node to the Low Power node.

A Low Power node that is in a friendship with a Friend node shall send a Friend Poll message to the Friend node before the PollTimeout timer expires.

As a general rule, the Low Power node should continue sending Friend Poll messages until it receives a Friend Update message with the MD field set to 0. This is illustrated in Figure 3.25.

The Low Power node may terminate friendship with a Friend by sending a Friend Clear. The Friend Clear message should be sent using a TTL field value set to 0.

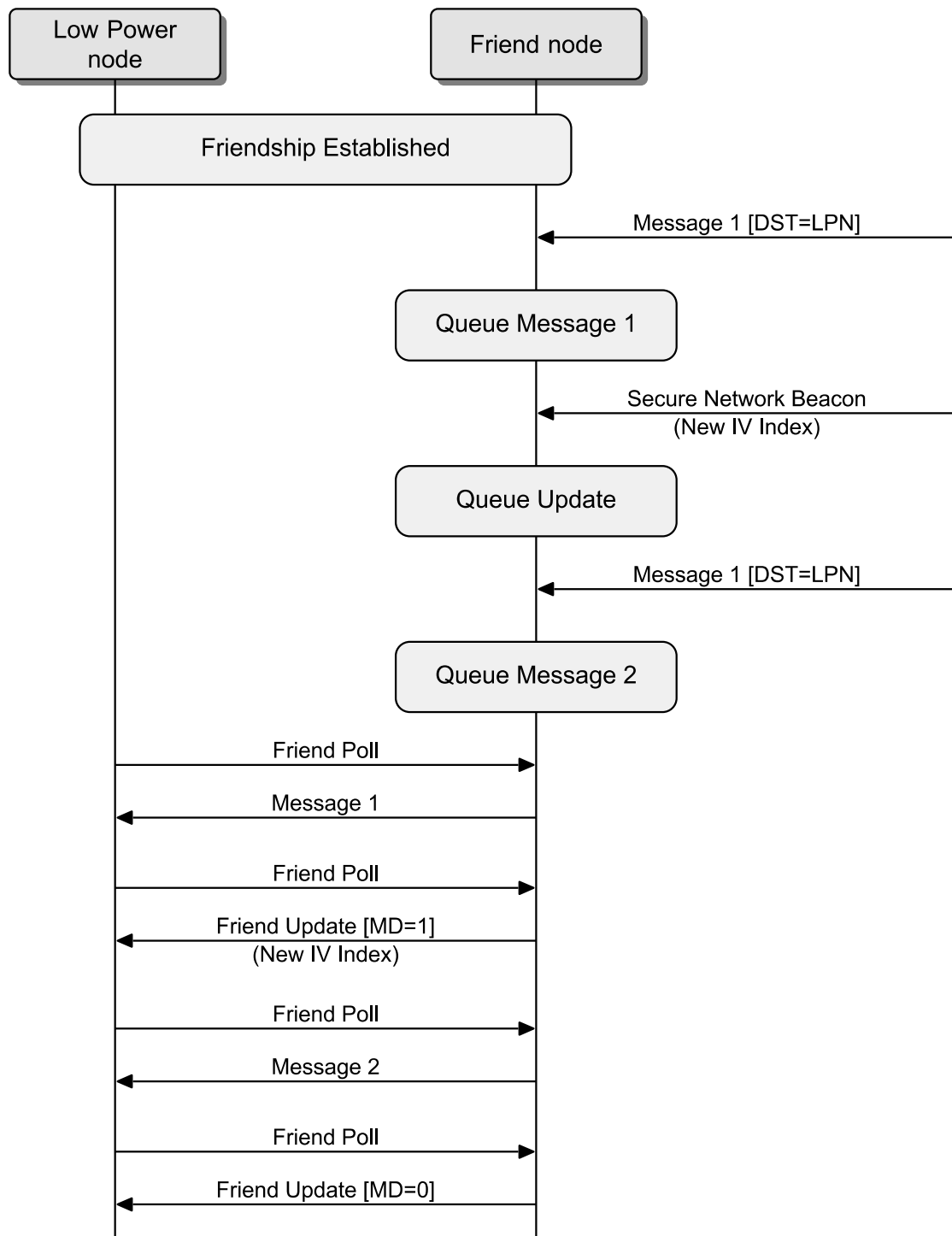


Figure 3.25: Friend Update with security updates

The FSN field shall be set to the value of the Friend Sequence Number.

If the Low Power node receives a response from the Friend node, and the response is not a duplicate of the last received Lower Transport PDU, then the Low Power node shall toggle the Friend Sequence Number.

If the Low Power node does not receive a response within the ReceiveWindow, it should resend the Friend Poll message. It is recommended to resend this message 3 times, which assures a good balance between reliability and power consumption.

If no response has been received to multiple Friend Poll messages before the PollTimeout timer expires, the friendship is terminated. The Low Power node may repeat the Low Power establishment procedure.

If the Low Power node receives a Friend Update message, it shall process the Flags and IV Index fields using the same rules as if they had been received in a Secure Network beacon (see Sections 3.10.3.1, 3.11.4, and 3.11.5) or in a Mesh Private beacon (see Section 3.10.4).

3.6.6.4.3 Low Power management

The Low Power management procedure is used to manage the subscription list in a Friend node.

The Low Power node may send one or more Friend Subscription List Add messages to the Friend node containing lists of group addresses or virtual addresses to which the Low Power node is subscribed. This type of message may be sent at any time by the Low Power node when its subscriptions change.

In the Network PDU of a Friend Subscription List Add message, the TTL field shall be set to 0.

The Friend Subscription List Add message shall be sent using the friendship security credentials and shall be tagged with the immutable-credentials tag.

The Low Power node may send one or more Friend Subscription List Remove messages to the Friend node containing lists of group addresses or virtual addresses to which the Low Power node is no longer subscribed. This type of message may be sent at any time by the Low Power node when its subscriptions change.

In the Network PDU of a Friend Subscription List Remove message, the TTL field shall be set to 0.

The Friend Subscription List Remove message shall be sent using the friendship security credentials and shall be tagged with the immutable-credentials tag.

The Low Power node shall start with a TransactionNumber value set to 0x00. It shall increment the TransactionNumber for each new Friend Subscription List Add or Friend Subscription List Remove such that the TransactionNumber can be matched with the TransactionNumber field of the Friend Subscription List Confirm message.

3.6.6.5 Examples of segmentation and reassembly

The segmentation and reassembly behaviors defined in Sections 3.5.3.3 and 3.5.3.4 also apply to segmented messages sent to and from a Low Power node. The only difference is that since the Low Power node relies on the Friend Queue for all incoming messages, including segments and segment acknowledgments, the Friend node will acknowledge segmented transactions for the Low Power node.

This section provides two examples of segmentation and reassembly on the Low Power node.



3.6.6.5.1 *Incoming segmented message*

The message sequence chart (MSC) in [Figure 3.26](#) is an example of a segmented message directed toward a Low Power node. The Friend node performs the reassembly separately and sends the acknowledgments needed until it receives all segments, at which point the Friend node places the segments in the Friend Queue so that they can be delivered to the Low Power node. An unsegmented message from another source is received in the middle of the transaction and is handled independently.

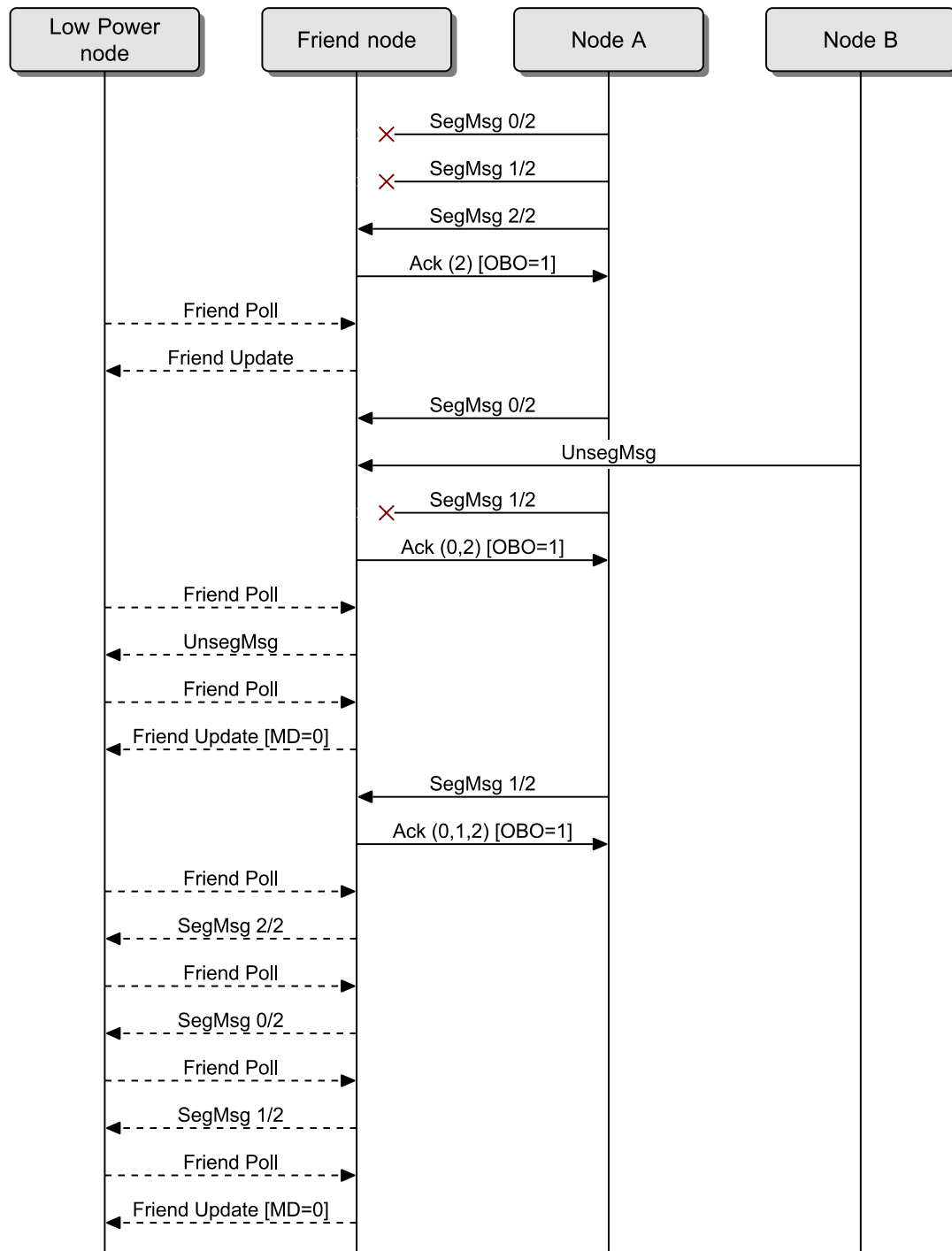


Figure 3.26: Example of incoming segmented message directed to a Low Power node

3.6.6.5.2 Outgoing segmented message

The MSC illustrated in Figure 3.27 shows an example of a segmented message sent by the Low Power node. Since the Low Power node relies on the Friend Queue for all incoming messages, it needs to poll for the acknowledgment as well. An unsegmented message from another source is received in the middle of the transaction and is handled independently.

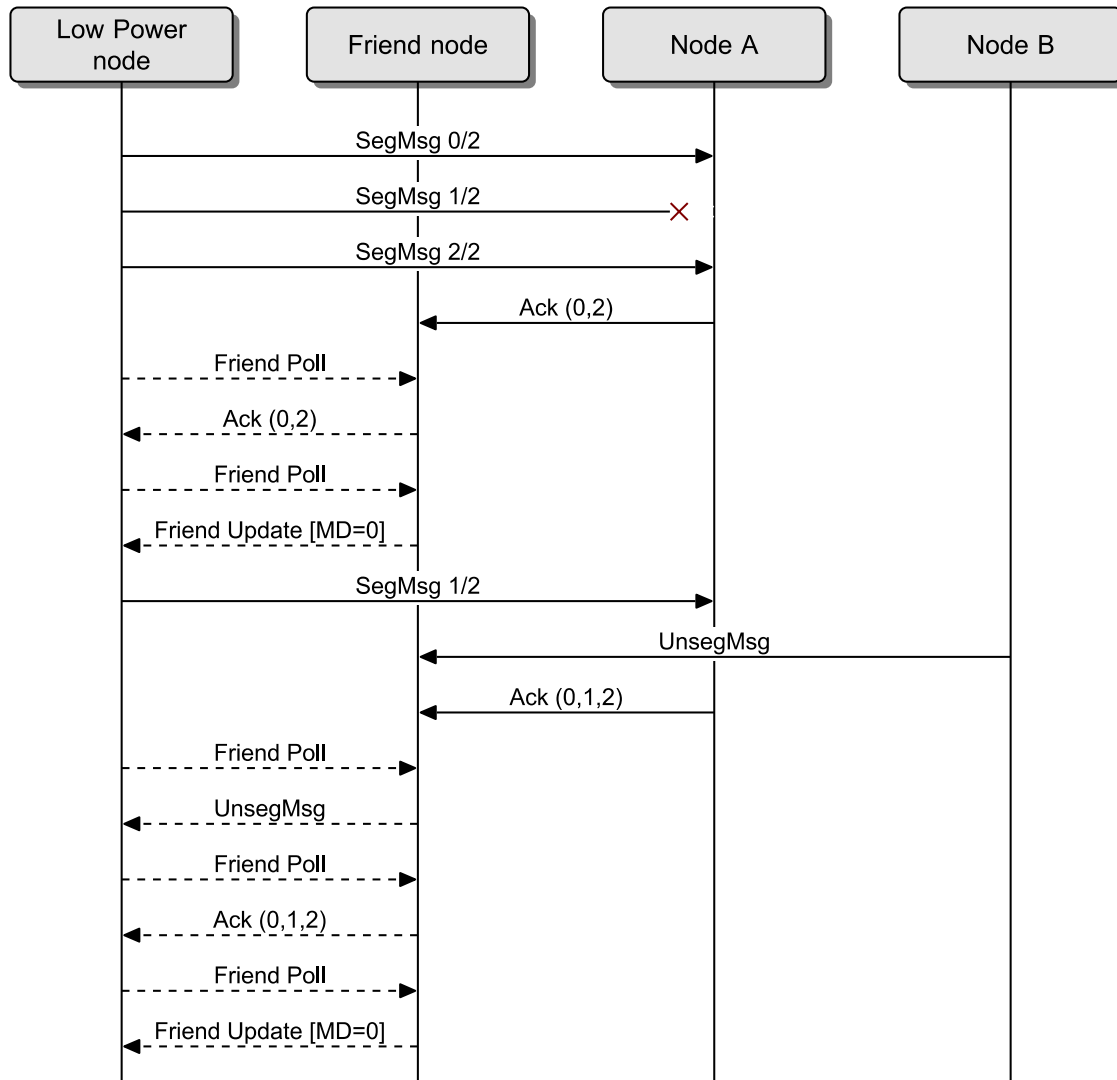


Figure 3.27: Example of outgoing segmented message sent by a Low Power node

3.6.7 Heartbeat

Heartbeat is used to monitor nodes on a network and discover how far nodes are apart from each other.

3.6.7.1 Functional overview

In order to determine if a node is still present and active within a mesh network, it is necessary to receive a message from this node. Sending a message to each and every node in the mesh network to elicit a

response would be very wasteful of energy, and therefore each node can be configured to send a single message periodically. This message is called the Heartbeat message.

The Heartbeat message can be used for three main functions. The first function is the determination that a node is still active within a mesh network. The second function is the determination of how far a node is away. The third function is the determination that a feature of the node has been enabled or disabled.

Publishing of the Heartbeat messages is configured by the Configuration Server model. Periodic Heartbeat messages can be sent a limited number of times or they can be sent indefinitely. Heartbeat messages are sent to a configured destination, and it is recommended that a group address is used for sending Heartbeat messages. The messages can also be configured with a specific TTL value.

When Heartbeat messages are received, they are counted. The number of Heartbeat messages received can help determine the reliability of the mesh network for delivering messages from the node sending Heartbeat messages.

Each Heartbeat message includes the initial TTL value used when sending the original Heartbeat message. This allows a receiving device to determine the number of times this message was retransmitted, known as the number of hops, and a record of the minimum and maximum number of hops can also be used to determine how reliable the mesh network is.

The use of Heartbeat messages can therefore be used to determine the best TTL value to use to address a given node.

Heartbeat messages also include the features of a node that are currently in use. A node can be configured to publish a triggered Heartbeat message when various features are enabled or disabled. This allows the features available on various nodes within a mesh network to be determined.

3.6.7.2 Publishing Heartbeat messages

Publishing of Heartbeat messages is controlled by the Heartbeat Publication state (see Section 4.2.18).

When the Heartbeat Publication Destination (see Section 4.2.18.1) address is set to an unassigned address, Heartbeat messages shall not be published. When the value of the Heartbeat Publication Count state (see Section 4.2.18.2) is 0x0000, periodic Heartbeat messages shall not be published.

Heartbeat messages shall be published with the DST field set to the value of the Heartbeat Publication Destination state and with the TTL field set to the value of the Heartbeat Publication TTL state (see Section 4.2.18.4).

Periodic publishing of Heartbeat messages is enabled by the Heartbeat Publication Count state (see Section 4.2.18.2). After publishing a periodic Heartbeat message, if the Heartbeat Publication Count counter is less than 0xFFFF, the Heartbeat Publication Count counter shall be decreased by 1. The counter shall stop at 0x0000. The first periodic Heartbeat message shall be published as soon as possible after the Heartbeat Publication Period state (see Section 4.2.18.3) has been configured for periodic publishing. The next periodic Heartbeat message shall be published after the number of seconds defined by the Heartbeat Publication Period state (see Section 4.2.18.3).

Triggered publishing of Heartbeat messages is enabled by the Heartbeat Publication Features state (see Section 4.2.18.5):

- If the Relay bit is set to 1, a triggered Heartbeat message shall be published when the Relay state of a node (see Section 4.2.9) changes.
- If the Proxy bit is set to 1, a triggered Heartbeat message shall be published when the GATT Proxy state of a node (see Section 4.2.12) changes.
- If the Friend bit is set to 1, a triggered Heartbeat message shall be published when the Friend state of a node (see Section 4.2.14) changes.
- If the Low Power bit is set to 1, a triggered Heartbeat message shall be published when the Low Power node establishes or loses Friendship (see Section 3.6.6.1).

3.6.7.3 Receiving Heartbeat messages

Receiving of Heartbeat messages is controlled by the Heartbeat Subscription state (see Section 4.2.19).

The Heartbeat Subscription Period state is a countdown timer identifying the number of seconds remaining for the period when Heartbeat messages are received. When the timer expires, the receiving of Heartbeat messages shall be disabled.

Heartbeat messages with the SRC field set to a value other than the Heartbeat Subscription Source state (see Section 4.2.19.1) or the DST field set to a value other than the Heartbeat Subscription Destination state (see Section 4.2.19.2) shall not be processed.

Upon receiving a Heartbeat message, the value of the Heartbeat Subscription Count state (see Section 4.2.19.3) shall be increased. The counter does not wrap. It stops counting at 0xFFFF.

Upon receiving the Heartbeat message, a hops value shall be calculated using the InitTTL value from the message, and the received Network PDU TTL field value, known as RxTTL, as follows:

$$hops = InitTTL - RxTTL + 1$$

Note: If the message is received directly (for example, the InitTTL value and the received Network PDU TTL field value are the same), then the hops value would be 0x01. If the message has been delivered using the maximum length path, then InitTTL would be 0x7F and the received Network PDU TTL field value would be 0x01, and therefore hops would be 0x7F.

If the hops value is lower than the Heartbeat Subscription Min Hops state, it shall be set as the new value of the Heartbeat Subscription Min Hops state. If the hops value is higher than the Heartbeat Subscription Max Hops state, it shall be set as the new value of the Heartbeat Subscription Max Hops state.

3.6.8 Directed forwarding

Directed forwarding is used to establish paths between Directed Forwarding nodes as described in Section 2.3.11. Paths are established between a source node, known as the Path Origin, and one or more destination nodes, known as Path Targets, with Directed Relay nodes forwarding messages along the paths.

3.6.8.1 Functional overview of directed forwarding

In order to initiate the discovery of a path, a Path Origin sends a `PATH_REQUEST` message indicating its intended destination address. This procedure is named Directed Forwarding Initialization (see Section 3.6.8.2.1).

Directed Relay nodes discover the path by re-generating and broadcasting the `PATH_REQUEST` message until it reaches all Path Targets. When a node receives a `PATH_REQUEST`, it stores information about discovered paths temporarily in the Discovery Table (see Section 3.6.8.6). This procedure is named Directed Forwarding Discovery (see Section 3.6.8.2.2).

The path is made available by unicasting, hop by hop, a `PATH_REPLY` from each Path Target back to the Path Origin. Each Directed Forwarding node that receives the `PATH_REPLY` message stores path information in the Forwarding Table state (see Section 3.6.8.5). This procedure is named Directed Forwarding Establishment (see Section 3.6.8.2.3).

A `PATH_REPLY` may trigger the Path Origin to send a `PATH_CONFIRMATION` message that is propagated, hop by hop, along the path from the Path Origin to a Path Target to confirm that the path is a two-way path. This procedure is named Directed Forwarding Confirmation (see Section 3.6.8.2.4). A two-way path can be used to forward messages from the Path Origin to the Path Target (along the Forward Path) and from the Path Target to the Path Origin (along the Backward Path).

A dependent node (see Section 3.6.8.3) can use a Path Origin or a Path Target to perform directed forwarding on its behalf. By sending a `DEPENDENT_NODE_UPDATE` message, a Path Origin or a Path Target notifies each node along the paths either to add the dependent node addresses to its Forwarding Table entry or to remove them. This procedure is named Directed Forwarding Dependents Update (see Section 3.6.8.2.5).

A path from a Path Origin to a Path Target can be monitored by using a `PATH_ECHO_REQUEST` message. The Path Target responds with a `PATH_ECHO_REPLY` message. This procedure is named Directed Forwarding Echo (see Section 3.6.8.2.6).

A Directed Forwarding node or a Configuration Manager can solicit the discovery of paths toward unicast addresses, group addresses, or virtual addresses by using a `PATH_REQUEST_SOLICITATION` message. This procedure is named Directed Forwarding Solicitation (see Section 3.6.8.2.7).

Directed forwarding functionality uses a control sequence number known as the forwarding number to distinguish Transport Control messages sent by a node in the context of different Directed Forwarding Initialization procedures. The forwarding number is separate from the sequence numbers used for the message caching and the replay protection procedures (that is, the `SEQ` field defined in Section 3.4.4.5). The forwarding number is increased when a node initiates the discovery of a new path to any destination (see Section 3.6.8.4).

3.6.8.1.1 Multilane paths

To help improve reliability and resilience, paths may have multiple lanes that take advantage of several relays overhearing a message relayed on each hop. The number of lanes to be established is defined by the Wanted Lanes state (see Section 4.2.30). Each additional lane in a path is discovered by transmitting a `PATH_REQUEST` message with the same forwarding number as in the previous `PATH_REQUEST`

after a duration of Lane Discovery Guard Interval (see Section 4.2.38.4) after the Path Discovery timer expires (see Section 3.6.8.2.1).

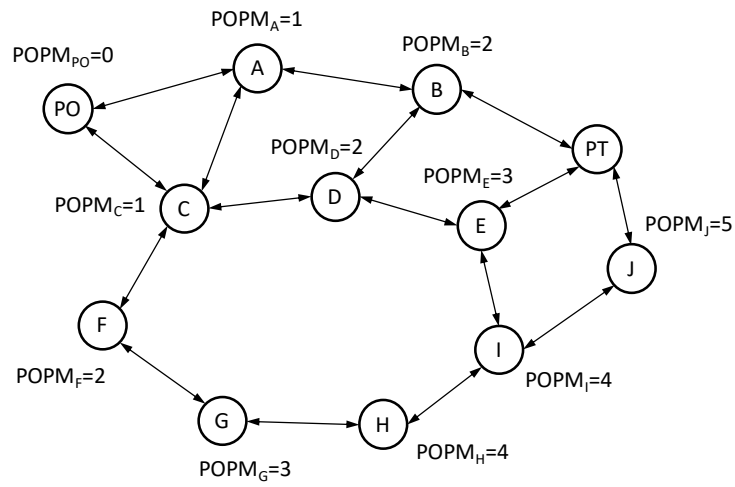
3.6.8.1.2 Path metrics

Path metrics are used to measure, rank, and select the best lane of a path (for example, the lane with the lowest number of hops) that messages will traverse.

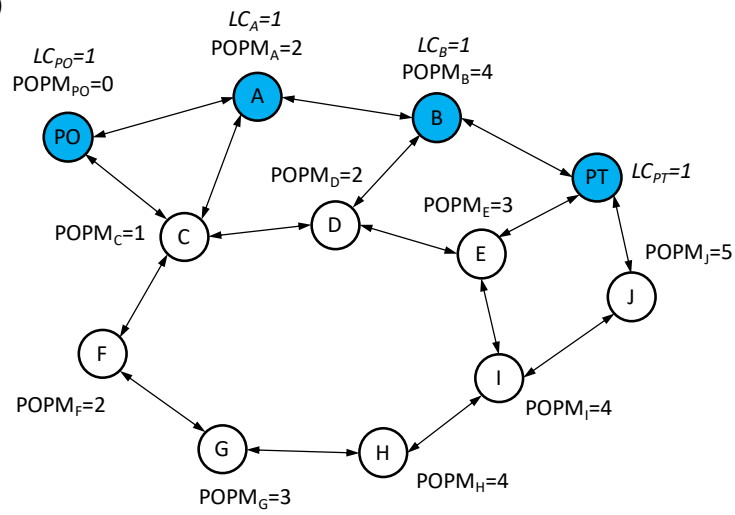
This specification uses a path metric based on the hop count between the Path Origin and the Path Target. The path metric also takes into account the number of lanes that traverse the node. In this way, a longer but disjointed lane (i.e., the lane uses different intermediate nodes than other lanes in that path) might be preferred to a shorter lane using one or more of the same intermediate nodes (see Section 4.2.27.1).

Figure 3.28 shows an example of a network composed a Path Origin (PO), a Path Target (PT), and Directed Relay nodes (identified by capital letters from A to J). The Path Origin initiates a path discovery toward the Path Target and is configured to discover two lanes (Figure 3.28 (a), (b) and (c)). In the Figure 3.28 (b) and Figure 3.28 (c), white circles represent nodes that are not chosen to be part of the path between the Path Origin and the Path Target, and blue circles represent nodes that are chosen to be part of the path. The transmission of a PATH_REQUEST message from a node is represented by arrows that connect the node with its neighbors. Before transmitting a PATH_REQUEST message, every node (say A) sets the Path-Origin_Path_Metric field in the message to the path metric value ($POPM_A$) that the node calculates as defined in Section 4.2.27.1, based on the Path-Origin_Path_Metric field in the Discovery Table (see Section 3.6.8.6), and the Lane_Counter field value (LC_A), if a Forwarding Table entry corresponding to the message exists. Figure 3.28 (a) and (b) show the lowest path metric values calculated by the nodes during the discovery of the first and second lanes, respectively. During the first lane discovery illustrated in Figure 3.28 (a), nodes PO, A, and B are selected as members of the lane. During the second lane discovery illustrated in Figure 3.28 (b), nodes PO, C, D, and E are selected as members of the lane. If the Path Origin is configured to discover three lanes, nodes PO, A, and B are selected again as members of the lane during the third discovery attempt that is illustrated in Figure 3.28 (c); this suggests that, in this example network topology and using the metric defined in this specification, using two lanes is a good compromise between path redundancy and traffic generated for path discovery.

(a)



(b)



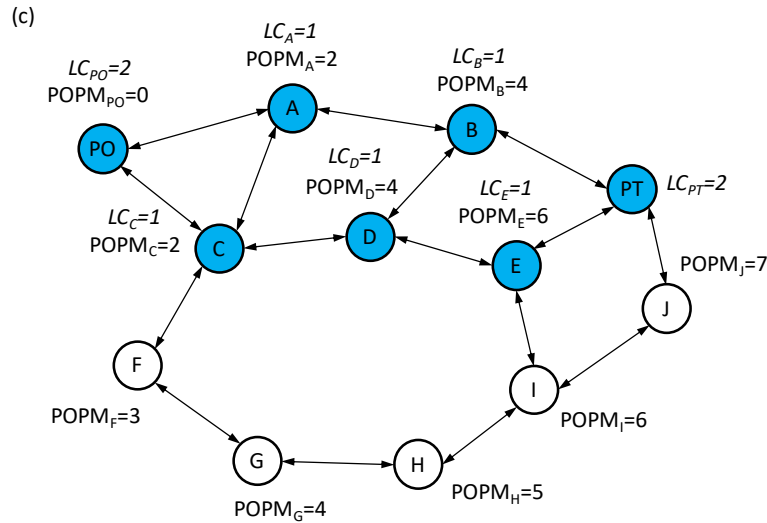


Figure 3.28: Example of network with the discovery of two path lanes

3.6.8.1.3 Network interfaces and bearers

The network layer supports sending and receiving messages via multiple bearers, and each instance of a bearer is connected to the network layer via a network interface. To be able to support additional bearers that introduce different costs to be evaluated by the path metric, the Forwarding Table entries are associated with path bearers over which the messages are transmitted (see Section 4.2.29).

PATH_REQUEST messages may be sent over multiple bearers. The Discovery Table contains an indicator of the selected bearers over which the PATH_REPLY is expected to be sent (see Section 3.6.8.6). The Forwarding Table entries are only considered valid in the context of those bearers. Therefore, when forwarding a message along a path, the message is sent over the path bearers indicated by the Forwarding Table.

3.6.8.1.4 Group destinations

Directed forwarding supports group addresses.

During configuration of a mesh node, models of elements within the node can be subscribed to one or more group addresses. A message sent to a group address using directed forwarding is processed by the subscribed nodes that support directed forwarding (see Section 3.6.8.1).

In the Directed Forwarding Initialization procedure toward a group address, a Path Origin broadcasts a PATH_REQUEST message in which the intended destination is the group address. Each node that is subscribed to the group address behaves as a Path Target and responds with a PATH_REPLY message. Intermediate Directed Relay nodes can uniquely associate the received PATH_REPLY with the forwarded PATH_REQUEST by looking for matching entries in the Discovery Table (see Section 3.6.8.6).

Within the Directed Forwarding Establishment procedure, a lane of the path is considered established if at least one PATH_REPLY from a node that is subscribed to the group address is received (see Section 3.6.8.2.1). The Path Origin receives at most a single PATH_REPLY for the primary element of each node that is subscribed to the group address.

3.6.8.1.5 *Virtual destinations*

Directed forwarding supports virtual addresses, which are treated as group addresses.

If a PATH_REQUEST message is sent to an intended destination that is a virtual address, every Directed Forwarding node that authenticates the PATH_REQUEST at the network layer, and that is subscribed to a Label UUID from which the virtual address can be derived, is considered a valid Path Target and responds with a PATH_REPLY message.

There is potential for establishing paths toward nodes that are subscribed to a different Label UUID associated with the same virtual address. In this case, some messages are forwarded also toward destinations that are not subscribed to the original Label UUID.

3.6.8.1.6 *Friendship and directed forwarding*

To enable directed forwarding procedures for a Low Power node that does not support directed forwarding functionality, the Low Power node establishes friendship with a Directed Friend node and uses the friendship security credentials for outgoing messages. After receiving a message secured with friendship security credentials from the Low Power node, the Directed Friend node can form a node dependence (see Section 3.6.8.3).

A Directed Friend node provides discovery, management, and message forwarding for a dependent Low Power node.

3.6.8.1.7 *Proxy and directed forwarding*

To enable directed forwarding procedures for a Proxy Client that does not support directed forwarding functionality, the Proxy Client connects to a Directed Proxy node. The Proxy Client configures the accept list filter instantiated by the Proxy Server to let the Proxy Server determine the Proxy Client element addresses, group addresses, and virtual addresses to which the Proxy Client is subscribed (see Section 6.7).

A Directed Proxy node provides discovery, management, and message forwarding for a Proxy Client as a dependent node (see Section 3.6.8.3).

3.6.8.1.8 *Fixed paths and non-fixed paths*

A Configuration Manager configures Directed Forwarding nodes to enable them to establish and maintain paths dynamically using directed forwarding procedures (see Section 3.6.8.2). These paths are known as non-fixed paths. In addition, a Configuration Manager can establish and maintain a path by statically selecting nodes and configuring them through the Directed Forwarding Configuration Server model (see Section 4.4.7). Paths established in this way are known as fixed paths.

Non-fixed paths have a limited lifetime set by the Configuration Manager, whereas a fixed path persists until removed by the Configuration Manager.

3.6.8.1.9 Path resilience

In directed forwarding, only nodes constituting a path participate in relaying a message from a source to a destination. The Path Origin follows the path discovery and maintenance policy. The policy is configured by a Configuration Manager and controls aspects such as path lifetime, verification and re-discovery cadence, the number of additional lanes, and whether the paths are unidirectional or bidirectional.

A path is assumed to be valid at the time of its establishment, but it could be difficult to estimate its stability. Network topologies change, and a path can become broken at any time. Increasing the number of lanes reduces the risk of a path being broken. Detecting that a path has been broken requires the exchange of additional messages.

Paths can be broken temporarily for short periods of time. Message delivery along a lane within a path can fail because a Directed Relay node was busy transmitting messages and could not listen to the incoming traffic, or because there was a temporary local interference. Multiple lanes within a path can help reduce the probability of paths being temporarily broken.

To help balance the traffic across the network, paths are distributed by randomly selecting Directed Relay nodes among candidates that are considered equally good based on the path metric (see Section 3.6.8.2.2).

The Configuration Manager configures the network by balancing the following strategies:

- Establish multiple lanes for each path. More lanes help provide redundancy, but they generate more traffic.
- Validate paths by using the Directed Forwarding Echo procedure. Faster cadence of path validation reduces the time it takes to detect a broken path, but it increases network traffic overhead.

The Configuration Manager can also set the RSSI Margin (see Section 4.2.35.2) to prevent Directed Relay nodes that receive PATH_REQUEST messages at an RSSI level close to their Default RSSI Threshold (see Section 4.2.35.1) from propagating these messages and becoming part of a path.

3.6.8.1.10 Coexistence of multiple subnets

A Directed Forwarding node maintains separate tables to store information about paths discovered (see Section 3.6.8.6) and established (see Section 4.2.29) in different subnets. Some states associated with directed forwarding support multiple subnets (see Sections 4.2.26 to 4.2.32), and some other states are common to all the subnets to which the node belongs (see Sections 4.2.33 to 4.2.40). For example, a node that supports directed forwarding can have such functionality enabled in one subnet and disabled in another subnet, while the control of timing for path discovery is independent of the subnet over which the path discovery is performed.

3.6.8.1.11 Subnet Bridge and Directed Forwarding

A Subnet Bridge node provides discovery, management, and message forwarding in the subnets where traffic from a source to a destination is bridged and where directed forwarding functionality is enabled. The source node in the first of the subnets is treated as a dependent node of the Subnet Bridge node in the second of the subnets, and the destination node in the second of the subnets is treated as a dependent node of the Subnet Bridge node in the first of the subnets (see Section 3.6.8.3).



3.6.8.2 Directed forwarding procedures

This section defines the requirements for the directed forwarding procedures.

If the Directed Forwarding state (see Section [4.2.26.1](#)) is disabled at a node in a subnet while a directed forwarding procedure is being executed in the context of that subnet, the procedure is canceled.

3.6.8.2.1 *Directed Forwarding Initialization*

The Directed Forwarding Initialization procedure is executed by a Path Origin to discover a new path or to add nodes to an existing path toward a destination in a given subnet.

[Figure 3.29](#) shows the flow chart of the Directed Forwarding Initialization procedure.

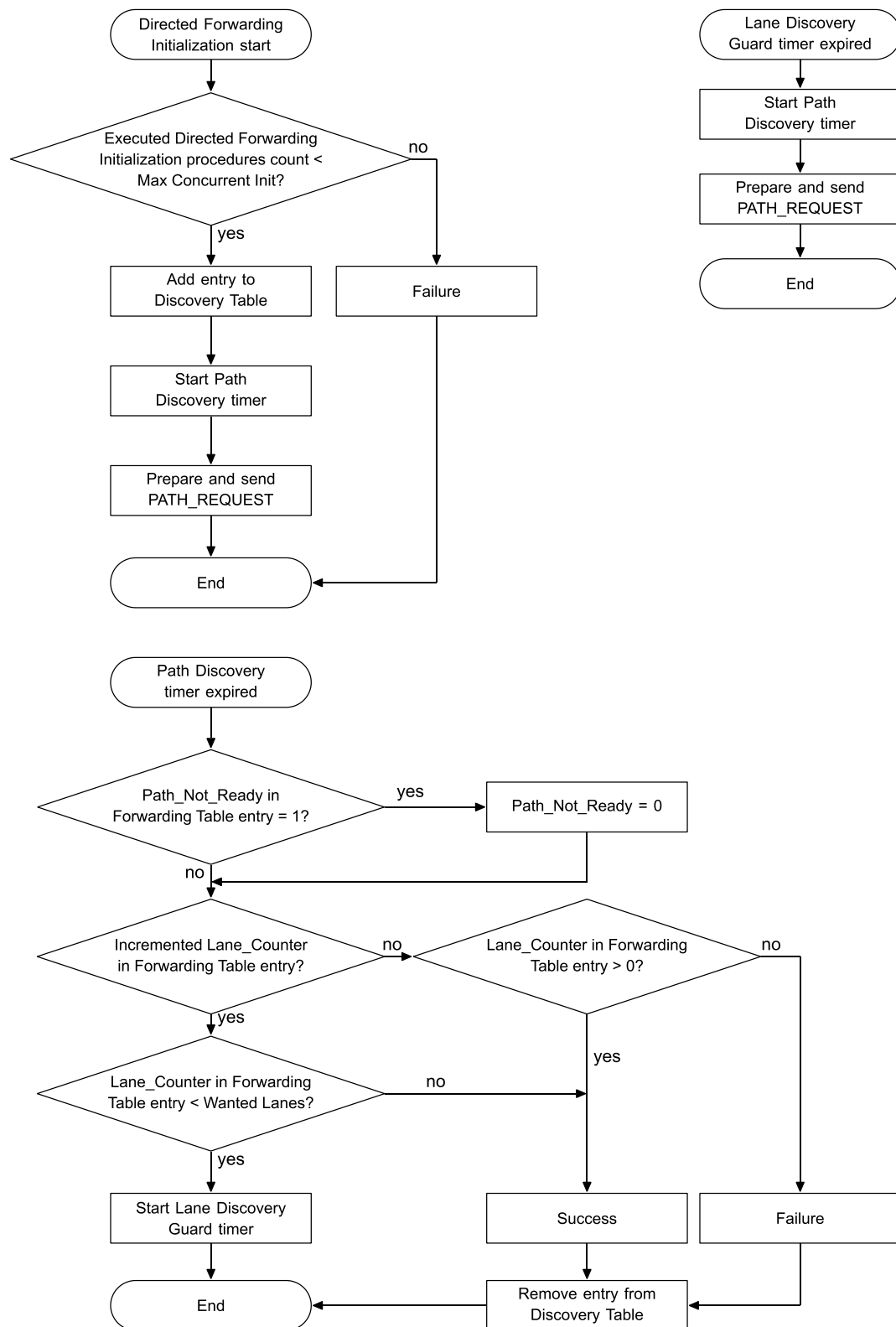


Figure 3.29: Directed Forwarding Initialization procedure

First, the Path Origin shall check whether or not the number of executed Directed Forwarding Initialization procedures (i.e., the number of Discovery Table entries that store the primary element address of the node in their Path_Origin field) is equal to the Max Concurrent Init state value (see Section 4.2.28.2).

If the number of executed Directed Forwarding Initialization procedures is equal to the Max Concurrent Init state value, then the Directed Forwarding Initialization procedure shall fail. Otherwise, a new entry shall be added to the Discovery Table according to Section 3.6.8.6.1, a new instance of the Path Discovery timer shall be started from the initial value set to the duration indicated by the Path Discovery Interval state value (see Section 4.2.38.3), and a PATH_REQUEST message shall be prepared and sent.

The new PATH_REQUEST shall have the following field values:

- If the PATH_REQUEST is originated on behalf of a dependent node of the Path Origin (see Section 3.4.6.3), the On_Behalf_Of_Dependent_Origin field shall be set to 1, and the Dependent_Origin_Unicast_Addr_Range field shall be set to the unicast address range of the dependent node; otherwise, the On_Behalf_Of_Dependent_Origin field shall be set to 0.
- The Path_Origin_Path_Metric_Type field shall be set to the Path_Origin_Path_Metric_Type field value in the Discovery Table (see Section 3.6.8.6).
- The Path_Origin_Path_Lifetime field shall be set to the Path_Origin_Path_Lifetime field value in the Discovery Table (see Section 3.6.8.6).
- The Path_Discovery_Interval field shall be set to the Path_Discovery_Interval field value in the Discovery Table (see Section 3.6.8.6).
- The Path_Origin_Forwarding_Number field shall be set according to Section 3.6.8.4.
- The Path_Origin_Path_Metric field shall be set to 0.
- The Destination field shall be set to the unicast address, group address, or virtual address of the destination.
- The Path_Origin_Unicast_Addr_Range field shall be set to the unicast address range (see Section 3.4.2.2.1) of the Path Origin.

The Network PDU for the new PATH_REQUEST message shall have the following configuration:

- The DST field shall be set to the all-directed-forwarding-nodes fixed group address (see Section 3.4.2.4).
- The TTL field shall be set to 0.

The Path Origin shall send the message using the directed security credentials of the subnet over which the message is sent and shall tag the message with the immutable-credentials tag.

When the Path Discovery timer associated with a Discovery Table entry expires, the Path Origin shall perform the following actions:

- If the value of the Path_Not_Ready field (see Section 4.2.29.2) in the corresponding Forwarding Table state entry is 1, then the Path_Not_Ready field in the entry shall be set to 0.
- If the Lane Counter value in the corresponding Forwarding Table state entry has been instantiated or incremented and is less than the Wanted Lanes state value (see Section 4.2.30), a Lane Discovery Guard timer shall be started from the initial value indicated by Lane Discovery Guard Interval state (see Section 4.2.38.4).
 - When the Lane Discovery Guard timer expires, a new PATH_REQUEST message shall be prepared and sent, and the Path Discovery timer shall be started from the initial value indicated by the Path Discovery Interval state value (see Section 4.2.38.3). All the fields in the new PATH_REQUEST message shall be copied from the previous PATH_REQUEST message.
- If the Lane Discovery Guard timer is not running, the Discovery Table entry shall be removed, and the Directed Forwarding Initialization procedure shall complete with a success or a failure based on the comparison between the Lane Counter field value in the corresponding Forwarding Table state entry and the Wanted Lanes state value:
 - If the Lane Counter field value is greater than 0 and less than the Wanted Lanes state value (i.e., at least one lane of the path had already been established), the Directed Forwarding Initialization procedure is successful.
 - If the Lane Counter field value is equal to the Wanted Lanes state value (i.e., the last lane of the path has been established), the Directed Forwarding Initialization procedure is successful.
 - If the Lane Counter field value is 0 (i.e., no lane of the path had been established), the Directed Forwarding Initialization procedure fails.

3.6.8.2.2 Directed Forwarding Discovery

The Directed Forwarding Discovery procedure shall be executed by any Directed Forwarding node that receives a PATH_REQUEST message.

Figure 3.30 shows the flow chart of the Directed Forwarding Discovery procedure.

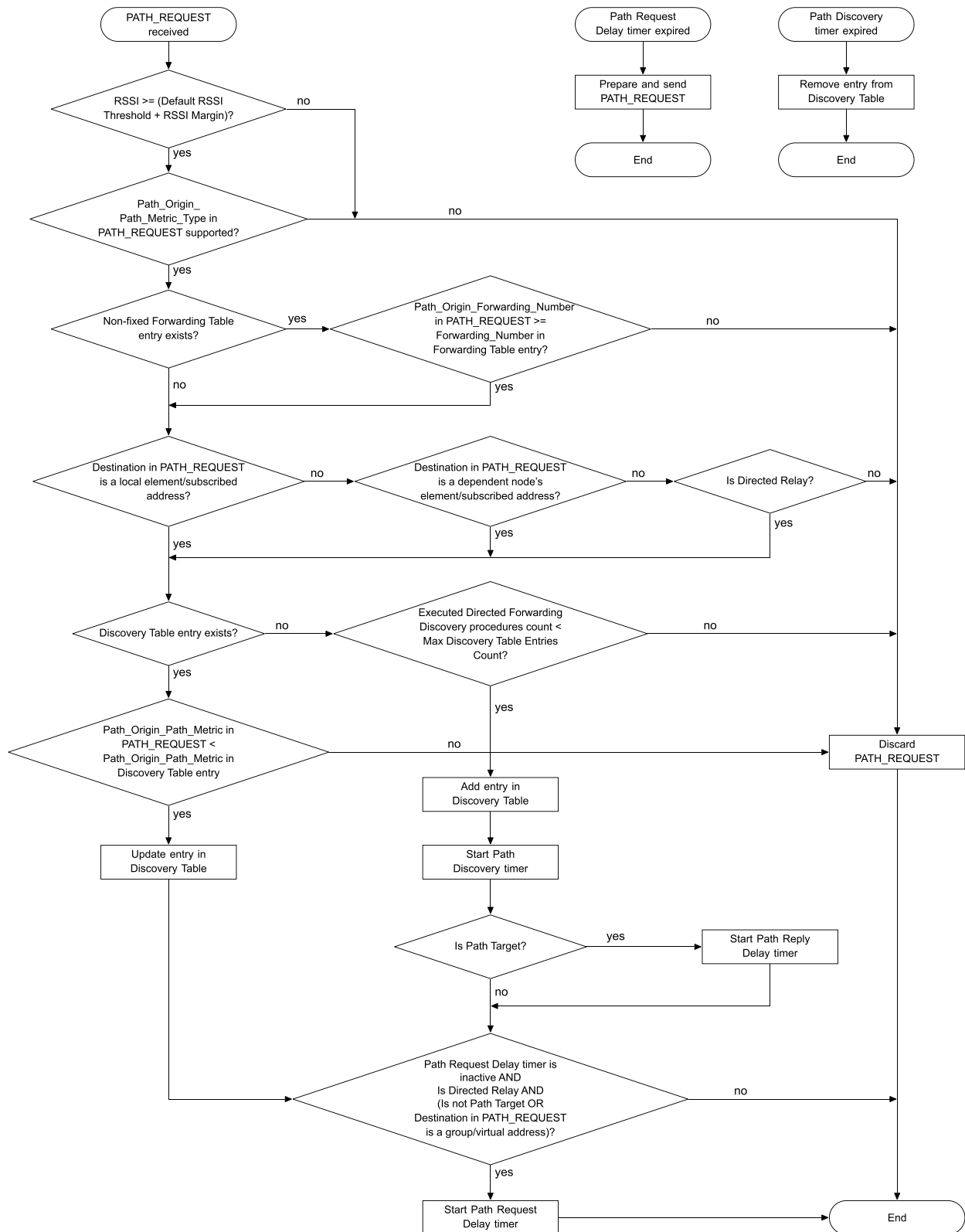


Figure 3.30: Directed Forwarding Discovery procedure

When a Directed Forwarding node receives a PATH_REQUEST message, the node shall check whether to process or discard the message.

The PATH_REQUEST message shall be discarded if any of the following conditions is met:

- The RSSI value measured for the PATH_REQUEST message is less than the sum of the Default RSSI Threshold state value (see Section 4.2.35.1) and the RSSI Margin state value (see Section 4.2.35.2).
- The node does not support the path metric type that is indicated in the PATH_REQUEST message Path-Origin_Path_Metric_Type field (see Section 4.2.27.1).
- A non-fixed Forwarding Table entry corresponding to the PATH_REQUEST message exists (see Section 3.6.8.5.2), and the Path-Origin_Forwarding_Number field value in the message is less than the Forwarding_Number value in the table entry. The comparison between the Path-Origin_Forwarding_Number and the Forwarding_Number shall follow the definitions in Section 3.6.8.4.
- Directed relay, directed friend, and directed proxy functionalities are all disabled, and the Destination field value of the PATH_REQUEST message is not an element address of the node or a group address or a virtual address that the node is subscribed to.
- Directed relay functionality is disabled; and either directed friend functionality, directed proxy functionality, or both are enabled; and the Destination field value of the PATH_REQUEST message is not an element address of the node or of a dependent node; and the Destination field value of the PATH_REQUEST message is not a group address or a virtual address that the node or a dependent node is subscribed to.
- A Discovery Table entry corresponding to the PATH_REQUEST message does not exist (see Section 3.6.8.6.2), and the number of Directed Forwarding Discovery procedures that are being executed in the subnet from which the PATH_REQUEST message is received (i.e., the number of non-empty Discovery Table entries in the subnet) is equal to the Max Discovery Table Entries Count state value (see Section 4.2.28.1) for the subnet.
- A Discovery Table entry corresponding to the PATH_REQUEST message exists, and the Path-Origin_Path_Metric field value in the message is not less than the Path-Origin_Path_Metric value in the table entry.

If the PATH_REQUEST message can be processed, and a Discovery Table entry corresponding to the PATH_REQUEST message does not exist, a new Discovery Table entry shall be added according to Section 3.6.8.6.2, and the corresponding instance of the Path Discovery timer shall be started. The Path Discovery timer initial value shall be set to the duration indicated by the value of the Path_Discovery_Interval field (see Section 4.2.38.3) in the PATH_REQUEST message.

If the PATH_REQUEST message can be processed, and a Discovery Table entry corresponding to the PATH_REQUEST message exists, the Discovery Table entry shall be updated according to Section 3.6.8.6.2. The corresponding instance of the Path Discovery timer is not restarted.

If the PATH_REQUEST message can be processed, the node checks whether it is or is not a Path Target for the PATH_REQUEST message.

The node shall be a Path Target for a PATH_REQUEST message if one of the following conditions is met:

- The Destination field value of the PATH_REQUEST message is either an element address of the node or one of the group addresses or virtual addresses that the node is subscribed to.
- The node has a list of dependent nodes, and the Destination field value of the PATH_REQUEST message is either an element address of one of the dependent nodes or is one of the group addresses or virtual addresses that any of the dependent nodes is subscribed to.

If the node is a Path Target for the PATH_REQUEST message, and the corresponding Discovery Table entry is added, the node shall start the Path Reply Delay timer for this entry. If the Destination field value in the PATH_REQUEST message is a unicast address, the Path Reply Delay timer initial value shall be set to the Path_Reply_Delay value. If the Destination field value in the PATH_REQUEST message is a group address or a virtual address, the Path Reply Delay timer initial value shall be set to the sum of the Path_Reply_Delay value and a random delay of 0 milliseconds to 500 milliseconds.

The node processing the PATH_REQUEST message shall start a Path Request Delay timer for the corresponding Discovery Table entry if all the following conditions are met:

- The Path Request Delay timer for the corresponding Discovery Table entry is inactive.
- The node is a Directed Relay node.
- One of the following conditions is met:
 - The node is not a Path Target for the PATH_REQUEST message.
 - The node is a Path Target for the PATH_REQUEST message, and the Destination field value in the PATH_REQUEST message is a group address or a virtual address.

When a Path Request Delay timer is started, it shall start from the initial value set to a random value in the interval [Path_Request_Delay, Path_Request_Delay + 30 ms].

When a Path Request Delay timer expires, the node shall prepare and send a new PATH_REQUEST message with the following field values:

- The On_Behalf_Of_Dependent_Origin, Path_Origin_Path_Metric_Type, Path_Origin_Path_Lifetime, Path_Discovery_Interval, Path_Origin_Forwarding_Number, Destination, and Path_Origin_Unicast_Addr_Range fields shall have the values of the corresponding Discovery Table entry (see Section 3.6.8.6.2).
- The Path_Origin_Path_Metric field value in the message shall be calculated from the Path_Origin_Path_Metric_Type and Path_Origin_Path_Metric values of the corresponding Discovery Table entry, according to Section 4.2.27.1.

- If the On_Behalf_Of_Dependent-Origin value in the corresponding Discovery Table entry is 1, the Dependent-Origin_Unicast_Addr_Range field shall be derived from the Dependent-Origin value and the Dependent-Origin_Secondary_Elements_Count value of the Discovery Table entry (see Section 3.4.2.2.4).

The Network PDU for the new PATH_REQUEST message shall have the following configuration:

- The DST field shall be set to the all-directed-forwarding-nodes fixed group address (see Section 3.4.2.4).
- The TTL field shall be set to 0.

The node shall send the message using the directed security credentials of the subnet over which the message is sent and shall tag the message with the immutable-credentials tag.

When a Path Discovery timer expires, the corresponding Discovery Table entry shall be removed.

3.6.8.2.3 *Directed Forwarding Establishment*

The Directed Forwarding Establishment procedure shall be executed by a Path Target when a Path Reply Delay timer expires; the procedure also shall be executed by any Directed Forwarding node that receives a PATH_REPLY message.

Figure 3.31 illustrates the Directed Forwarding Establishment procedure.

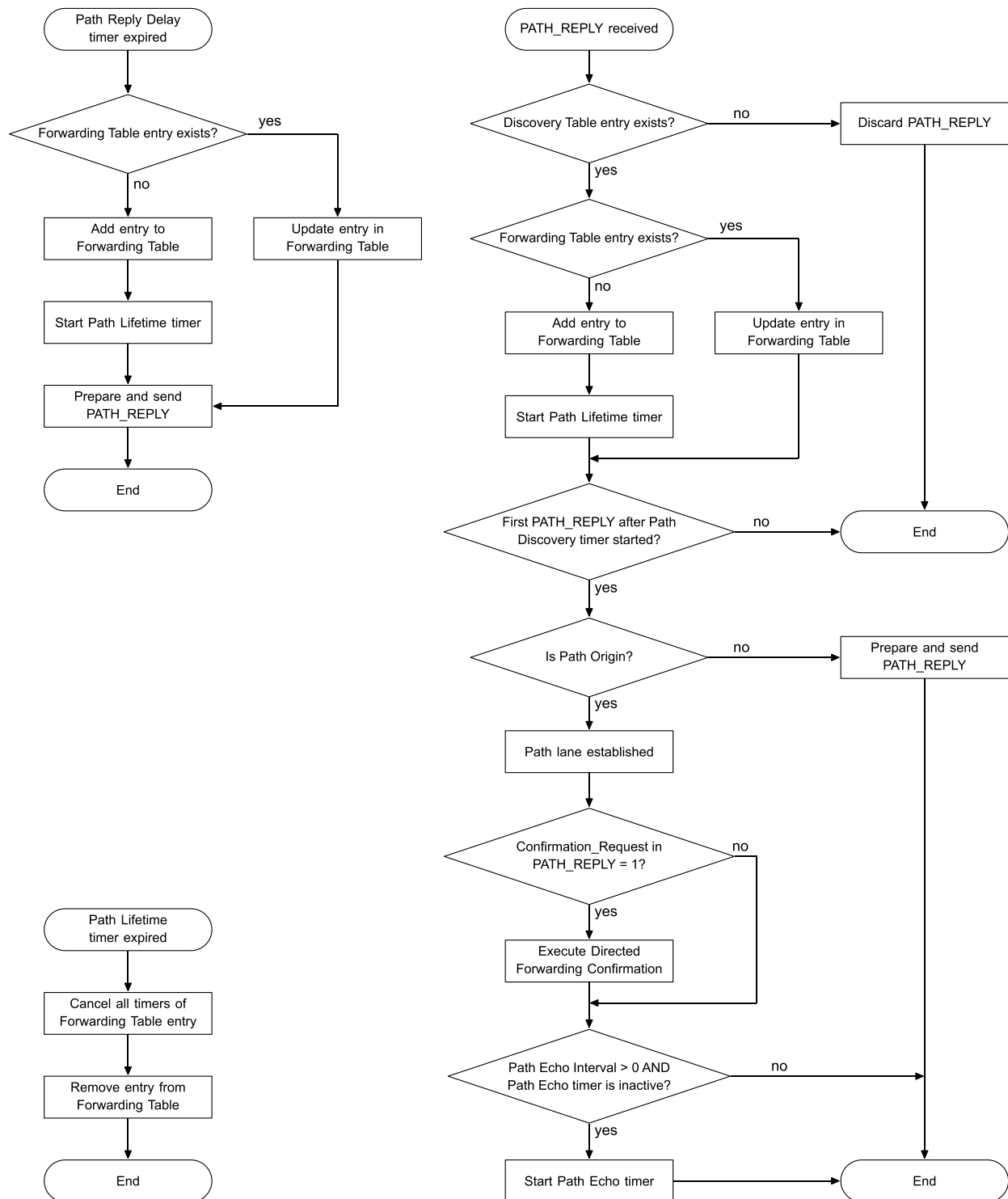


Figure 3.31: Directed Forwarding Establishment procedure

When a Path Reply Delay timer for a Discovery Table entry expires, the Path Target shall update the Forwarding Table state according to Section 3.6.8.5.3. If a new Forwarding Table entry is added, the

node shall also start a Path Lifetime timer for the table entry. The Path Lifetime timer initial value shall be set to the Path_Origin_Path_Lifetime value in the Discovery Table entry.

After updating the Forwarding Table state, the node shall prepare and send a PATH_REPLY message with the following field values:

- If the Destination value in the Discovery Table entry is a unicast address, the Unicast_Destination field shall be set to 1, and the Path_Target_Unicast_Addr_Range field shall be set to the unicast address range (see Section 3.4.2.2.1) of the Path Target. Otherwise, the Unicast_Destination field shall be set to 0 and the Path_Target_Unicast_Addr_Range field shall not be present.
- If the Destination value in the Discovery Table entry is a unicast address, and the PATH_REPLY message is originated on behalf of a dependent node of the Path Target, the On_Behalf_Of_Dependent_Target field shall be set to 1, and the Dependent_Target_Unicast_Addr_Range field shall be set to the unicast address range of the dependent node. Otherwise, the On_Behalf_Of_Dependent_Target field shall be set to 0, and the Dependent_Target_Unicast_Addr_Range field shall not be present.
- If the Destination value in the Discovery Table entry is a unicast address, and a Forwarding Table entry corresponding to a path from the primary element address of the node to the Path_Origin value in the Discovery Table entry does not exist (see Section 3.6.8.5.1), the Confirmation_Request field shall be set to the Two Way Path state value (see Section 4.2.31); otherwise, it shall be set to 0.
- The Path_Origin field shall be set to the Path_Origin value in the Discovery Table entry.
- The Path_Origin_Forwarding_Number field shall be set to the Path_Origin_Forwarding_Number value in the Discovery Table entry.

The Network PDU for the PATH_REPLY message shall have the following configuration:

- The DST field shall be set to the Next_Toward_Path_Origin value in the Discovery Table entry.
- The TTL field shall be set to 0.

The node shall send the message using the directed security credentials of the subnet over which the message is sent and shall tag the message with the immutable-credentials tag.

When a node receives a PATH_REPLY message, the node checks whether to process or discard the message. If a Discovery Table entry that corresponds to the received PATH_REPLY message exists (see Section 3.6.8.6.3), the received PATH_REPLY message shall be processed; otherwise, the PATH_REPLY message shall be discarded.

If the PATH_REPLY message is processed, the node shall update the Forwarding Table state according to Section 3.6.8.5.4. If the Path Lifetime timer for the Forwarding Table entry corresponding to the processed PATH_REPLY message is not running, the node shall start it. The Path Lifetime timer initial value shall be set to the Path_Origin_Path_Lifetime value in the Discovery Table entry.

If this is the first PATH_REPLY processed after the Path Discovery timer of the corresponding Discovery Table entry started, and the Path_Origin field value in the PATH_REPLY message is the primary element

address of the node, then a lane of the path is considered established for the purpose of the Directed Forwarding Initialization procedure (see Section 3.6.8.2.1).

If this is the first PATH_REPLY processed after the Path Discovery timer of the corresponding Discovery Table entry started, and the Path-Origin field value in the PATH_REPLY message is the primary element address of the node, and the Confirmation_Request field value of the PATH_REPLY message is 1, then the node shall execute a Directed Forwarding Confirmation procedure (see Section 3.6.8.2.4).

If this is the first PATH_REPLY processed after the Path Discovery timer of the corresponding Discovery Table entry started, the Path-Origin field value in the PATH_REPLY message is the primary element address of the node, the Unicast_Destination field value in the PATH_REPLY message is 1, and the Unicast Echo Interval state is greater than 0 (see Section 4.2.32.1), then the node checks whether or not a Path Echo timer for the corresponding Forwarding Table entry is running. If the Path Echo timer is not running, the node shall start the Path Echo timer from the initial value set as defined in Section 4.2.32.1.

If this is the first PATH_REPLY processed after the Path Discovery timer of the corresponding Discovery Table entry started, the Path-Origin field value in the PATH_REPLY message is the primary element address of the node, the Unicast_Destination field value in the PATH_REPLY message is 0, and the Multicast Echo Interval state is greater than 0 (see Section 4.2.32.2), then the node checks whether or not a Path Echo timer for the corresponding Forwarding Table entry is running. If the Path Echo timer is not running, the node shall start the Path Echo timer from the initial value set as defined in Section 4.2.32.2.

If this is the first PATH_REPLY processed after the Path Discovery timer of the corresponding Discovery Table entry started, and the Path-Origin field value in the PATH_REPLY message is not the primary element address of the node, then the node shall prepare and send a new PATH_REPLY message in which the field values are copied from the received PATH_REPLY message.

The Network PDU for the new PATH_REPLY message shall have the following configuration:

- The DST field shall be set to the Next_Toward_Path-Origin value in the Discovery Table entry corresponding to the received PATH_REPLY message.
- The TTL field shall be set to 0.

The node shall send the message using the directed security credentials of the subnet over which the message is sent and shall tag the message with the immutable-credentials tag.

When a Path Lifetime timer for a Forwarding Table entry expires, the Forwarding Table entry shall be removed and all timers corresponding to the entry shall be stopped.

3.6.8.2.4 Directed Forwarding Confirmation

The Directed Forwarding Confirmation procedure shall be executed:

- By a node that is the Path Origin indicated by the Path-Origin field in a received PATH_REPLY message with the Confirmation_Request field value of 1 after receiving the first PATH_REPLY message after the Path Discovery timer of the corresponding Discovery Table entry started.
- By any Directed Forwarding node that receives a PATH_CONFIRMATION message.

Figure 3.32 is a flow chart illustrating the Directed Forwarding Confirmation procedure.

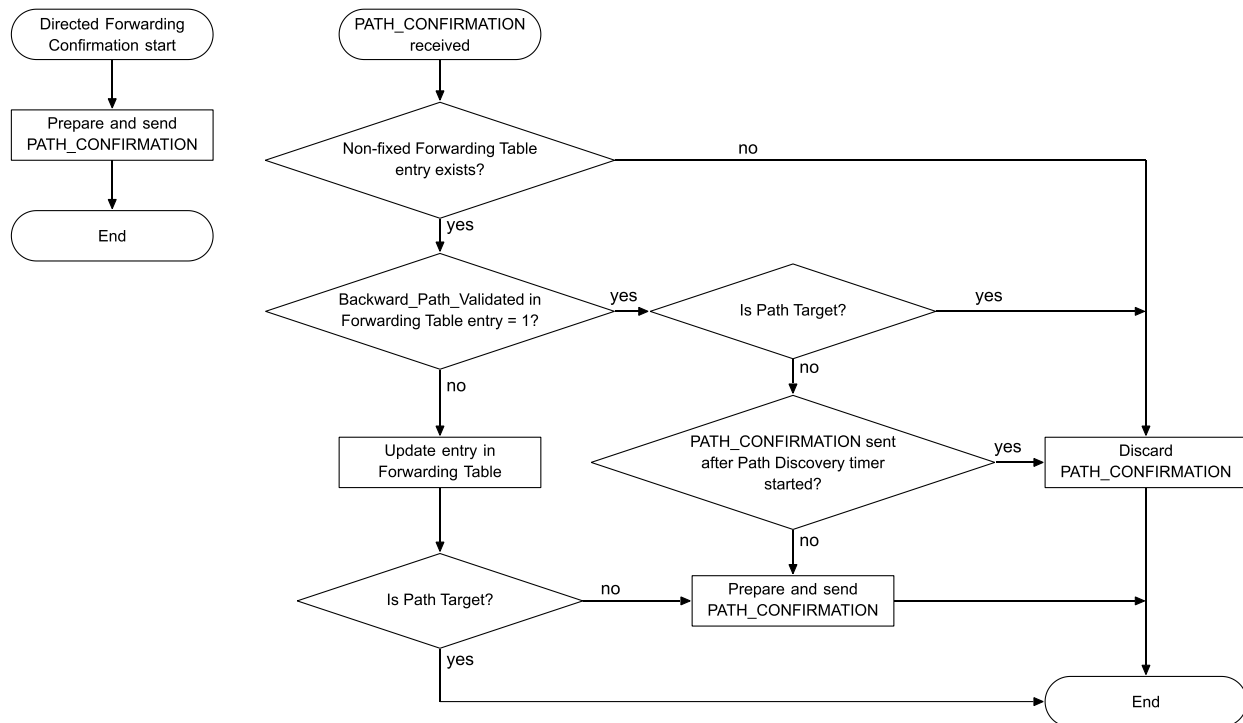


Figure 3.32: Directed Forwarding Confirmation procedure

If the Path Origin receives a PATH_REPLY message with the Confirmation_Request field value of 1, the node shall prepare and send a PATH_CONFIRMATION message with the following field values:

- The Path_Target field shall be set to the Destination value in the Forwarding Table entry corresponding to the received PATH_REPLY message (see Section 3.6.8.5.4).
- The Path-Origin field shall be set to the primary element address of the Path Origin.

The Network PDU for the new PATH_CONFIRMATION message shall have the following configuration:

- The DST field shall be set to the all-directed-forwarding-nodes fixed group address (see Section 3.4.2.4).
- The TTL field shall be set to 0.

The Path Origin shall send the message using the directed security credentials of the subnet over which the message is sent and shall tag the message with the immutable-credentials tag.

When a Directed Forwarding node receives a PATH_CONFIRMATION message, the node checks whether to process or discard the message.

The PATH_CONFIRMATION message shall be discarded if any of the following conditions is met:

- A non-fixed Forwarding Table entry corresponding to the PATH_CONFIRMATION message does not exist (see Section 3.6.8.5.5).
- The Backward_Path_Validated value in the Forwarding Table entry corresponding to the PATH_CONFIRMATION message is 1, and the node is the Path Target indicated by the Path_Target field in the PATH_CONFIRMATION message.
- The node has already sent a PATH_CONFIRMATION message after starting the Path Discovery timer for the Discovery Table entry that has the same Path_Origin and Path_Origin_Forwarding_Number values as the Forwarding Table entry that corresponds to the PATH_CONFIRMATION message. Once true, this condition shall persist until the Path Discovery timer expires.

If the received PATH_CONFIRMATION message can be processed, and the Backward_Path_Validated value in the corresponding Forwarding Table entry is 0, the table entry shall be updated as defined in Section 3.6.8.5.5. Then, if the node is not the Path Target indicated by the Path_Target field in the received PATH_CONFIRMATION message, the node shall prepare and send a new PATH_CONFIRMATION message in which the field values are copied from the received PATH_CONFIRMATION message.

A random delay from 0 to 30 milliseconds should be introduced between preparing and sending the PATH_CONFIRMATION message, to avoid collisions with messages being sent by other nodes at the same time.

The Network PDU for the new PATH_CONFIRMATION message shall have the following configuration:

- The DST field shall be set to the all-directed-forwarding-nodes fixed group address (see Section 3.4.2.4).
- The TTL field shall be set to 0.

The node shall send the message using the directed security credentials of the subnet over which the message is sent and shall tag the message with the immutable-credentials tag.

3.6.8.2.5 Directed Forwarding Dependents Update

The Directed Forwarding Dependents Update procedure is executed in the following contexts:

- The procedure is executed by a Path Origin to notify the nodes along paths from that Path Origin that unicast addresses of its dependent nodes are to be added or removed from the corresponding Forwarding Table entries (see Section 3.6.8.3).
- The procedure is executed by a Path Target to notify the nodes along paths to that Path Target that unicast addresses of its dependent nodes are to be added or removed from the corresponding Forwarding Table entries (see Section 3.6.8.3).

The conditions that trigger the Forwarding Dependents Update procedure in the contexts above are defined in Section 3.6.8.3.



The procedure shall also be executed by any Directed Forwarding node that receives a `DEPENDENT_NODE_UPDATE` message.

[Figure 3.33](#) is a flow chart of the Directed Forwarding Dependents Update procedure.

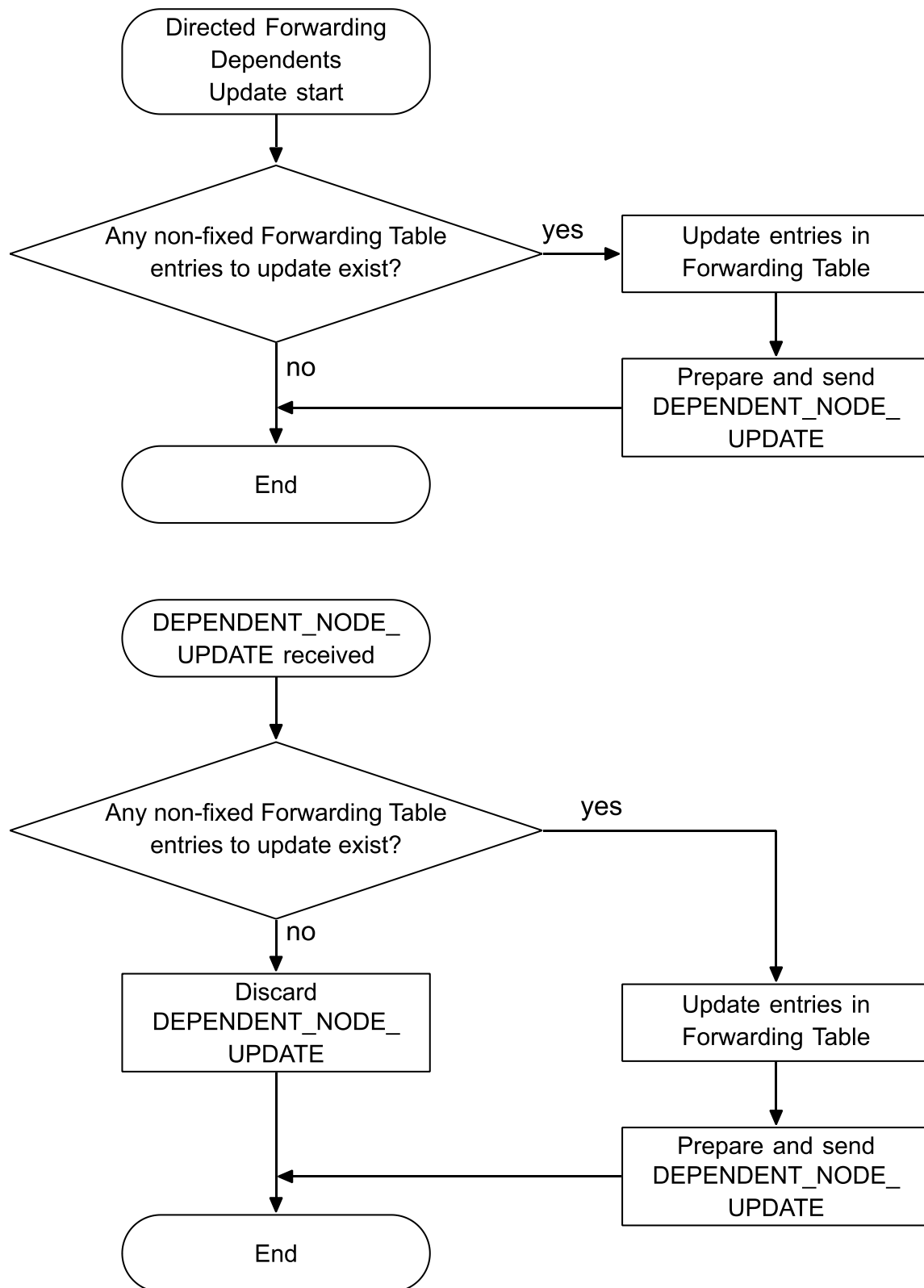


Figure 3.33: Directed Forwarding Dependents Update procedure

When the Directed Forwarding Dependents Update procedure starts, the node shall update all the Forwarding Table entries that correspond to paths with the node as a Path Origin or Path Target, as defined in Section 3.6.8.5.8.

If any Forwarding Table entries are updated, the node shall also prepare and send a `DEPENDENT_NODE_UPDATE` message with the following configuration:

- The Type field shall be set to 1 if one or more addresses of a dependent node are to be added to the Forwarding Table state. If one or more addresses of a dependent node are to be removed from the Forwarding Table state, the Type field shall be set to 0.
- The Path_Endpoint field shall be set to the primary element address of the Path Origin if the node is the Path Origin of any updated Forwarding Table entry. If the node is the Path Target of any updated Forwarding Table entry, the Path_Endpoint field shall be set to the primary element address of the Path Target.
- The Dependent_Node_Unicast_Addr_Range field shall be set to the unicast address range (see Section 3.4.2.2.1) of the dependent node that is to be added to or to be removed from the Forwarding Table state.

The Network PDU for the `DEPENDENT_NODE_UPDATE` message shall have the following configuration:

- The DST field shall be set to the all-directed-forwarding-nodes fixed group address (see Section 3.4.2.4).
- The TTL field shall be set to 0.

The node shall send the message using the directed security credentials of the subnet over which the message is sent and shall tag the message with the immutable-credentials tag.

When a node receives a `DEPENDENT_NODE_UPDATE` message, the node shall check whether the Forwarding Table state contains at least one entry that corresponds to the `DEPENDENT_NODE_UPDATE` message (see Section 3.6.8.5.8).

Each entry that corresponds to the `DEPENDENT_NODE_UPDATE` message shall be updated as defined in Section 3.6.8.5.8. The node shall then send a new `DEPENDENT_NODE_UPDATE` message in which the field values are copied from the received `DEPENDENT_NODE_UPDATE` message.

The Network PDU for the new `DEPENDENT_NODE_UPDATE` message shall have the following configuration:

- The DST field shall be set to the all-directed-forwarding-nodes fixed group address (see Section 3.4.2.4).
- The TTL field shall be set to 0.

The node shall send the message using the directed security credentials of the subnet over which the message is sent and shall tag the message with the immutable-credentials tag.

If there is no Forwarding Table entry that corresponds to the `DEPENDENT_NODE_UPDATE` message, the message shall be discarded.

3.6.8.2.6 *Directed Forwarding Echo*

The Directed Forwarding Echo procedure shall be executed by a Path Origin when an instance of the Path Echo timer expires or by a Path Target when it receives a `PATH_ECHO_REQUEST` message in order to validate a path between the two nodes.

[Figure 3.34](#) is a flow chart of the Directed Forwarding Echo procedure.

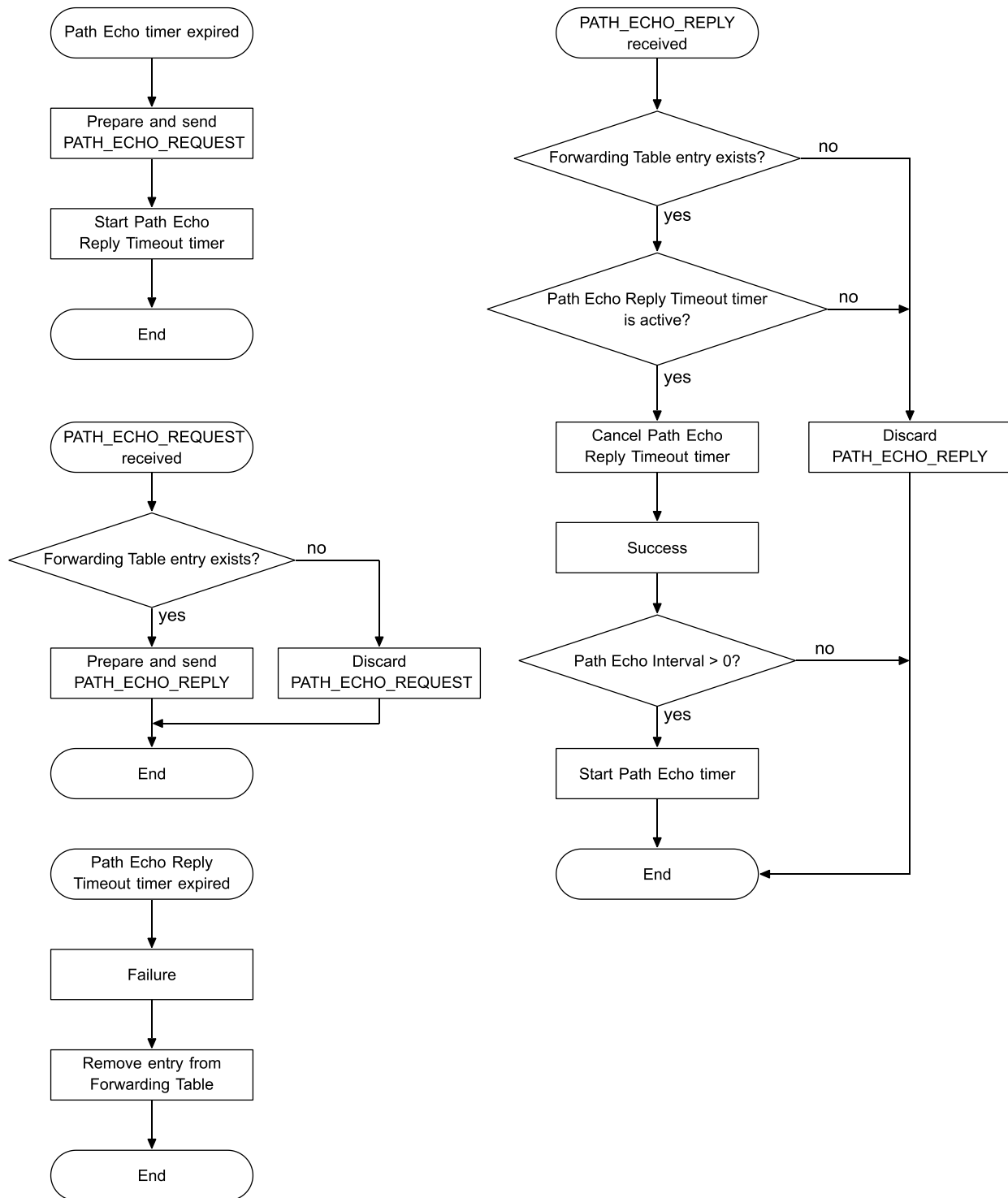


Figure 3.34: Directed Forwarding Echo procedure

When the Path Echo timer corresponding to a Forwarding Table entry expires, the Path Origin shall send a PATH_ECHO_REQUEST message to the Destination value of the Forwarding Table entry in order to validate the path.

The Network PDU for the PATH_ECHO_REQUEST message shall have the following configuration:

- The DST field shall be set to the Destination value of the Forwarding Table entry.
- The TTL field shall be set to 0x7F.

The Path Origin shall send the message using the directed security credentials of the subnet over which the message is sent and shall tag the message with the immutable-credentials tag.

When the message is sent, an instance of the Path Echo Reply Timeout timer corresponding to the Forwarding Table entry shall be started from the initial value indicated by the default value of the Path Discovery Interval state (see Section 4.2.38.3).

When a Path Target receives a PATH_ECHO_REQUEST message, the node shall check whether a Forwarding Table entry that corresponds to the PATH_ECHO_REQUEST message exists as defined in Section 3.6.8.5.6.

If a Forwarding Table entry that corresponds to the PATH_ECHO_REQUEST message does not exist, the message shall be discarded. Otherwise, the Path Target shall prepare and send a PATH_ECHO_REPLY message to the Path Origin with the Destination field set to the DST field value of the PATH_ECHO_REQUEST message.

The Network PDU for the PATH_ECHO_REPLY message shall have the following configuration:

- The DST field shall be set to the primary element address of the Path Origin.
- The TTL field shall be set to 0x7F.

If the Backward_Path_Validated value in the Forwarding Table entry is 0, the Path Target shall send the message using the managed flooding security credentials of the subnet over which the message is sent; otherwise, the Path Target shall send the message using the directed security credentials of the subnet over which the message is sent.

When the Path Origin receives the PATH_ECHO_REPLY message, the node shall check whether a Forwarding Table entry that corresponds to the PATH_ECHO_REPLY message exists as defined in Section 3.6.8.5.7.

If the Forwarding Table entry that corresponds to the PATH_ECHO_REPLY message does not exist; or if the Forwarding Table entry that corresponds to the PATH_ECHO_REPLY message exists, and the corresponding Path Echo Reply Timeout timer is not running, then the PATH_ECHO_REPLY message shall be discarded.

If the Forwarding Table entry that corresponds to the PATH_ECHO_REPLY message exists, and the corresponding Path Echo Reply Timeout timer is running, then the Path Echo Reply Timeout timer shall be stopped, and the Directed Forwarding Echo procedure succeeds. In addition, if the destination is a unicast address and the Unicast Echo Interval state is greater than 0 (see Section 4.2.32.1), or if the destination is a group address or a virtual address and the Multicast Echo Interval state is greater than 0 (see Section 4.2.32.2), then the node shall start an instance of the Path Echo timer for the Forwarding Table entry from the initial value set as defined in Section 4.2.32.

When the Path Echo Reply Timeout timer expires, the Directed Forwarding Echo procedure fails.

When the Directed Forwarding Echo procedure fails, the Forwarding Table entry corresponding to the failed Directed Forwarding Echo procedure shall be deleted and the Forwarding Table Update Identifier state (see Section 4.2.29.1) shall be changed.

3.6.8.2.7 Directed Forwarding Solicitation

The Directed Forwarding Solicitation procedure is executed by a Directed Forwarding node or by a Configuration Manager to solicit the discovery of paths toward unicast addresses, group addresses, or virtual addresses.

The Directed Forwarding Solicitation procedure shall be executed by a Directed Forwarding node in any of the following contexts:

- The Subscription List state of the Directed Forwarding node has been updated with new addresses.
- The Directed Forwarding node receives a PATH_REQUEST_SOLICITATION message.

The procedure is also executed if the Directed Forwarding node is a supporting node, and it is notified of new group addresses or virtual addresses to which its dependent node is subscribed. The conditions that trigger the Directed Forwarding Solicitation procedure in this case are listed in Section 3.6.8.3.

The Directed Forwarding Solicitation procedure should be executed by a Path Target that may no longer be reachable via existing paths (e.g., because the device has been physically moved).

Figure 3.35 is a flow chart of the Directed Forwarding Solicitation procedure.

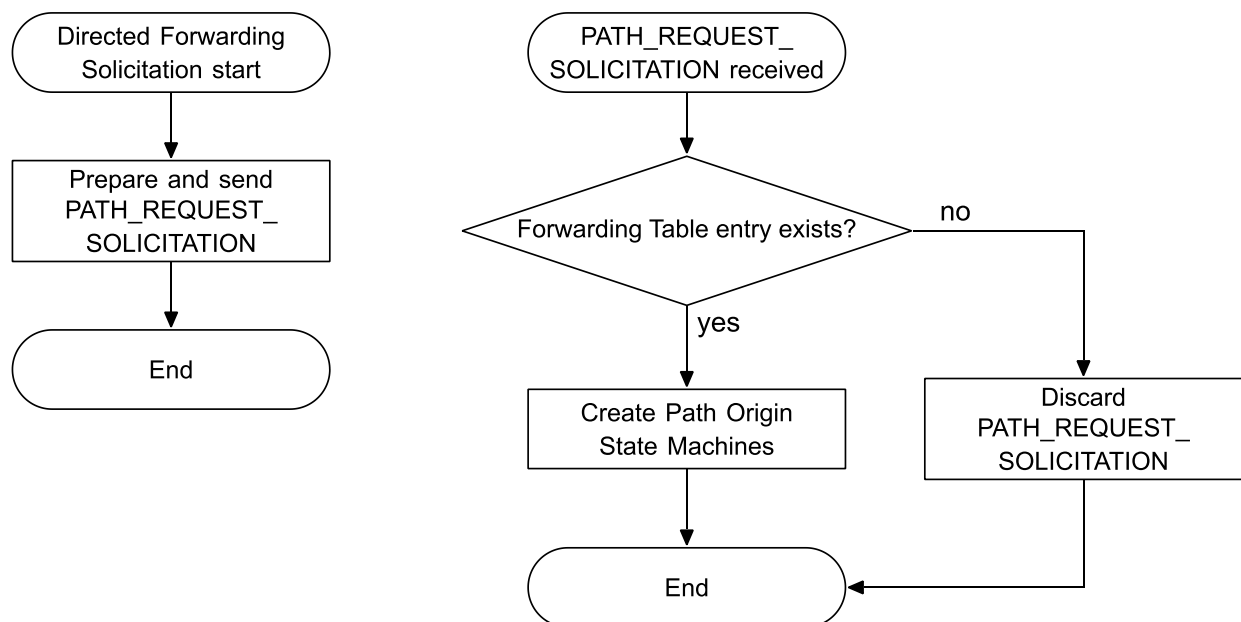


Figure 3.35: Directed Forwarding Solicitation procedure

When the Directed Forwarding Solicitation procedure starts, a `PATH_REQUEST_SOLICITATION` message shall be prepared and sent with the following configuration:

- The `Addr_List` field shall contain a list of unicast addresses, group addresses, and virtual addresses that are intended destinations of the path discovery.

The Network PDU for the `PATH_REQUEST_SOLICITATION` message shall have the following configuration:

- The `DST` field shall be set to the all-directed-forwarding-nodes fixed group address (see Section 3.4.2.4).
- The `TTL` field shall be set to 0x7F.

The Network PDU shall be sent using the directed security credentials of the subnet over which the message is sent and shall be tagged with the immutable-credentials tag.

When a node receives a `PATH_REQUEST_SOLICITATION` message, the node checks whether a non-fixed path entry in the Forwarding Table state that corresponds to the `PATH_REQUEST_SOLICITATION` message exists as defined in Section 3.6.8.5.9.

If a Forwarding Table entry that corresponds to the `PATH_REQUEST_SOLICITATION` message exists, the node shall delete the entry from the Forwarding Table state and shall create a Path Origin State Machine (see Section 4.4.7.2) for each of the destinations in the `Addr_List` field of the `PATH_REQUEST_SOLICITATION` message.

If a Forwarding Table entry that corresponds to the `PATH_REQUEST_SOLICITATION` message does not exist, the message shall be discarded.

3.6.8.2.8 *Examples of Transport Control message exchange and updates to the Forwarding Table state*

Figure 3.36 shows an example of the Transport Control message exchange and updates to the Forwarding Table state that are reported during Directed Forwarding Initialization, Directed Forwarding Discovery, and Directed Forwarding Establishment procedures between two nodes (Nodes A and B). Node A creates a Discovery Table entry for the path from Node A to Node B and transmits a `PATH_REQUEST` message. Node B receives the message, creates a Discovery Table entry for the path, and stores the address of Node A in the `Next_Toward_Path-Origin` field. After the Path Reply Delay timer expires, Node B creates a Forwarding Table entry for the path and transmits a `PATH_REPLY` message to Node A. When Node A receives the `PATH_REPLY` message, it creates a Forwarding Table entry for the path and starts using directed forwarding to transmit messages.

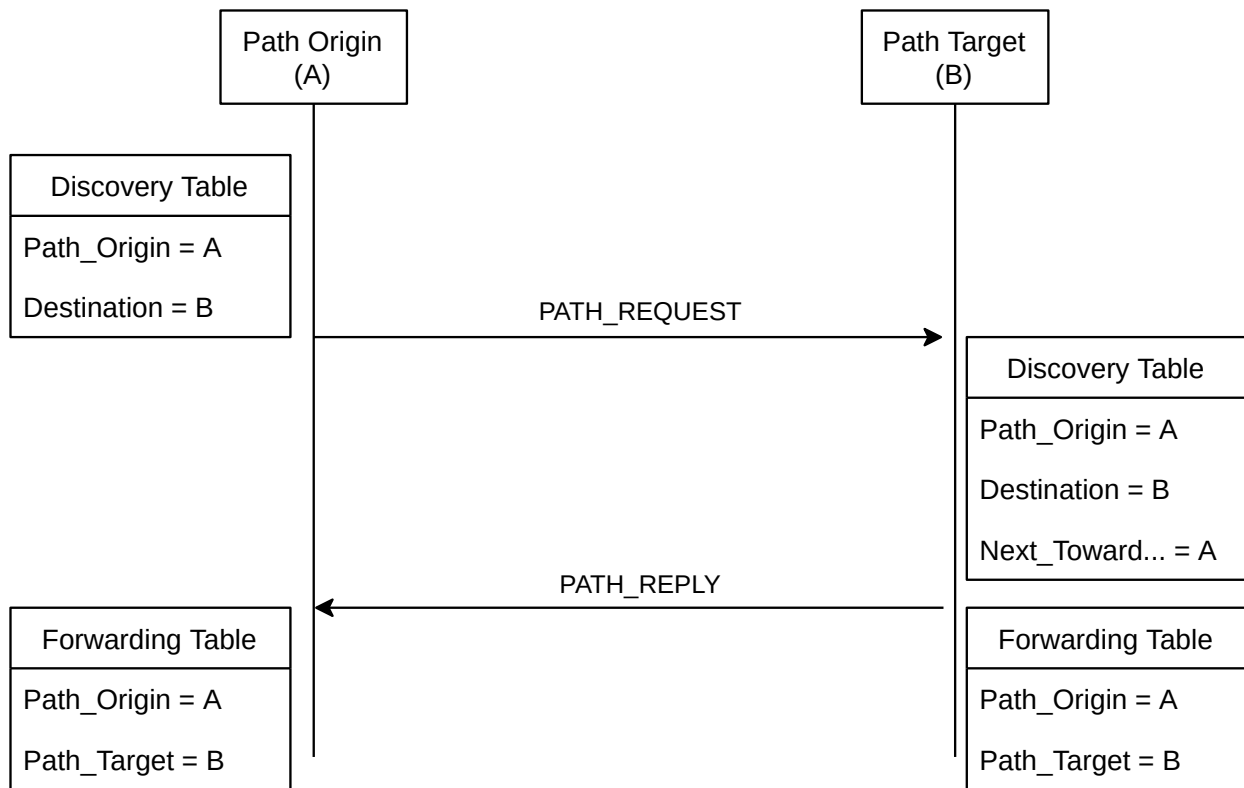


Figure 3.36: Example of directed forwarding procedures during the establishment of a path between two nodes

Similarly, [Figure 3.37](#) shows an example of the exchange of Transport Control messages and updates to Forwarding Table states that are reported during the Directed Forwarding Initialization, Directed Forwarding Discovery, and Directed Forwarding Establishment procedures.

In this example, the Transport Control messages are exchanged among Node A and two other nodes (Nodes C and D), which are subscribed to group address G, via an intermediate node (Directed Relay B) in the following sequence:

1. Node A creates a Discovery Table entry for the path to group address G and transmits a PATH_REQUEST message.
2. Directed Relay B receives the message, creates a Discovery Table entry for the path, stores the address of Node A in the Next_Toward_Path_Origin field, and then forwards the PATH_REQUEST message.
3. Both Node C and Node D receive the PATH_REQUEST message and create a Forwarding Table entry associated with the path to group address G.
4. When the Path Reply Delay timer at Node C expires, Node C creates a Forwarding Table entry and transmits a PATH_REPLY message to Directed Relay B.
5. Directed Relay B creates a Forwarding Table entry and transmits a PATH_REPLY message to Node A.

6. When the Path Reply Delay timer at Node D expires, Node D creates a Forwarding Table entry and transmits a PATH_REPLY message to Directed Relay B.

Directed Relay B does not perform additional actions because it already stores a Forwarding Table entry for the path to group G.

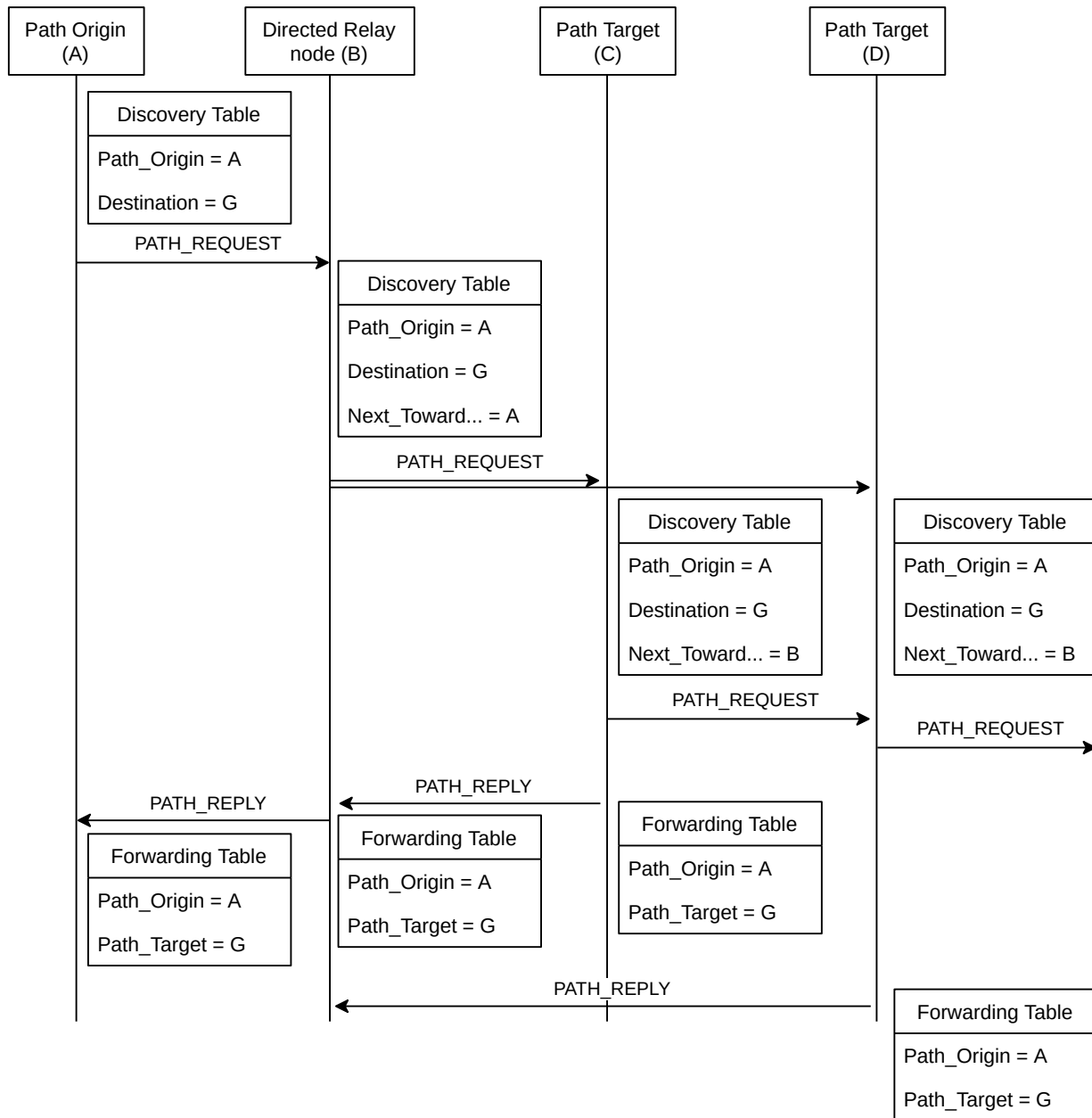


Figure 3.37: Example of directed forwarding procedures for the establishment of paths via a Directed Relay node to two nodes that are subscribed to a group address

3.6.8.3 Node dependence in directed forwarding

In directed forwarding, node dependence is a relationship between two nodes in a subnet, a dependent node and a supporting node. A supporting node performs the following actions on behalf of a dependent node:

- Performs directed forwarding procedures.
- Uses directed security credentials to forward messages generated by the dependent node.

A supporting node can have many dependent nodes.

Node dependence within a subnet includes the following scenarios:

- A Low Power node has established a friendship with a Friend node that is functioning as a Directed Friend node and has sent at least one message using the friendship security credentials. The Low Power node is the dependent node. The Directed Friend node is the supporting node. Node dependence is terminated when the friendship is terminated or when directed friend functionality in the Friend node is disabled.
- A Proxy Client is connected to a Proxy Server, and the Proxy_Client_Type parameter of the connection is Proxy Client, and the Use_Directed parameter of the connection is set to Enabled for the subnet (see Section 6.7.1), and the Proxy Client sent at least one Network PDU via the GATT connection. The Proxy Client is a dependent node. The Directed Proxy node is the supporting node. The node dependence is terminated when the proxy filter type is set to reject list, when directed proxy functionality in the Proxy Server is disabled, or when the Proxy Client disconnects.
- A Proxy Client is connected to a Directed Proxy Server, and the Proxy_Client_Type parameter of the connection is Directed Proxy Client, and the Use_Directed parameter of the connection is set to Enabled for the subnet (see Section 6.7.1). The Directed Proxy Client is a dependent node for the subnet. The Directed Proxy node is the supporting node. The node dependence is terminated when the proxy filter type is set to reject list, when directed proxy functionality in the Proxy Server is disabled, when the Directed Proxy Client sets the Use_Directed parameter of the connection to Disabled, or when the Direct Proxy Client disconnects.
- A node's address is included in the Bridging Table state of a Subnet Bridge in the Address1 field or in the Address2 field (see Section 4.2.42) and directed forwarding functionality is enabled in at least one of the bridged subnets. The nodes identified by the addresses included in the Bridging Table state are dependent nodes (see Section 3.6.8.1.11). The Subnet Bridge node is the supporting node in the subnets where directed forwarding is enabled and only for addresses from which traffic is bridged or toward which traffic is bridged. Node dependence is terminated when the address of the dependent node is removed from the Bridging Table state, or subnet bridge functionality is disabled, or directed forwarding functionality of the Subnet Bridge is disabled in both subnets of the Bridging Table state entry.

Directed forwarding procedures at a supporting node can be initiated when a Network PDU is received from the dependent node, as described in Section 3.4.6.3.



If a Network PDU received from a dependent node is retransmitted by a supporting node (see Section 3.4.6.3), and a path to the destination does not exist (see Section 3.6.8.5.1), the supporting node shall create a Path Origin State Machine (see Section 4.4.7.2).

If the supporting node is a Directed Friend node, the following behaviors apply:

- If one or more group addresses, which are different from the all-directed-forwarding-nodes, all-nodes, and all-relays fixed group addresses, or one or more virtual addresses are added to the Friend Subscription List of the supporting node (see Section 3.6.5.7), then the supporting node shall execute a Directed Forwarding Solicitation procedure (see Section 3.6.8.2.7) for a set of group addresses and virtual addresses that contains addresses that are in the Friend Subscription List and are not in the Destination field of any Forwarding Table entry for the subnet.
- If the node dependence is terminated, and the element addresses of the dependent node are listed in at least one Forwarding Table entry of the supporting node for the subnet, then the supporting node shall execute a Directed Forwarding Dependents Update procedure for the unicast address range of the dependent node.

If the supporting node is a Directed Proxy node, the following behaviors apply:

- If one or more group addresses, which are different from the all-directed-forwarding-nodes, all-nodes, and all-relays fixed group addresses, or one or more virtual addresses are added to the accept list filter of the supporting node (see Section 6.7), then the supporting node shall execute a Directed Forwarding Solicitation procedure (see Section 3.6.8.2.7) for a set of group addresses and virtual addresses that contains addresses that are in the accept list filter and are not in the Destination field of any Forwarding Table entry for the subnet.
- If the node dependence is terminated, and the Proxy_Client_Type parameter of the connection is Directed Proxy Client, and at least one element address of the dependent node is listed in at least one Forwarding Table entry of the supporting node for the subnet, then the supporting node shall execute a Directed Forwarding Dependents Update procedure for the unicast address range of the dependent node for the subnet.
- If the node dependence is terminated, and the Proxy_Client_Type parameter of the connection is Proxy Client, and at least one element address of the dependent node is listed in at least one Forwarding Table entry of the supporting node for the subnet, then the supporting node shall execute a Directed Forwarding Dependents Update procedure for each known unicast address of the dependent node for the subnet.

If the supporting node is a Subnet Bridge node, the following behaviors apply:

- If one or more group addresses, which are different from the all-directed-forwarding-nodes, all-nodes, and all-relays fixed group addresses, or one or more virtual addresses are added to the Bridging Table state of the supporting node (see Section 4.2.42), and directed forwarding functionality is enabled in the subnet identified by NetKeyIndex1 of the added Bridging Table state entries, then the supporting node shall execute a Directed Forwarding Solicitation procedure (see Section 3.6.8.2.7) for a set of group addresses and virtual addresses that contains addresses that are in the Bridging Table state and are not in the Destination field of any Forwarding Table entry in that subnet.

- If the node dependence is terminated, and the element addresses of the dependent node are listed in at least one Forwarding Table entry of the supporting node for the associated subnet, then the supporting node shall execute a Directed Forwarding Dependents Update procedure for the unicast address range of the dependent node.

3.6.8.3.1 *MSC example*

The MSC in [Figure 3.38](#) illustrates message sequencing for a Directed Friend interacting with a Low Power Node. The friendship is established over subnet 1, and the illustrated Directed Friend is configured as a supporting node for the dependent node on the subnet 1.

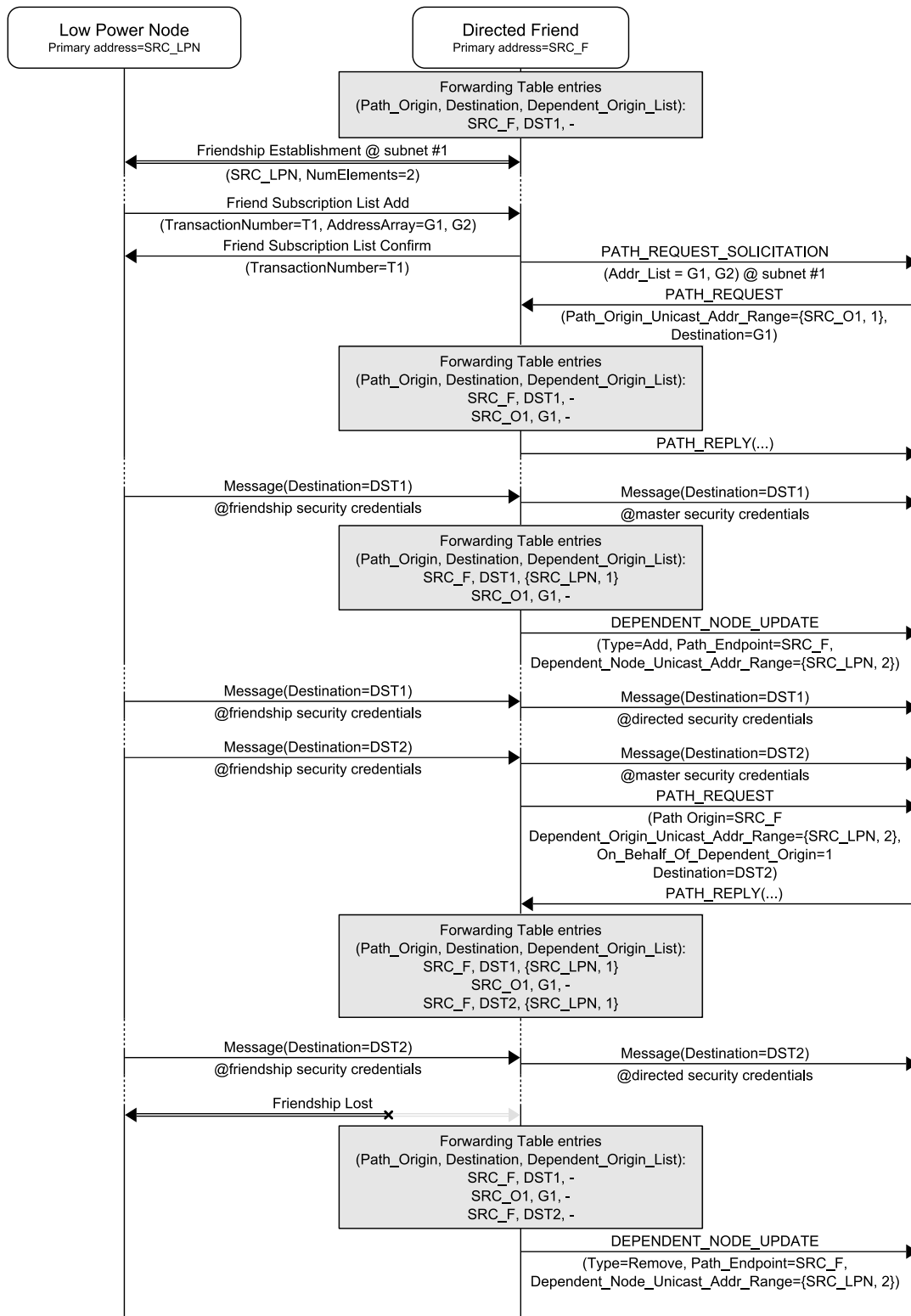


Figure 3.38: Directed Friend and Low Power Node interaction example

3.6.8.4 Forwarding number

Each node manages a forwarding number for each subnet. The forwarding number is used to distinguish Transport Control messages sent by a node in the context of different Directed Forwarding Initialization procedures. This enables discarding of outdated messages (with an old forwarding number), discovery of new lanes (with the same forwarding number), and creation of new paths (with a new forwarding number).

When a node is added to a subnet, the forwarding number for that subnet shall be set to 255. Before a Directed Forwarding Initialization procedure is executed, the forwarding number shall be set according to the following equation:

$$\text{next forwarding number} = (\text{forwarding number} + 1) \text{ modulo } 256$$

The following comparison shall be performed to determine whether one forwarding number is less than, equal to, or greater than another forwarding number:

- Forwarding number a is equal to forwarding number b if a equals b.
- Forwarding number b is greater than forwarding number a if b is one of the 127 numbers succeeding a, taking the wrap into account.
- Forwarding number b is less than forwarding number a if b is one of the 128 numbers preceding a, taking the wrap into account.

For example, forwarding numbers 1 to 127 are greater than forwarding number 0, but forwarding numbers 128 to 255 are less than forwarding number 0.

3.6.8.5 Forwarding Table processing

A Forwarding Table stores pairs of source and destination addresses for each path that a Directed Forwarding node is part of. The node is part of a path if it is the Path Origin, the Path Target, or any intermediate Directed Relay node. The Forwarding Table is represented by the Forwarding Table state (see Section 4.2.29). The Directed Forwarding node stores a separate Forwarding Table for each subnet it belongs to. The Forwarding Table initially is empty when the node is added to a new subnet.

Each Directed Forwarding node filters Network PDUs to be retransmitted using the directed security credentials based on its Forwarding Table state (see Section 3.4.6.3). The Forwarding Table state can be updated by directed forwarding procedures that the node executes (see Section 3.6.8.1) and by configuration messages received from a Configuration Manager (see Section 4.3.5).

Section 3.6.8.5.1 defines the correspondence between a Forwarding Table entry and a path. This definition is valid for both fixed paths (i.e., the Fixed_Path value in the Forwarding Table entry is 1) and non-fixed paths (i.e., the Fixed_Path value in the Forwarding Table entry is 0).

Sections 3.6.8.5.2 through 3.6.8.5.9 describe how the processing of Transport Control messages and expiration of timers in directed forwarding results in the addition, modification, or deletion of non-fixed path entries in the Forwarding Table state.

3.6.8.5.1 Forwarding Table entry corresponding to a path

A Forwarding Table entry corresponds to a path from a source address to a destination address if one of the following conditions is met:

- The source address is equal to any address in the range specified in the Originator Address Range entry for any row in [Table 3.58](#), and the destination address is equal to any address in the range specified in the Destination Address Range entry for that row.
- The Backward_Path_Validated value in the table entry is 1, and the source address is equal to any address in the range specified in the Destination Address Range entry for any row in [Table 3.58](#), and the destination address is equal to any address in the range specified in the Originator Address Range entry for that row.

Moreover, a path shall exist for a destination address that is the all-directed-forwarding-nodes fixed group address (see [Section 3.4.2.4](#)).

Originator Address Range	Destination Address Range
Path-Origin to (Path-Origin + Path-Origin_Secondary_Elements_Count), where: • Path-Origin and Path-Origin_Secondary_Elements_Count are field values of the table entry	Destination to (Destination + Path_Target_Secondary_Elements_Count), where: • Destination and Path_Target_Secondary_Elements_Count are field values of the table entry
Path-Origin to (Path-Origin + Path-Origin_Secondary_Elements_Count), where: • Path-Origin and Path-Origin_Secondary_Elements_Count are field values of the table entry	DependentTarget to (DependentTarget + DependentTargetSecondaryElementsCount), where: • DependentTarget is an address in the Dependent_Target_List field of the table entry • DependentTargetSecondaryElementsCount is the value of the Dependent_Target_Secondary_Elements_Count_List field of the table entry that corresponds to the DependentTarget

Originator Address Range	Destination Address Range
DependentOrigin to (DependentOrigin + DependentOriginSecondaryElementsCount), where: - DependentOrigin is an address in the Dependent_Origin_List field of the table entry - DependentOriginSecondaryElementsCount is the value of the Dependent_Origin_Secondary_Elements_Count_List field of the table entry that corresponds to the DependentOrigin	Destination to (Destination + Path_Target_Secondary_Elements_Count), where: Destination and Path_Target_Secondary_Elements_Count are field values of the table entry
DependentOrigin to (DependentOrigin + DependentOriginSecondaryElementsCount), where: - DependentOrigin is an address in the Dependent_Origin_List field of the table entry - DependentOriginSecondaryElementsCount is the value of the Dependent_Origin_Secondary_Elements_Count_List field of the table entry that corresponds to the DependentOrigin	DependentTarget to (DependentTarget + DependentTargetSecondaryElementsCount), where: - DependentTarget is an address in the Dependent_Target_List field of the table entry - DependentTargetSecondaryElementsCount is the value of the Dependent_Target_Secondary_Elements_Count_List field of the table entry that corresponds to the DependentTarget

Table 3.58: Address ranges used to find correspondence of a Forwarding Table entry to a path

3.6.8.5.2 *PATH_REQUEST message received*

When a PATH_REQUEST message (see Section 3.6.5.11) is received, the node derives the primary element address of the Path Origin from the Path_Origin_Unicast_Addr_Range field (see Section 3.4.2.2.4) in the message, and the node checks whether the Forwarding Table contains a non-fixed entry corresponding to the PATH_REQUEST message.

A Forwarding Table entry corresponds to the PATH_REQUEST message when all of the following conditions are met:

- The Fixed_Path value of the Forwarding Table entry is 0.
- The primary element address of the Path Origin is equal to the Path_Origin value of the Forwarding Table entry.
- The Destination field value of the message is equal to any address in the range specified in the Destination Address Range entry in the first or second row of Table 3.58.

The Forwarding Table state is not updated when a PATH_REQUEST message is received.

3.6.8.5.3 *Path Reply Delay timer expired*

When a Path Reply Delay timer for a Discovery Table entry expires, the node checks whether the Forwarding Table contains a non-fixed entry corresponding to the Discovery Table entry.

A Forwarding Table entry corresponds to a Discovery Table entry when all of the following conditions are met:

- The Fixed_Path value of the Forwarding Table entry is 0.
- The Path-Origin value of the Discovery Table entry is equal to the Path-Origin value of the Forwarding Table entry.
- The Path-Origin_Forwarding_Number value of the Discovery Table entry is equal to the Forwarding_Number value of the Forwarding Table entry.

No existing Forwarding Table entry. If the Forwarding Table does not already contain an entry that corresponds to the Discovery Table entry, then the Forwarding Table Update Identifier shall be changed (see Section 4.2.29.1), and a new entry shall be added to the Forwarding Table state.

The new Forwarding Table entry shall have the following values:

- Fixed_Path, Backward_Path_Validated, and Path_Not_Ready: Set to 0.
- Path-Origin, Path-Origin_Secondary_Elements_Count, Forwarding_Number, and Bearer_Toward_Path-Origin: Set to the corresponding Path-Origin, Path-Origin_Secondary_Elements_Count, Forwarding_Number, and Bearer_Toward_Path-Origin values of the Discovery Table entry.
- Dependent-Origin_List: Initialized with the Dependent-Origin value of the Discovery Table entry if the Dependent-Origin value is not the unassigned address; otherwise, the Dependent-Origin_List is empty.
- Dependent-Origin_Secondary_Elements_Count_List: Initialized with the Dependent-Origin_Secondary_Elements_Count value of the Discovery Table entry if the Dependent-Origin value is not the unassigned address; otherwise, the Dependent-Origin_Secondary_Elements_Count_List is empty.
- Destination: Set to the Destination value of the Discovery Table entry if it is a group address or a virtual address; otherwise, set to the primary element address of the Path Target.
- Path_Target_Secondary_Elements_Count: Set to the number of secondary element addresses of the Path Target if the Destination value of the Discovery Table entry is a unicast address; otherwise, set to 0.
- Dependent_Target_List: Initialized with the primary element address of a dependent node if the Destination value of the Discovery Table entry is an element address of that dependent node; otherwise, the Dependent_Target_List is empty.

- **Dependent_Target_Secondary_Elements_Count_List:** Initialized with the number of secondary element addresses of a dependent node if the Destination value of the Discovery Table entry is an element address of that dependent node; otherwise, the Dependent_Target_Secondary_Elements_Count_List is empty.
- **Bearer_Toward_Path_Target:** Set to the unassigned bearer index (see Section 4.3.1.4).
- **Lane_Counter:** Set to 1.

If the Forwarding Table contains other non-fixed path entries with the same Path_Origin and Destination values as in the added Forwarding Table entry, these other entries shall be removed.

Existing Forwarding Table entry. If the Forwarding Table already contains an entry that corresponds to the Discovery Table entry, then the Forwarding Table Update Identifier shall be changed (see Section 4.2.29.1), and the Forwarding Table entry shall be updated with the following values:

- **Bearer_Toward_Path_Origin:** The bit representing the bearer in the Bearer_Toward_Path_Origin value of the Discovery Table entry shall be set to 1 (see Section 4.3.1.4). All the other bits shall be left unchanged.
- **Lane_Counter:** Incremented by 1.

3.6.8.5.4 *PATH_REPLY message received*

When a PATH_REPLY message (see Section 3.6.5.12) is received, the node checks whether an existing Forwarding Table entry corresponds to the PATH_REPLY message.

A Forwarding Table entry corresponds to the PATH_REPLY message when both of the following conditions are met:

- The Path_Origin field value in the message is equal to the Path_Origin value in the table entry.
- The Path_Origin_Forwarding_Number field value in the message is equal to the Forwarding_Number value in the table entry.

No existing Forwarding Table entry. If the Forwarding Table does not already contain an entry that corresponds to the PATH_REPLY message, the Forwarding Table Update Identifier shall change (see Section 4.2.29.1), and a new entry shall be added to the Forwarding Table state, based on values in the Discovery Table entry that corresponds to the PATH_REPLY message (see Section 3.6.8.6.3).

The new Forwarding Table entry shall have the following values:

- **Fixed_Path and Backward_Path_Validated:** Set to 0.
- **Path_Not_Ready:** Set to 1 if the node is the Path Origin and the Unicast_Destination field value in the PATH_REPLY message is 0; otherwise, set to 0.
- **Path_Origin, Path_Origin_Secondary_Elements_Count, and Forwarding_Number:** Set to the Path_Origin, Path_Origin_Secondary_Elements_Count, and Path_Origin_Forwarding_Number values of the Discovery Table entry.

- **Bearer_Toward_Path-Origin:** Set to the **Bearer_Toward_Path-Origin** value of the Discovery Table entry if the node is not the Path Origin; otherwise, set to the unassigned bearer index (see Section 4.3.1.4).
- **Dependent-Origin-List:** Initialized with the **Dependent-Origin** value of the Discovery Table entry if that value is not the unassigned address; otherwise, **Dependent-Origin-List** is empty.
- **Dependent-Origin-Secondary-Elements-Count-List:** Initialized with the **Dependent-Origin-Secondary-Elements-Count** value of the Discovery Table entry if the **Dependent-Origin** value in the Discovery Table entry is not the unassigned address; otherwise, **Dependent-Origin-Secondary-Elements-Count-List** is empty.
- **Destination:** Set to the **Destination** value of the Discovery Table entry if the **Destination** value is a group address or virtual address; otherwise, **Destination** is set to the primary element address derived from the **Path_Target_Unicast_Addr_Range** field in the **PATH_REPLY** message (see Section 3.4.2.2.4).
- **Path_Target_Secondary-Elements-Count:** Set to the **Path_Target_Secondary-Elements-Count** value derived from the **Path_Target_Unicast_Addr_Range** field in the **PATH_REPLY** message (see Section 3.4.2.2.4) if the **Unicast_Destination** field value in the **PATH_REPLY** message is 1; otherwise, set to 0.
- **Dependent_Target-List:** Initialized with the **Dependent_Target** value derived from the **Dependent_Target_Unicast_Addr_Range** field in the **PATH_REPLY** message (see Section 3.4.2.2.4) if the **Unicast_Destination** and **On_Behalf_Of_Dependent_Target** field values in the **PATH_REPLY** message both are 1; otherwise, **Dependent_Target-List** is empty.
- **Dependent_Target_Secondary-Elements-Count-List:** Initialized with the **Dependent_Target_Secondary-Elements-Count** value derived from the **Dependent_Target_Unicast_Addr_Range** field in the **PATH_REPLY** message (see Section 3.4.2.2.4) if the **Unicast_Destination** and **On_Behalf_Of_Dependent_Target** field values in the **PATH_REPLY** message both are 1; otherwise, **Dependent_Target_Secondary-Elements-Count-List** is empty.
- **Bearer_Toward_Path_Target:** The bit representing the bearer from which the Network PDU of the **PATH_REPLY** message was received shall be set to 1 (see Section 4.3.1.4). All other bits shall be set to 0.
- **Lane_Counter:** Set to 1.

If the Forwarding Table contains other non-fixed path entries with the same **Path-Origin** and **Destination** values as in the added Forwarding Table entry, these other entries shall be removed.

Existing Forwarding Table entry. If the Forwarding Table already contains an entry that corresponds to the **PATH_REPLY** message, the Forwarding Table Update Identifier shall change (see Section 4.2.29.1), and the entry shall be updated based on values in the Discovery Table entry that corresponds to the **PATH_REPLY** message (see Section 3.6.8.6.3).

The updated Forwarding Table entry shall have the following values:

- **Bearer_Toward_Path_Origin:** If the node is not the Path Origin, the bit representing the bearer in the Bearer_Toward_Path_Origin value of the Discovery Table entry shall be set to 1. All other bits shall be left unchanged.
- **Bearer_Toward_Path_Target:** The bit representing the bearer from which the Network PDU of the PATH_REPLY message was received shall be set to 1. All other bits shall be left unchanged.
- **Lane_Counter:** Shall be incremented by 1 if this is the first PATH_REPLY received after the Path Discovery timer of the corresponding Discovery Table entry started; otherwise, left unchanged.

3.6.8.5.5 *PATH_CONFIRMATION message received*

When a PATH_CONFIRMATION message (see Section 3.6.5.13) is received, the node checks whether an existing non-fixed Forwarding Table entry corresponds to the PATH_CONFIRMATION message.

A Forwarding Table entry corresponds to a PATH_CONFIRMATION message when the following conditions are met:

- The Fixed_Path value of the table entry is 0.
- The Path_Origin field value in the message is equal to the Path_Origin value in the table entry.
- The Path_Target field value in the message is equal to the Destination value in the table entry.

If a Forwarding Table entry that corresponds to the PATH_CONFIRMATION message exists, the Backward_Path_Validated field in the table entry shall be set to 1, and the Forwarding Table Update Identifier shall change (see Section 4.2.29.1).

3.6.8.5.6 *PATH_ECHO_REQUEST message received*

When a PATH_ECHO_REQUEST message (see Section 3.6.5.14) is received, the node checks whether an existing non-fixed Forwarding Table entry corresponds to the PATH_ECHO_REQUEST message.

A Forwarding Table entry corresponds to a PATH_ECHO_REQUEST message when the following conditions are met:

- The Fixed_Path value of the table entry is 0.
- The SRC field value of the Network PDU of the message is equal to the Path_Origin value in the table entry.
- The DST field value of the Network PDU of the message is equal to the Destination value in the table entry.

The Forwarding Table state is not updated when a PATH_ECHO_REQUEST message is received.

3.6.8.5.7 *PATH_ECHO_REPLY message received*

When a PATH_ECHO_REPLY message (see Section 3.6.5.15) is received, the node checks whether an existing non-fixed Forwarding Table entry corresponds to the PATH_ECHO_REPLY message.

A Forwarding Table entry corresponds to a PATH_ECHO_REPLY message when the following conditions are met:

- The Fixed_Path value of the table entry is 0.
- The DST field value in the Network PDU of the message is equal to the Path-Origin value in the table entry.
- The Destination field value of the message is equal to the Destination value in the table entry.
- The message was secured using the directed security material and the Backward_Path_Validated value in the table entry is 1; or the message was secured using the managed flooding security material and the Backward_Path_Validated value in the table entry is 0.

The Forwarding Table state is not updated when a PATH_ECHO_REPLY message is received.

3.6.8.5.8 *DEPENDENT_NODE_UPDATE message received*

When a DEPENDENT_NODE_UPDATE message is received (see Section 3.6.5.16), the node derives the primary element address and the number of secondary elements of a dependent node from the Dependent_Node_Unicast_Addr_Range field (see Section 3.4.2.2.4) in the message. (In this discussion, the primary element address and the number of secondary elements of the dependent node are referred to as the DependentNode and the DependentNodeSecondaryElementsCount, respectively.)

Then, the node checks whether one or more of the non-fixed existing Forwarding Table entries correspond to the DEPENDENT_NODE_UPDATE message.

A non-fixed Forwarding Table entry corresponds to a DEPENDENT_NODE_UPDATE message if one of the following conditions is met:

- The Path_Endpoint field value in the message is equal to the Path-Origin value of the table entry, the Type field value in the message is 0, and the DependentNode is included in the Dependent-Origin_List field of the table entry.
- The Path_Endpoint field value in the message is equal to the Path-Origin value of the table entry, the Type field value in the message is 1, and the DependentNode is not included in the Dependent-Origin_List field of the table entry.
- The Path_Endpoint field value in the message is equal to the Destination value of the table entry, the Type field value in the message is 0, and the DependentNode is included in the Dependent-Target_List field of the table entry.
- The Path_Endpoint field value in the message is equal to the Destination value of the table entry, the Type field value in the message is 1, the Backward_Path_Validated value of the table entry is 1, and the DependentNode is not included in the Dependent-Target_List field of the table entry.

Path endpoint is the Path Origin: If the Path_Endpoint field value in the message is equal to the Path-Origin value of the table entry, and the Type field value in the message is 0, then the node shall remove the DependentNode from the Dependent-Origin_List field of the table entry, and shall remove the corresponding value from the Dependent-Origin_Secondary_Elements_Count_List field of the table entry.

If the Path_Endpoint field value in the message is equal to the Path-Origin value of the table entry, and the Type field value in the message is 1, then the node shall add the DependentNode to the Dependent-Origin_List field of the table entry, and shall add the DependentNodeSecondaryElementsCount to the Dependent-Origin_Secondary_Elements_Count_List field of the table entry.

Path endpoint is the Path Target: If the Path_Endpoint field value in the message is equal to the Destination value of the table entry, and the Type field value in the message is 0, then the node shall remove the DependentNode from the Dependent-Target_List field of the table entry, and shall remove the corresponding value from the Dependent-Target_Secondary_Elements_Count_List field of the table entry.

If the Path_Endpoint field value in the message is equal to the Destination value of the table entry, and the Backward_Path_Validated value of the table entry is 1, and the Type field value in the message is 1, then the node shall add the DependentNode to the Dependent-Target_List field of the table entry, and shall add the DependentNodeSecondaryElementsCount to the Dependent-Target_Secondary_Elements_Count_List field of the table entry.

If the Forwarding Table is updated when a DEPENDENT_NODE_UPDATE message is received, the Forwarding Table Update Identifier shall change (see Section 4.2.29.1).

3.6.8.5.9 *PATH_REQUEST_SOLICITATION message received*

When a PATH_REQUEST_SOLICITATION message (see Section 3.6.5.17) is received, the node checks whether an existing non-fixed path entry in the Forwarding Table state corresponds to the PATH_REQUEST_SOLICITATION message. If a non-fixed path entry in the Forwarding Table state corresponding to the PATH_REQUEST_SOLICITATION message exists, the node shall delete the entry. If any of the Forwarding Table entries are deleted, the Forwarding Table Update Identifier shall change (see Section 4.2.29.1).

A Forwarding Table entry corresponds to the PATH_REQUEST_SOLICITATION message when the following conditions are met:

- The Fixed_Path value in the table entry is equal to 0b0.
- The Path-Origin value in the table entry is equal to the primary element address of the node (i.e., the node is the Path Origin of the path corresponding to the table entry).
- The Destination value in the table entry is equal to at least one of the addresses in the Addr_List field of the message.

3.6.8.6 Discovery Table processing

Each node manages a Discovery Table containing a list of temporary entries created during a path discovery. If the node belongs to multiple subnets, then a Discovery Table is maintained for each subnet.

The format of Discovery Table entries is defined in [Table 3.59](#).

Field	Description
Path_Origin	Primary element address of the Path Origin
Path_Origin_Secondary_Elements_Count	Number of secondary elements of the Path Origin
Dependent_Origin	Primary element address of a dependent node of the Path Origin, if the path is discovered on behalf of that dependent node; otherwise, the unassigned address.
Dependent_Origin_Secondary_Elements_Count	Number of secondary elements of the dependent node of the Path Origin, if the path is discovered on behalf of that dependent node; otherwise, 0.
Path_Origin_Forwarding_Number	Forwarding number generated by the Path Origin for the path discovery
Path_Origin_Path_Metric_Type	Path metric type used to calculate the Path_Origin_Path_Metric field value
Path_Origin_Path_Lifetime	Path lifetime specified by the Path Origin
Path_Discovery_Interval	Path discovery interval specified by the Path Origin
Path_Origin_Path_Metric	Path metric value for the path
Destination	The unicast address, group address, or virtual address of the destination
Next_Toward_Path_Origin	Primary element address of the node selected as the next hop toward the Path Origin
Bearer_Toward_Path_Origin	Index of the bearer from which a PATH_REQUEST was received.

Table 3.59: Fields of a Discovery Table entry

The Discovery Table initially is empty. A Path Origin updates its Discovery Table when a Directed Forwarding Initialization procedure is executed. A Path Target or a Directed Relay node updates its Discovery Table when a PATH_REQUEST message is received and processed.

3.6.8.6.1 *Directed Forwarding Initialization procedure executed*

When a Path Origin executes a Directed Forwarding Initialization procedure, the node shall add a new entry with the following format to the Discovery Table:

- Path_Origin: Set to the primary element address of the node.
- Path_Origin_Secondary_Elements_Count: Set to the number of secondary elements of the node.
- Dependent_Origin: Set to the primary element address of the dependent node of the Path Origin if the Directed Forwarding Initialization procedure is executed on behalf of that dependent node; otherwise, set to the unassigned address.
- Dependent_Origin_Secondary_Elements_Count: Set to the number of secondary elements of the dependent node of the Path Origin if the Directed Forwarding Initialization procedure is executed on behalf of that dependent node; otherwise, set to 0.
- Path_Origin_Forwarding_Number: Set to the forwarding number (see Section 3.6.8.4) of the Path Origin.
- Destination: Set to the destination of the path.
- Path_Origin_Path_Metric_Type: Set to the Path Metric Type state value (see Section 4.2.27.1).
- Path_Origin_Path_Lifetime: Set to the time interval derived from the Path Lifetime state (see Section 4.2.27.2).
- Path_Discovery_Interval: Set to the Path Discovery Interval state (see Section 4.2.38.3).
- Path_Origin_Path_Metric: Set to 0.
- Next_Toward_Path_Origin: Set to the unassigned address.
- Bearer_Toward_Path_Origin: Set to the unassigned bearer index (see Section 4.3.1.4).

3.6.8.6.2 *PATH_REQUEST message received*

When a PATH_REQUEST message (see Section 3.6.5.11) is received, the node checks whether an existing Discovery Table entry corresponds to the PATH_REQUEST message.

A Discovery Table entry corresponds to the PATH_REQUEST message if both of the following conditions are met:

- The Path_Origin field value in the message is equal to the Path_Origin value of the table entry.
- The Path_Origin_Forwarding_Number field value in the message is equal to the Path_Origin_Forwarding_Number value of the table entry.

If no existing Discovery Table entry corresponds to the PATH_REQUEST message, a new entry shall be added to the Discovery Table in the following format:

- Path-Origin: Set to the Path-Origin value derived from the Path-Origin_Unicast_Addr_Range field value in the PATH_REQUEST message (see Section 3.4.2.2.4).
- Path-Origin_Secondary_Elements_Count: Set to the Path-Origin_Secondary_Elements_Count value derived from the Path-Origin_Unicast_Addr_Range field value in the PATH_REQUEST message (see Section 3.4.2.2.4).
- Dependent-Origin: Set to the Dependent-Origin value derived from the Dependent-Origin_Unicast_Addr_Range field value in the PATH_REQUEST message (see Section 3.4.2.2.4) if the On_Behalf_Of_Dependent-Origin field value in the message is 1; otherwise, set to the unassigned address.
- Dependent-Origin_Secondary_Elements_Count: Set to the Dependent-Origin_Secondary_Elements_Count value derived from the Dependent-Origin_Unicast_Addr_Range field value in the PATH_REQUEST message (see Section 3.4.2.2.4) if the On_Behalf_Of_Dependent-Origin field value in the message is 1; otherwise, set to 0.
- Path-Origin_Forwarding_Number, Destination, Path-Origin_Path_Metric_Type, Path-Origin_Path_Metric, Path-Origin_Path_Lifetime, and Path_Discovery_Interval: Set to the values of the corresponding fields in the PATH_REQUEST message.
- Next_Toward_Path-Origin: Set to the SRC field value of the Network PDU of the PATH_REQUEST message.
- Bearer_Toward_Path-Origin: The bit representing the bearer from which the Network PDU of the PATH_REQUEST message was received shall be set to 1 (see Section 4.3.1.4). All other bits shall be set to 0.

If a Discovery Table entry corresponds to the PATH_REQUEST message, and the Path-Origin_Path_Metric value in the PATH_REQUEST message is less than the Path-Origin_Path_Metric value of the table entry, then the following values of the table entry shall be updated:

- Path-Origin_Path_Metric: Set to the Path-Origin_Path_Metric field value of the message.
- Next_Toward_Path-Origin: Set to the SRC field value of the Network PDU of the PATH_REQUEST message.
- Bearer_Toward_Path-Origin: The bit representing the bearer from which the Network PDU of the PATH_REQUEST message was received shall be set to 1 (see Section 4.3.1.4). All other bits shall be set to 0.

3.6.8.6.3 *PATH_REPLY message received*

When a PATH_REPLY message (see Section 3.6.5.12) is received, the node checks whether an existing Discovery Table entry corresponds to the PATH_REPLY message.



A Discovery Table entry corresponds to a PATH_REPLY message when all the following conditions are met:

- The Path-Origin field value in the message is equal to the Path-Origin value of the table entry.
- The Path-Origin-Forwarding-Number field value in the message is equal to the Path-Origin-Forwarding-Number value of the table entry.
- The Destination value of the table entry is a group address, a virtual address, or one of the following addresses:
 - A unicast address in the range [PathTarget, PathTarget + PathTargetSecondaryElementsCount] (i.e., an element address of the Path Target node) if the On_Behalf_Of_Dependent_Target field value in the message is 0. The PathTarget and PathTargetSecondaryElementsCount values are derived from the Path_Target_Unicast_Addr_Range field value in the message (see Section 3.4.2.2.4).
 - A unicast address in the range [DependentTarget, DependentTarget + DependentTargetSecondaryElementsCount] (i.e., an element address of the dependent node of the Path Target node) if the On_Behalf_Of_Dependent_Target field value in the message is 1. The DependentTarget and DependentTargetSecondaryElementsCount values are derived from the Dependent_Target_Unicast_Addr_Range field value in the message (see Section 3.4.2.2.4).

If a Discovery Table entry corresponds to a PATH_REPLY message, field values of the table entry shall be copied into the Forwarding Table state according to Section 3.6.8.5.4.

3.6.8.7 Directed forwarding constants

The following constants are defined for use in directed forwarding procedures:

- Path_Reply_Delay: Minimum delay that a Path Target waits between the reception of a PATH_REQUEST message and the transmission of a PATH_REPLY message. The Path_Reply_Delay is 500 milliseconds.
- Path_Request_Delay: Minimum delay that a Directed Relay node waits between the reception of a PATH_REQUEST message and the transmission of a new PATH_REQUEST message for the same path discovery. The Path_Request_Delay is 150 milliseconds.

3.7 Access layer

The access layer defines how higher-layer applications can use the upper transport layer. It defines the format of the application data; it defines and controls the application data encryption and decryption performed in the upper transport layer; and it checks whether the incoming application data has been received in the context of the right network and application keys before forwarding it to the higher layer.

3.7.1 Endianness

All multiple-octet numeric values in this layer shall be little-endian as described in Section 3.1.1.2.

3.7.2 Access message

The Access message is logically composed of the fields shown in Table 3.60.

Field Name	Size (octets)	Description	Req.
Opcode	1, 2, or 3	Operation code	M
Parameters	0 to 379	Parameters of the operation	M

Table 3.60: Access message fields

An Access message may be sent in up to 32 segments of 12 octets each. This implies that the maximum number of octets is 384 including the TransMIC field.

With a 32-bit TransMIC field, the maximum size of the Access message is 380 octets, and therefore with a single-octet opcode, the parameters field can be up to 379 octets. With a 2-octet opcode, the parameters field can be up to 378 octets. With a 3-octet opcode, the parameters field can be up to 377 octets.

The lower transport layer may segment messages into smaller PDUs for delivery over the network layer. Table 3.61 shows the maximum useful application packet size depending on the number of packets and the size of the TransMIC field.

Number of Packets	Maximum useful Access message size (octets)	
	32-bit TransMIC	64-bit TransMIC
1	11 (unsegmented)	n/a
1	8 (segmented)	4 (segmented)
2	20	16
3	32	28
n	$(n \times 12) - 4$	$(n \times 12) - 8$
32	380	376

Table 3.61: Maximum useful Access message sizes for various sizes of TransMIC

3.7.2.1 Opcode field

The Opcode field contains an operation code (opcode). The opcode is an array of octets comprising 1, 2, or 3 octets. The first octet of the opcode determines the number of octets of the opcode thus defining the size of the Opcode field.

If the most significant bit of the first octet of the opcode is zero, then the opcode contains a single octet. If the two most significant bits of the first octet are 0b10, then the opcode contains two octets. If the two most significant bits of the first octet are 0b11, then the opcode contains three octets. This is shown in Table 3.62.



Opcode Format	Description
0xxxxxxx (excluding 01111111)	1-octet Opcodes
01111111	Reserved for Future Use
10xxxxxx xxxxxxxx	2-octet Opcodes
11xxxxxx zzzzzzzz zzzzzzzz	3-octet Opcodes

Table 3.62: Opcode formats

The 1-octet opcodes are used for Bluetooth SIG defined models. There are 127 1-octet opcodes that can be defined and allocated by the Bluetooth SIG. Opcode 0x7F is reserved for future possible extension.

The 2-octet opcodes are used for Bluetooth SIG defined models. There are 16384 2-octet opcodes that can be defined and allocated by the Bluetooth SIG.

The 3-octet opcodes are used for manufacturer-specific opcodes. There are 64 3-octet opcodes available per company identifier. These opcodes are identified using "x" in [Table 3.62](#), although a company may further sub-class opcodes if desired. The company identifiers are 16-bit values defined by the Bluetooth SIG and are coded into the second and third octets of the 3-octet opcodes, identified using "z" in [Table 3.62](#), using endianness as defined in [Section 3.7.1](#). The company-specific opcodes are managed by the company associated with the identifier.

For example, when the manufacturer-specific opcode is equal to 0x23 and the company identifier is equal to 0x0136 [\[4\]](#), then the 3-octet opcode is equal to 0xE3 0x36 0x01.

3.7.2.2 Parameters field

The Parameters field is defined individually for each opcode. The specific parameters are defined within the message definitions in [Section 4.3](#) or in other compatible specifications (for example see [\[9\]](#)). The Parameters field can be zero octets in length.

3.7.3 Access layer behavior

3.7.3.1 Transmitting an Access message

There are four contexts in which an Access message is transmitted, as illustrated in [Figure 3.39](#): the message is published by a model (arrow 1a); the message is transmitted by a model (arrow 2a) in response to message that the model received (arrow 2b); the message is transmitted as a message originated by a model (arrow 3a); or the message is transmitted as a message originated by an application (arrow 4a).

Receipt of the message may trigger a message sent back in response (arrows 1b and 4b). Each context is discussed in more detail in this section.

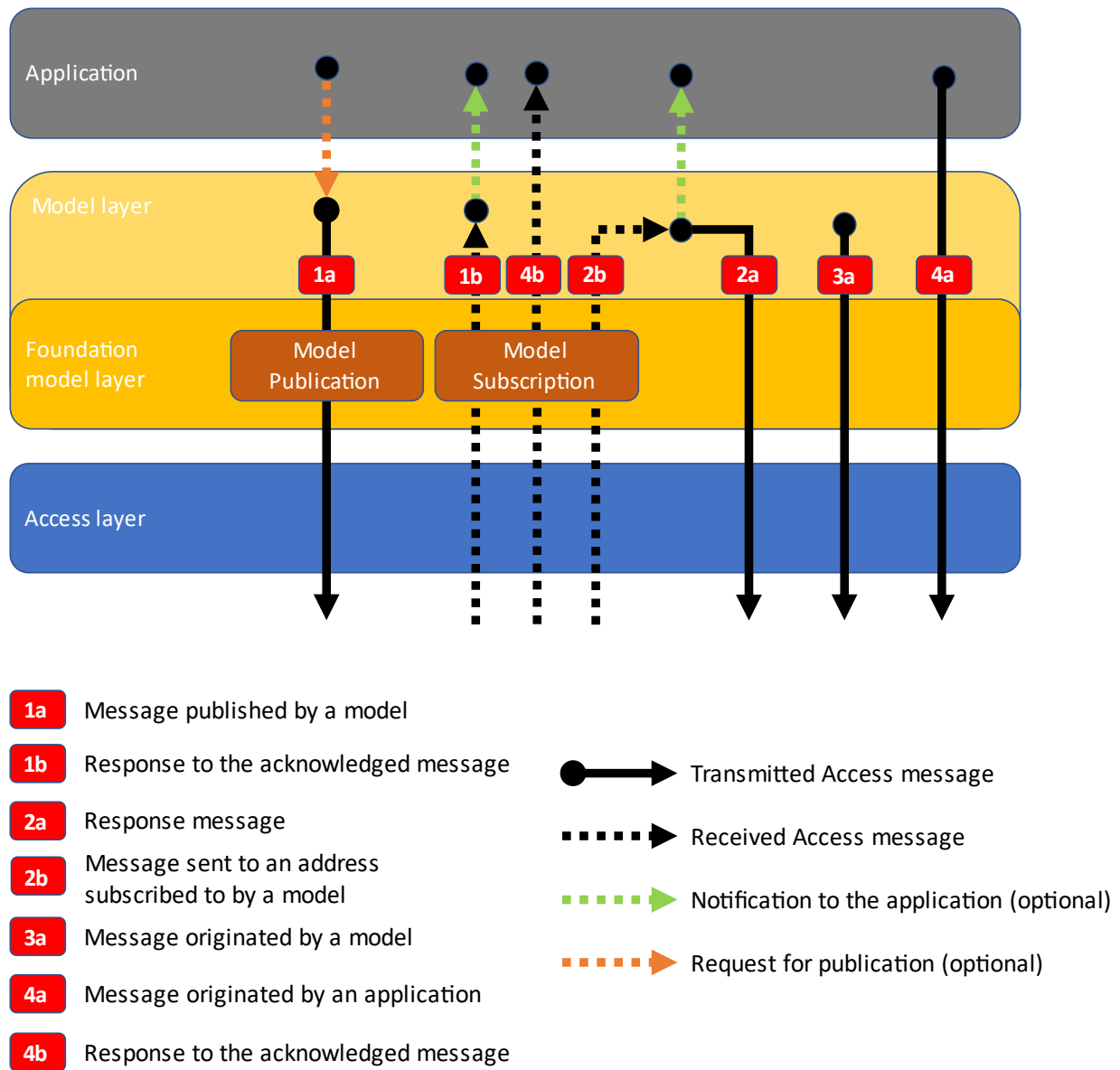


Figure 3.39: Transmitting an Access message

The message is published by a model. In this case, the model is configured to publish a message in response to a local state change, to periodically indicate state value (see Section 3.7.5.1.3), to indicate the progress of a transition to a new state, or to indicate that a state transition has completed (see Section 3.7.5.1.2). If the state transition is caused by a user action, the model should publish the status message as soon as possible.

When the publication of a message is the result of a power-up, a state transition progress update, or completion of a state transition, multiple nodes may be reporting the state change at the same time. To reduce the probability of a message collision, these messages should be sent with a random delay between 20 and 500 milliseconds.

When the publication of a message is the result of periodic publishing, multiple nodes may be publishing at the same time. To reduce the probability of a message collision, these messages should be sent with a random delay between 20 and 50 milliseconds.

The parameters of the message are determined by the Model Publication state (see Section 4.2.3):

- The DST field shall be set to the value of the Publish Address state (see Section 4.2.3.1).
- The TTL field shall be set to the value of the Publish TTL state (see Section 4.2.3.5).
- The use of friendship security material shall be determined by the Publish Friendship Credentials Flag state (see Section 4.2.3.4). The Low Power node using friendship security credentials can become a dependent node of a Directed Friend node (see Section 3.6.8.3).
- The AppKey and NetKey to be used for securing the message shall be determined by the Publish AppKey Index state (see Section 4.2.3.3) and the NetKey that the AppKey is bound to.
- The access layer retransmissions are determined by the Publish Retransmit Count state (see Section 4.2.3.6) and the Publish Retransmit Interval Steps state (see Section 4.2.3.7).

If directed forwarding functionality is supported, and the Directed Publish Policy state value for the publishing model is Directed Forwarding (see Section 4.2.37), then the message shall be tagged with the use-directed tag. If directed forwarding functionality is supported, and the Directed Publish Policy state value for the publishing model is Managed Flooding, then the message shall be tagged with the immutable-credentials tag.

The message is transmitted by a model in response to a message that it has received.

The response message shall be configured as follows:

- The DST field shall be set to the value of the SRC field of the received message.
- If the TTL field of the received message is greater than 0, the TTL field shall be set to the value of the Default TTL state (see Section 4.2.8).
- If the TTL field of the received message is equal to 0, the TTL field shall be set to 0 or to the value of the Default TTL state (see Section 4.2.8). The TTL field should be set to 0.
- The AppKey or DevKey and the NetKey values to be used for securing the response message shall be the AppKey or DevKey and the NetKey used to secure the received message.
- The response message shall use managed flooding security credentials. However, the security credentials may be changed by a lower layer unless the received message uses the managed flooding security credentials. If the received message uses managed flooding security credentials, then the response message shall be tagged with the immutable-credentials tag, and the security credentials will not be changed by any lower layer (see Section 3.4.6.4). If the received message uses friendship security credentials, a Low Power node may use friendship security credentials for the response message; however, only a Friend node can process these response messages (see Section 3.4.6.3 and Section 3.6.8.3).

For a node that is not a Low Power node with an established friendship, the response message should be transmitted within 5 seconds after receiving the acknowledged message.

To reduce the probability of multiple nodes responding to the received message at exactly the same time, and therefore increase the probability of message delivery rather than message collisions, the node should transmit the response message with a random delay:

- If the received message was sent to a unicast address, the node should transmit the response message with a random delay between 20 and 50 milliseconds.
- If the received message was sent to a group address or a virtual address, the node should transmit the response message with a random delay between 20 and 500 milliseconds.

The message is transmitted as an unsolicited message originated by a model, which may trigger a response message.

The parameters of the message are determined by the model:

- The DST field shall be set by the model.
- The TTL field should be set by the model; if not set, the Default TTL state (see Section 4.2.8) shall be used.
- The use of managed flooding security material or friendship security material shall be determined by the model. The Low Power node in friendship should use friendship security material.
- The model shall determine the keys to be used to secure the message using either of the following:
 - An AppKey and the NetKey that the AppKey is bound to
 - A DevKey and a NetKey

The model may tag a message with additional metadata to allow or prevent lower layers from changing the security credentials of the message:

- If the model does not add any tag to the message, the security credentials of the Access message may be changed in lower layers.
- If directed forwarding functionality is supported, the model may indicate the preference to use directed forwarding by adding the use-directed tag to the message.
- If the model adds the immutable-credentials tag to the message, the security credentials of the Access message shall not be changed in lower layers.

The message is transmitted as an unsolicited message originated by an application, which may trigger a response message.

The parameters of the message are determined by the application:

- The DST field shall be set by the application.
- The TTL field should be set by the application; if not set, the Default TTL state (see Section 4.2.8) shall be used.
- The use of managed flooding security material or friendship security material shall be determined by the application. The Low Power node in friendship should use friendship security material.
- The application shall determine the keys to be used to secure the message using either of the following:
 - An AppKey and the NetKey that the AppKey is bound to
 - A DevKey and a NetKey

The application may tag a message with additional metadata to allow or prevent lower layers from changing the security credentials of the message:

- If the application does not add any tag to the message, the security credentials of the Access message may be changed in lower layers.
- If directed forwarding functionality is supported, the application may indicate the preference to use directed forwarding by adding the use-directed tag to the message.
- If the application adds the immutable-credentials tag to the message, the security credentials of the Access message shall not be changed in lower layers.

All Access messages.

The SRC field shall be set to the unicast address of the element within the node that is originating the message and shall be delivered to the upper transport layer for processing (see Section 3.6.4.1).

The message may be tagged with the send-segmented tag.

If directed forwarding functionality is supported and enabled in the subnet over which the message is transmitted, and the friendship security credentials have not been selected, and the destination is different from the all-nodes and all-relays fixed group addresses, and the TTL field has the value 2 or greater, and the message is tagged with the use-directed tag, then the element checks whether a Path Origin State Machine associated with the destination exists (see Section 4.4.7.2). If there is no existing Path Origin State Machine and no existing Forwarding Table entry for a path from the source address to the destination address (see Section 3.6.8.5.1), then the element shall instantiate a Path Origin State Machine in the Initial state for that destination.

Due to limited bandwidth available that is shared among all nodes and other Bluetooth devices, it is important to observe the volume of traffic a node is originating. A node should originate less than 100 Lower Transport PDUs in a moving 10-second window.

3.7.3.2 Receiving an Access message

A message shall be delivered to each instance of a model for processing if all the following conditions are met:

- The Opcode indicates the message is supported by the model instance available on an element on the device.
- Either the access layer security of the model instance is using application keys, and the model instance is bound to the AppKey used to secure the message, or the access layer security of the model is using the DevKey, and the DevKey was used to secure the message.
- The destination address is set to any one of the following:
 - A fixed group address that is defined in [Table 3.63](#) and meets the corresponding condition. When the fixed group address meets the corresponding condition, the message shall be delivered only to the model instance on the primary element.
 - A group address (including any of the fixed group addresses mentioned in [Table 3.7](#), except the all-nodes address) or a virtual address that the instance of the model is subscribed to.
 - The unicast address of the element of the model instance.

Note: A message with a fixed group address can be delivered to model instances on any element of the device, irrespective of any condition defined in [Table 3.63](#), because the model subscription lists can contain one or more fixed group addresses.

Destination Address	Condition
all-directed-forwarding-nodes	Directed forwarding functionality is enabled
all-proxies	Proxy functionality is enabled
all-friends	Friend functionality is enabled
all-relays	Relay functionality is enabled
all-nodes	—

Table 3.63: Fixed group destination addresses and conditions

[Figure 3.40](#) illustrates an example of processing steps for an incoming Access message.

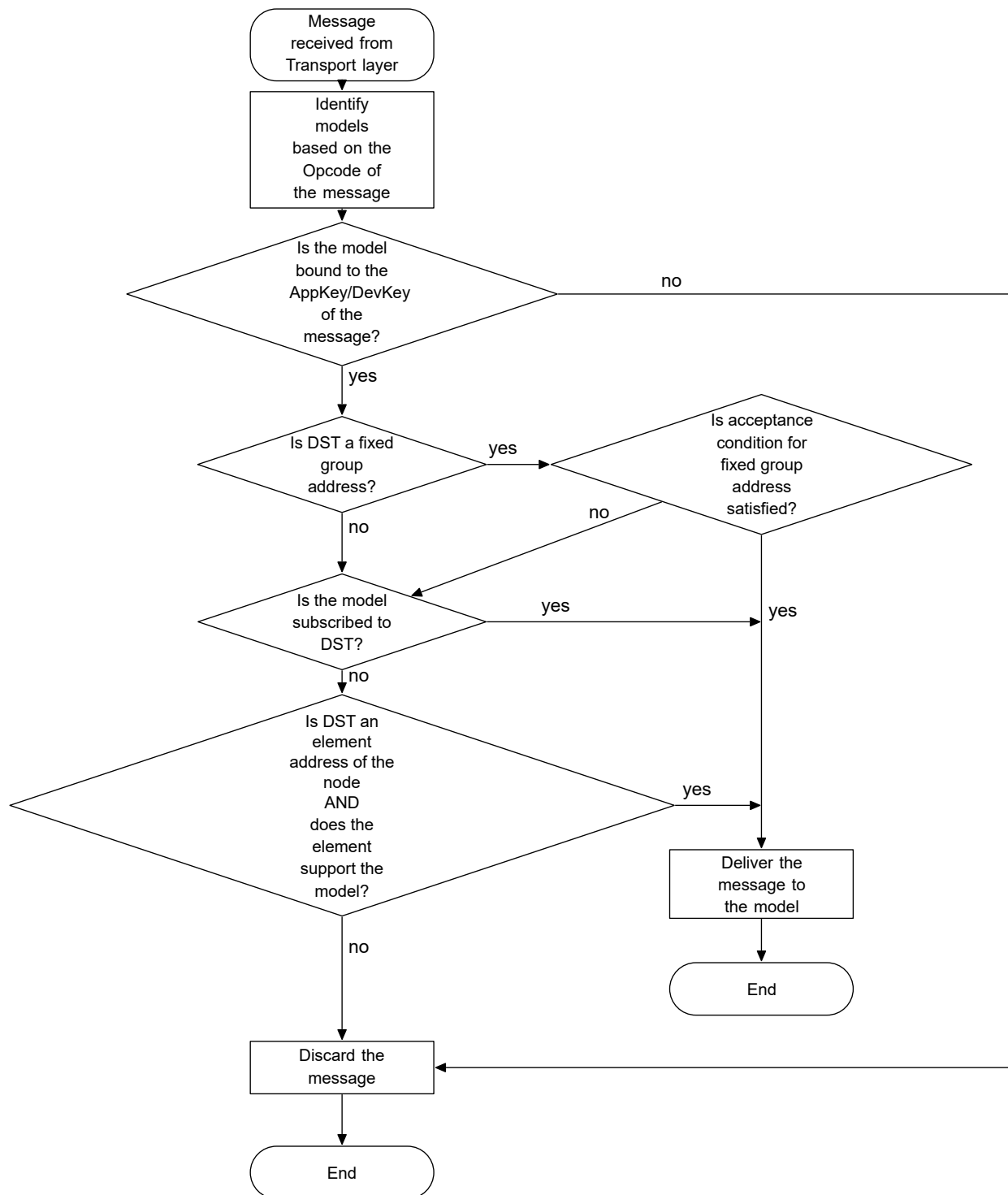


Figure 3.40: Example of Access message processing steps

3.7.3.3 Security considerations

A message is encrypted and authenticated by the upper transport layer. Messages originated by a node shall use either the AppKey configured for the Model or the DevKey.

A response message shall always use the same DevKey or AppKey and NetKey combination used by the corresponding request message.

3.7.3.4 Message error procedure

When receiving a message that is not understood by an element, it shall ignore the message.

Note: A message can be falsely identified as a valid message, passing the NetMIC and TransMIC fields authentication using a known network key and application key even though that message was sent using different keys. The decryption of that message using the wrong keys would result in a message that is not understood by the element. The probability of such a situation occurring is small but not insignificant.

A message that is not understood includes messages that have one or more of the following conditions:

- The opcode field of the Access message is unknown by the receiving element.
- The Access message size for the identified opcode is incorrect.
- The parameters field of the Access message contains values that are Prohibited.

Note: An element that sends an acknowledged message that is not understood by a peer node will not receive any response message.

3.7.4 Unacknowledged and acknowledged messages

At the access layer, a message is defined as unacknowledged or acknowledged.

3.7.4.1 Unacknowledged message

An unacknowledged message is transmitted when a response is not required.

For example, a message is published by a model when a state changes, or an unacknowledged message may be sent to a group address to change the states of the group members without causing multiple messages to be sent back.

There is no response to an unacknowledged message, therefore it is not possible for the sending element to determine if that message has been delivered or processed.

3.7.4.2 Acknowledged message

An acknowledged message is transmitted and acknowledged by each receiving element by responding to that message. The response is typically a status message.

If a response is not received within an arbitrary time period, the message originator may retransmit the message. The time period used is application specific.

If an element transmits a message to more than one element, for example it has set the destination address to a group address, the element may not know how many elements may respond to the message. It is not recommended to send an acknowledged message to the all-nodes address. To increase the probability of successful delivery of messages, the sending element should determine how many message retransmissions are required before it considers that all the nodes that should have received the message have actually received it.

If the element does not receive a response within a period of time known as the acknowledged message timeout, then the element may consider the message has not been delivered, without sending any additional messages.

The acknowledged message timeout should be set to a minimum of 30 seconds. The exact value is application specific.

When an acknowledged message is delivered to the model, it shall send the associated response message to acknowledge that message. The response message may include information such as state information. The response message is an unacknowledged message.

3.7.5 Publish and subscribe

A higher layer specification can describe messages containing data as being published by a model or subscribed to by a model's element.

Publishing and subscribing is performed using destination addresses.

The configuration of the destination addresses used for publishing and subscribing is managed by the Foundation models (see Section 4).

3.7.5.1 Publish

A model publishes data if it transmits an unsolicited message to a destination address. Messages can be transmitted to destination addresses that can be unicast, group, or virtual, known as the publish addresses. Each model within a node has a single publish address.

3.7.5.1.1 *State transitions*

States within an element either can change instantaneously or can transition over time to a new state, as illustrated in [Figure 3.41](#). The time from the initial state to the target state is the transition time. The time from the current state to the target state is the remaining time. When a message is received to set a new state value, this new value may not be immediately applied, but may instead be stored as a target state. The state will transition from the initial state to the target state. A status message can be sent at any time and will always include the current state even if the transition time has not elapsed. This status message may include the remaining time between the current state and the target state. When the current state reaches the target state, the state transition ends.

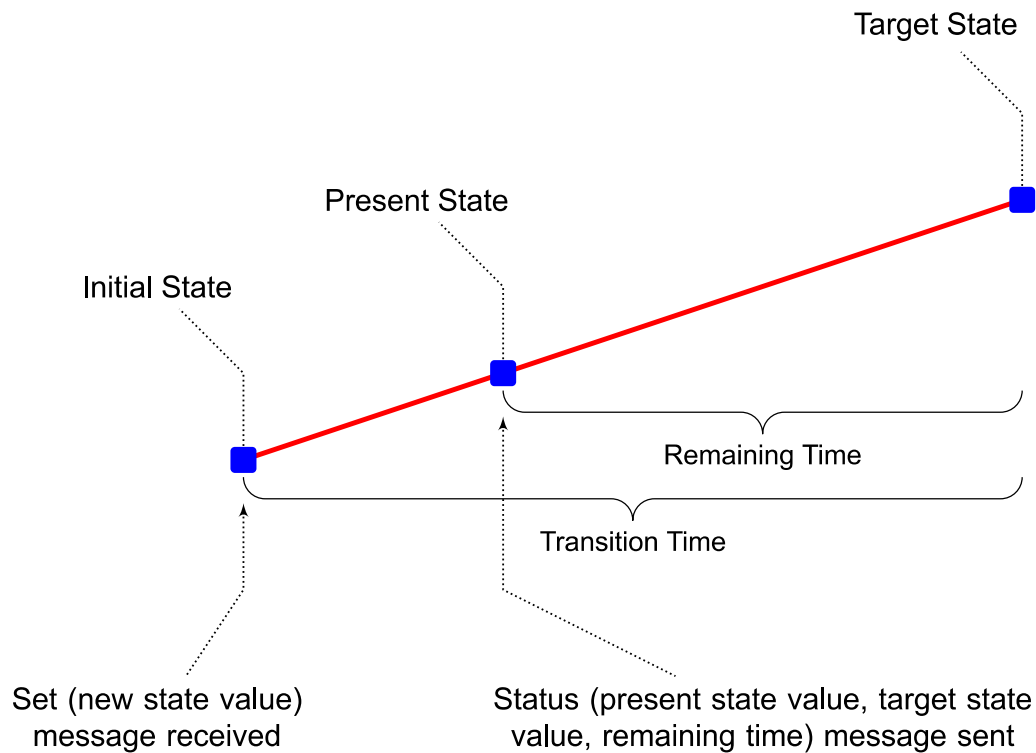


Figure 3.41: State transition

3.7.5.1.2 State change publishing

Publishing a message on a state change is enabled by setting a Model Publication state (see Section 4.2.3) for a model. When publishing is enabled for a model, unless specified otherwise by a higher layer specification, a corresponding status message shall be published immediately after the state transition ends. For transitions that take more than 2 seconds, it is recommended to publish an additional status message within 1 second from the start of the state transition.

3.7.5.1.3 Periodic publishing

A model may be configured to send status messages periodically regardless of whether the state has changed or not. This is done by using a Publish Period (see Section 4.2.3.2). When the Publish Period is set to a non-zero value, unless specified otherwise by a higher layer specification, a status message shall be published at least once every Publish Period. When the Publish Period is set to 0, the status messages shall only be published on a state change when enabled.

3.7.5.1.4 Publish retransmissions

When a new message is being published by a model instance, all pending retransmissions of the previous message published by the model instance shall be canceled, and the model instance shall retransmit the new message as specified by the Publish Retransmit Count and Publish Retransmit Interval Steps states.

3.7.5.2 Subscribe

Each model may have one or more subscription lists (see Section 4.2.4) of one or more addresses that determines which destination addresses this model's element subscribes to. These subscription addresses can be a group address or a virtual address.

Note: A model is, in effect, always subscribed to its element unicast address as described in Section 3.7.3.2.

3.7.6 Example message sequence charts

This section shows typical message sequence charts.

3.7.6.1 Acknowledged get

Figure 3.42 shows a client getting a state of a peer element using an acknowledged get message. The server responds with the associated status message.

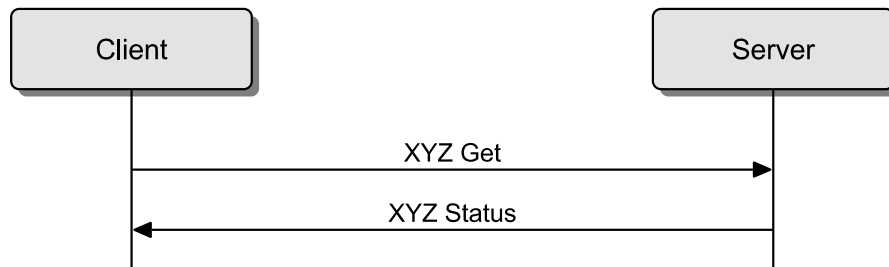


Figure 3.42: Acknowledged get message

3.7.6.2 Acknowledged set

Figure 3.43 shows a client setting a state of a peer element with an acknowledged set message. The server responds with the associated status message. The server then publishes a status message to the model's publish address according to the publishing rules (see Section 3.7.5.1.2). If the client is subscribed to this model's publish address, then it will receive both status messages.

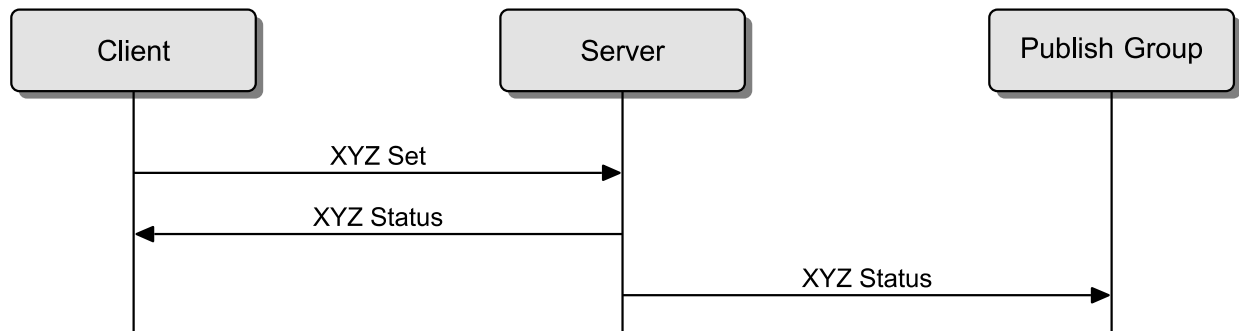


Figure 3.43: Acknowledged set message

3.7.6.3 Unacknowledged set

Figure 3.44 shows a client setting a state of a peer element with an unacknowledged set message. No response is sent, but the server publishes the new state information to the model's publish address

according to the publishing rules (see Section 3.7.5.1.2). If the client is subscribed to this model's publish address, then it will receive the status message.

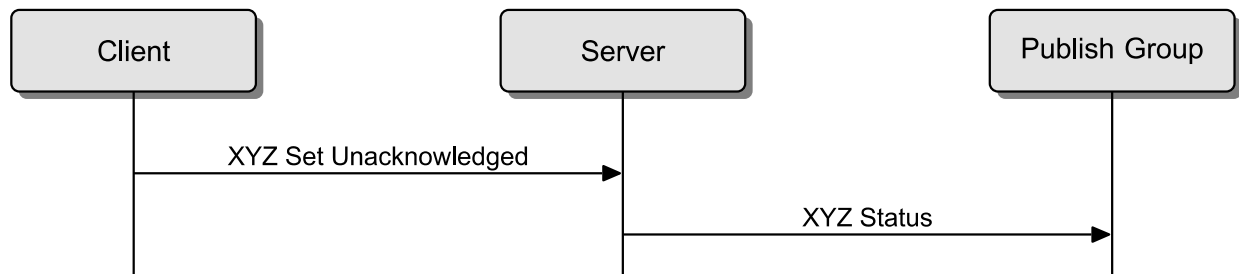


Figure 3.44: Unacknowledged set message

3.7.6.4 Acknowledged set with periodic publishing

Figure 3.45 shows a client setting a state of a peer element with an acknowledged set message. The server responds with the associated status message. The server then periodically publishes the new state information to the model's publish address, according to the periodic publishing rules (see Section 3.7.5.1.2).

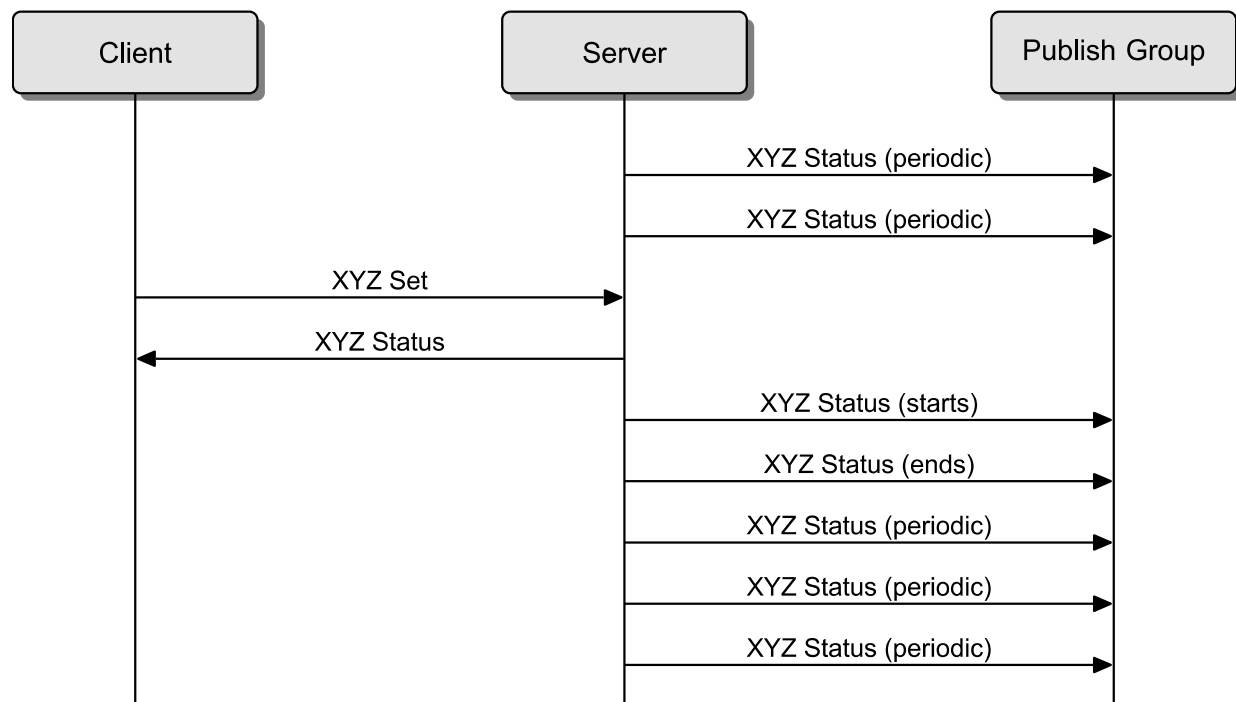


Figure 3.45: Acknowledged set message with periodic publishing

3.8 Model layer

Models are used to define certain functionalities supported by a node.

The model layer specifies the rules for allocating models on elements within a node.

The model layer also includes models which are defined in the Mesh Model specification [9] or in other higher-layer specifications.

3.8.1 Endianness

All multiple-octet numeric values in this layer shall be little-endian as described in Section 3.1.1.2.

3.8.2 Model identifier

A model is identified by a unique identifier. The identifier can be 16 bits in length for a Bluetooth SIG adopted model (SIG Model ID) or 32 bits in length for a Vendor Model (Vendor Model ID). See Section 4.4.23 for additional information on SIG Model IDs.

The Vendor Model ID is composed of two fields: a 16-bit Company Identifier [4] assigned by the Bluetooth SIG and a 16-bit Vendor Model Identifier assigned by the vendor. The format of each Vendor Model ID is defined in Table 3.64.

Field	Size (octets)	Description	Req.
Company Identifier	2	16-bit Company Identifier, see [4]	M
Vendor Model Identifier	2	16-bit Vendor Model Identifier assigned by the vendor	M

Table 3.64: Vendor Model ID format

3.8.3 Functionalities

Models are grouped together in order to support functionalities (such as dimmable light or occupancy sensing).

Message dispatch from the access layer (see Section 3.7.3.2) to the model layer is based on the destination address (see Section 3.4.4.7) and the opcode of the message (see Section 3.7.2.1). Given that unicast addresses are assigned to each element on a node, an element cannot have instances of models with overlapping opcodes. Therefore, a functionality may require a set of models that must be located on different elements within a node, as required by Section 3.7.3.

To exchange messages with a Bluetooth SIG adopted model, a Vendor Model shall use the Access message defined for the Bluetooth SIG model (see Section 3.7.2).

A node supports a functionality by instantiating the main model for that functionality, which may require a set of base, extending, and corresponding models:

- A main model may require one or more base models. A node can include multiple instances of a main model to implement the same functionality multiple times. A main model may be instantiated on either the primary element or a secondary element of a node.
- Base models must be instantiated with the extending models and corresponding models for a functionality and must be indicated in the Composition Data of a node (see Section 4.2.2), as required in Section 3.8.4.

A base model might be extended by multiple models if unambiguous message dispatch is preserved by this extension.

For a more detailed description and examples see Section 1.4.4 in [9].

3.8.4 Model instantiation

The following rules apply to all models instantiated within a node:

- All models shall be instantiated on a node by using the smallest number of elements to achieve the desired functionality (i.e., by using the smallest possible element index).
- Corresponding models that cannot be instantiated on the same element as the main model, as a result of message dispatch limitations, shall be instantiated on an element with a larger element index.
- Multiple instances of the same main model shall not interleave their corresponding models on subsequent elements. All the corresponding models for the main model shall be instantiated before the next instance of the main model.

3.9 Mesh security

This section describes how mesh security is implemented.

3.9.1 Endianness

All multiple-octet numeric values in this layer shall be marshalled in big-endian, as described in Section 3.1.1.1.

3.9.2 Security toolbox

This section describes functions that together provide a security toolbox for mesh networking.

3.9.2.1 Encryption function

The same encryption function e , as defined in Volume 3, Part H, Section 2.2.1 of the Core Specification [1], shall be used. This can be summarized as:

$$ciphertext = e(key, plaintext)$$

3.9.2.2 CMAC function

RFC4493 [7] defines the Cipher-based Message Authentication Code (CMAC) function that uses AES-128 as the block cipher function, also known as AES-CMAC.

The inputs to AES-CMAC are:

k is the 128-bit key

m is the variable-length data to be authenticated

The 128-bit message authentication code (MAC) is generated¹ as follows:

$$MAC = AES-CMAC_k(m)$$

A node can implement AES functions in the host or can use the HCI_LE_Encrypt command (see Volume 2, Part E, Section 7.8.22 of the Core Specification [1]) in order to use the AES function in the controller.

3.9.2.3 AES-CCM function

The AES-CCM function provides encryption and authentication using Counter with Cipher Block Chaining-Message Authentication Code (CCM), which shall be implemented consistent with the algorithm as defined in IETF RFC 3610 [8] in conjunction with the AES-128 block cipher as defined in NIST Publication FIPS-197 [28].

Note: A description of the CCM algorithm can also be found in the NIST Special Publication 800-38C [29].

This specification defines AES-CCM as a function that takes four inputs and results in two outputs.

The inputs to AES-CCM are:

k is the 128-bit key

n is a 104-bit nonce

m is the variable-length data to be encrypted and authenticated – also known as “plaintext”

a is the variable-length data to be authenticated – also known as “Additional Data”

The ciphertext and mic are generated as follows:

$$ciphertext, mic = AES-CCM_k(n, m, a)$$

Where:

ciphertext is the variable-length data after it has been encrypted

mic is the message integrity check value of m and a – also known as the “Message Authentication Code” or the encrypted authentication value U in RFC3610 [8].

If only the k, n, and m parameters are provided to the AES-CCM, then the additional data shall be zero length.

¹ This is using the same notation used in other Bluetooth specifications. This is functionally the same as the notation in RFC 4493, where $MAC = AES-CMAC(k, m)$.

3.9.2.4 HMAC-SHA-256 function

RFC 2104 [26] defines HMAC, a mechanism for message authentication using cryptographic hash functions. FIPS 180-4 [27] defines the SHA-256 secure hash algorithm. The SHA-256 algorithm is used as a hash function for the HMAC mechanism for the HMAC-SHA-256 function. This specification defines HMAC-SHA-256 as a function that takes two inputs and results in one output.

The inputs to HMAC-SHA-256 are:

k is the 256-bit key

m is variable-length data to be authenticated

The HMAC-SHA-256 function generates a 256-bit message authentication code (MAC) defined in [26] as follows:

$$HMAC-SHA-256_k(m) = SHA-256((k_0 \oplus opad) \parallel SHA-256((k_0 \oplus ipad) \parallel m))$$

Where:

SHA-256 is the secure hash algorithm defined in [27]

k_0 is the k with the 0x00 octet appended 32 times to the end

opad is the 0x5C octet repeated 64 times

ipad is the 0x36 octet repeated 64 times

3.9.2.5 s1 SALT generation function

The input to function s1 is:

M is a non-zero length octet array or ASCII encoded string

If M is an ASCII encoded string, it shall be converted into an octet array by replacing each string character with its ASCII code preserving the order. For example, if M is the string "MESH", this is converted into the octet array: 0x4d, 0x45, 0x53, 0x48.

ZERO is the 128-bit value:

0x0000 0000 0000 0000 0000 0000 0000 0000

The output of the salt generation function s1 is as follows:

$$s1(M) = AES-CMAC_{ZERO}(M)$$

3.9.2.6 s2 SALT generation function

The input to function s2 is:

M is a non-zero length octet array or ASCII encoded string



ZERO is the 256-bit value:

0x0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000

The output of the salt generation function $s2$ is as follows:

$$s2(M)=HMAC-SHA-256_{ZERO}(M)$$

3.9.2.7 k1 derivation function

The network key material derivation function $k1$ is used to generate instances of IdentityKey, BeaconKey, and PrivateBeaconKey.

The definition of this key generation function makes use of the MAC function $AES-CMAC_T$ with a 128-bit key T .

The inputs to function $k1$ are:

N is 0 or more octets

$SALT$ is 128 bits

P is 0 or more octets

The key (T) is computed as follows:

$$T=AES-CMAC_{SALT}(N)$$

The output of the key generation function $k1$ is as follows:

$$k1(N, SALT, P)=AES-CMAC_T(P)$$

3.9.2.8 k2 network key material derivation function

The network key material derivation function $k2$ is used to generate instances of EncryptionKey, PrivacyKey, and NID for use as managed flooding security material, friendship security material, and directed security material.

The definition of this key generation function makes use of the MAC function $AES-CMAC_T$ with a 128-bit key T .

The inputs to function $k2$ are:

N is 128 bits

P is 1 or more octets

The key (T) is computed as follows:

$$T=AES-CMAC_{SALT}(N)$$

$SALT$ is the 128-bit value computed as follows

$$SALT=s1("smk2")$$



The output of the key generation function k2 is as follows:

$$\begin{aligned}
 T0 &= \text{empty string (zero length)} \\
 T1 &= \text{AES-CMAC}_T(T0 \parallel P \parallel 0x01) \\
 T2 &= \text{AES-CMAC}_T(T1 \parallel P \parallel 0x02) \\
 T3 &= \text{AES-CMAC}_T(T2 \parallel P \parallel 0x03) \\
 k2(N, P) &= (T1 \parallel T2 \parallel T3) \bmod 2^{263}
 \end{aligned}$$

3.9.2.9 k3 derivation function

The derivation function k3 is used to generate a public value of 64 bits derived from a private key.

The definition of this derivation function makes use of the MAC function AES-CMAC_T with a 128-bit key T.

The input to function k3 is:

N is 128 bits

The key (T) is computed as follows:

$$T = \text{AES-CMAC}_{SALT}(N)$$

SALT is a 128-bit value computed as follows:

$$SALT = s1("smk3")$$

The output of the derivation function k3 is as follows:

$$k3(N) = \text{AES-CMAC}_T("id64" \parallel 0x01) \bmod 2^{64}$$

3.9.2.10 k4 derivation function

The derivation function k4 is used to generate a public value of 6 bits derived from a private key.

The definition of this derivation function makes use of the MAC function AES-CMAC_T with a 128-bit key T.

The input to function k4 is:

N is 128 bits

The key (T) is computed as follows:

$$T = \text{AES-CMAC}_{SALT}(N)$$

SALT is a 128-bit value computed as follows:

$$SALT = s1("smk4")$$

The output of the derivation function k4 is as follows:

$$k4(N)=AES-CMAC_T("id6" || 0x01) \bmod 2^6$$

3.9.2.11 k5 provisioning material derivation function

The provisioning material derivation function k5 is used to generate the 256-bit key used in provisioning.

The definition of this derivation function makes use of the MAC function HMAC-SHA-256 with a 256-bit key T.

The inputs to function k5 are:

N is 32 or more octets

SALT is 256 bits

P is 1 or more octets

The key (T) is computed as follows:

$$T=HMAC-SHA-256_{SALT}(N)$$

The output of the derivation function k5 is as follows:

$$k5(N, SALT, P)=HMAC-SHA-256_T(P)$$

3.9.3 Sequence number

The sequence number, a 24-bit value contained in the SEQ field of the Network PDU, is primarily designed to protect against replay attacks. Elements within the same node may or may not share the sequence number space with each other. Having a different sequence number in each new Network PDU for every message source (identified by the unicast address contained in the SRC field) is critical for the security of the mesh network.

With a 24-bit sequence number, an element can transmit 16,777,216 messages before repeating a nonce. If an element transmits a message on average once every five seconds (representing a fairly high frequency message for known use cases), the element can transmit for 2.6 years before the nonce repeats.

Each element shall use strictly increasing sequence numbers for the Network PDUs it generates. Before the sequence number approaches the maximum value (0xFFFFF), the element shall update the IV Index using the IV Update procedure (see Section 3.11.5). This is done to ensure that the sequence number will never wrap around.

3.9.4 IV Index

The IV Index is a 32-bit value that is a shared network resource (i.e., all nodes in a mesh network share the same value of the IV Index and use it for all subnets they belong to).

The IV Index starts at 0x00000000 and is incremented during the IV Update procedure as described in Section 3.11.5. The timing when the value is incremented does not have to be exact, since the least

significant bit is communicated within every Network PDU. Since the IV Index value is a 32-bit value, a mesh network can function approximately 5 trillion years before the IV Index will wrap.

The IV Index is shared within a network via Secure Network beacons (see Section 3.10.3) or Mesh Private beacons (see Section 3.10.4). IV updates received on a subnet are processed and propagated to that subnet. The propagation happens by the device transmitting Secure Network beacons or Mesh Private beacons with the updated IV Index for that particular subnet. If a device on a primary subnet receives an update on the primary subnet, it shall propagate the IV update to all other subnets. If a device on a primary subnet receives an IV update on any other subnet, the update shall be ignored.

If a node is absent from a mesh network for a period of time, it can scan for Secure Network beacons (see Section 3.11.1) or for Mesh Private beacons, or it can use the IV Index Recovery procedure (see Section 3.11.6), and therefore set the IV Index value autonomously.

3.9.5 Nonce

The nonce is a 13-octet value that is unique for each new message encryption. There are four different nonces that are used, as shown in Table 3.65. The type of the nonce is determined by the first octet of the nonce, referred to as the Nonce Type.

Nonce Type	Nonce	Description
0x00	Network nonce	Used with an EncryptionKey for network authentication and encryption
0x01	Application nonce	Used with an application key for upper transport authentication and encryption
0x02	Device nonce	Used with a device key for upper transport authentication and encryption
0x03	Proxy nonce	Used with an EncryptionKey for proxy authentication and encryption
0x04	Proxy solicitation nonce	Used with an EncryptionKey for Proxy solicitation authentication and encryption
0x05–0xFF	RFU	Reserved for Future Use

Table 3.65: Nonce types

The TTL field value is used within the network nonce but not within the application nonce, device nonce, or proxy nonce. This means that when a message is relayed and the TTL field value is decremented, the application nonce or device nonce does not change; however, the network nonce does change, allowing the authentication of the TTL field value.

The DST field value is used within the application nonce and device nonce but not in the network nonce. This means that the destination of the message is authenticated, but at the network layer the destination address is encrypted.

3.9.5.1 Network nonce

The network nonce is defined in Table 3.66 and illustrated in Figure 3.46.



Field	Size (octets)	Description
Nonce Type	1	0x00
CTL and TTL	1	See Table 3.67
SEQ	3	Sequence number
SRC	2	Source address
Pad	2	0x0000
IV Index	4	IV Index

Table 3.66: Network nonce format

Field	Size (bits)	Description
CTL	1	See Section 3.4.4.3
TTL	7	See Section 3.4.4.4

Table 3.67: CTL and TTL field format

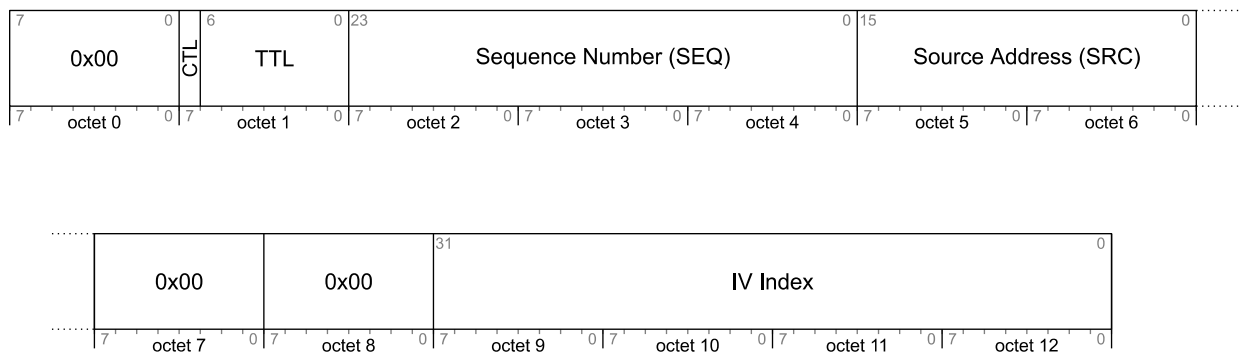


Figure 3.46: Network nonce format

The network nonce is used with an EncryptionKey for network data authentication and encryption (see [Section 3.9.7.2](#)).

3.9.5.2 Application nonce

The application nonce is defined in [Table 3.68](#) and illustrated in [Figure 3.47](#).

Field	Size (octets)	Description
Nonce Type	1	0x01
ASZMIC and Pad	1	See Table 3.69
SEQ	3	Sequence number of the Access message (24 lowest bits of SeqAuth in the context of segmented messages)
SRC	2	Source address

Field	Size (octets)	Description
DST	2	Destination address
IV Index	4	IV Index

Table 3.68: Application nonce format

Field	Size (bits)	Description
ASZMIC	1	SZMIC field value if a Segmented Access message or 0 for all other message formats
Pad	7	0b0000000

Table 3.69: ASZMIC and Pad field format

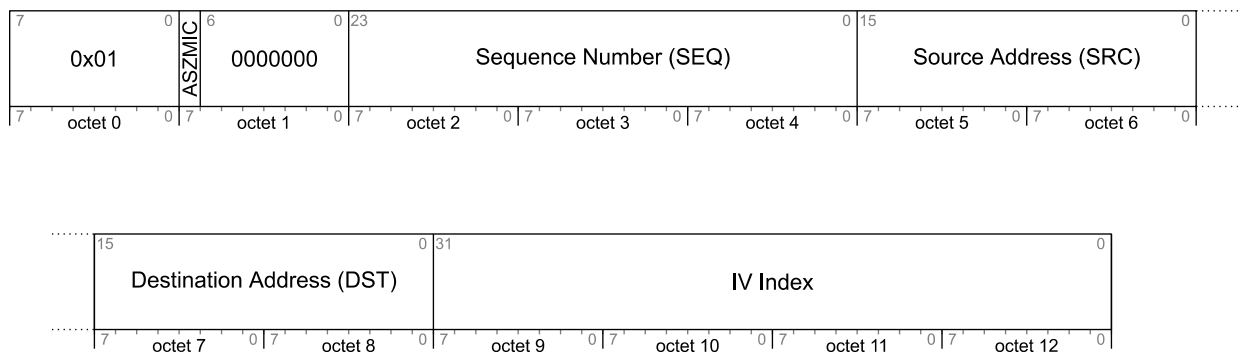


Figure 3.47: Application nonce format

The application nonce is used with an application key for application data authentication and encryption (see Section 3.9.6).

3.9.5.3 Device nonce

The device nonce is defined in Table 3.70 and illustrated in Figure 3.48.

Field	Size (octets)	Description
Nonce Type	1	0x02
ASZMIC and Pad	1	See Table 3.71
SEQ	3	Sequence number of the Access message (24 lowest bits of SeqAuth in the context of segmented messages)

Field	Size (octets)	Description
SRC	2	Source address
DST	2	Destination address
IV Index	4	IV Index

Table 3.70: Device nonce format

Field	Size (bits)	Description
ASZMIC	1	SZMIC field value if a Segmented Access message or 0 for all other message formats
Pad	7	0b0000000

Table 3.71: ASZMIC and Pad field format

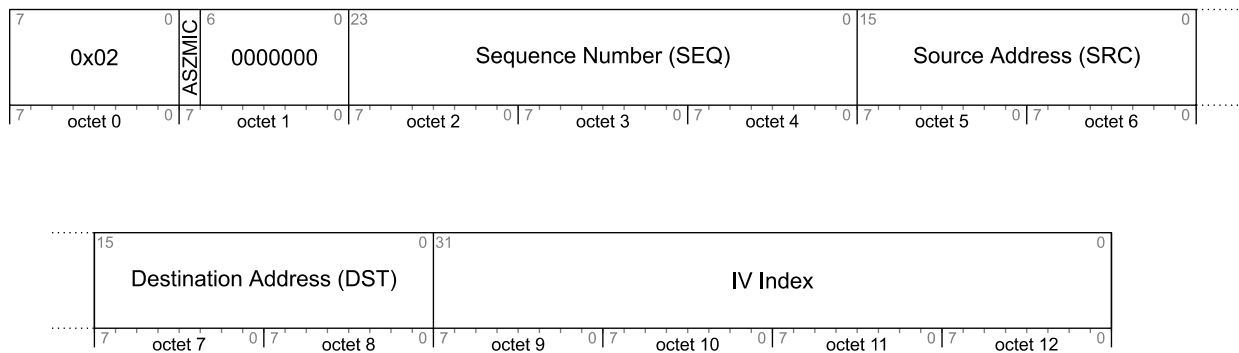


Figure 3.48: Device nonce format

The device nonce is used with a device key for application data authentication and encryption specific to a given device (see Section 3.9.6).

3.9.5.4 Proxy nonce

The proxy nonce is defined in Table 3.72 and illustrated in Figure 3.49.

Field	Size (octets)	Description
Nonce Type	1	0x03
Pad	1	0x00
SEQ	3	Sequence number
SRC	2	Source address
Pad	2	0x0000
IV Index	4	IV Index

Table 3.72: Proxy nonce format

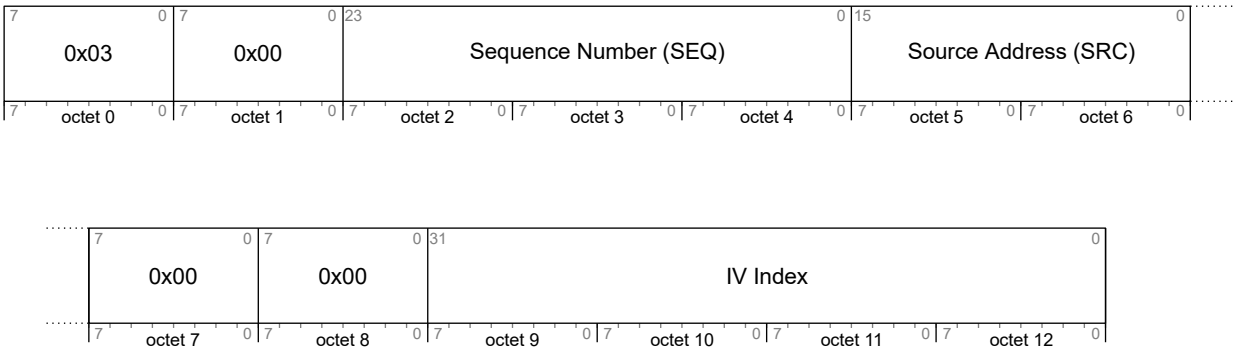


Figure 3.49: Proxy nonce format

The proxy nonce is used with an EncryptionKey for proxy configuration message authentication and encryption (see Section 6.5).

3.9.5.5 Proxy solicitation nonce

The proxy solicitation nonce is defined in Table 3.73 and illustrated in Figure 3.50.

Field	Size (octets)	Description
Nonce Type	1	0x04
Pad	1	0x00
SSEQ	3	Solicitation sequence number
SSRC	2	Solicitation source
Pad	6	0x000000000000

Table 3.73: Proxy solicitation nonce format

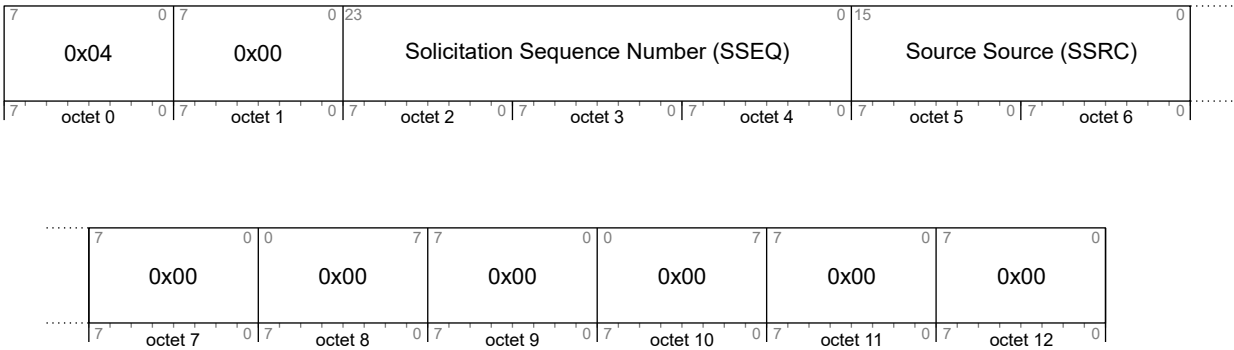


Figure 3.50: Proxy solicitation nonce format

The proxy solicitation nonce is used with an EncryptionKey for Solicitation PDU authentication and encryption (see Section 6.8).

3.9.6 Keys

This specification defines three types of keys: device keys (DevKey), application keys (AppKey) and network keys (NetKey). AppKeys are used to secure communications at the upper transport layer and NetKeys are used to secure communications at the network layer. The keys are shared between nodes. There is also a device key (DevKey), which is a special key that is unique to each node, is known only to the node and a Configuration Manager, and is used to secure communications between the node and a Configuration Manager.

Application keys are bound to network keys. This means application keys are only used in a context of a network key they are bound to. An application key shall only be bound to a single network key. A device key is implicitly bound to all network keys.

An example of binding application keys to network keys and models is illustrated in [Figure 3.51](#).

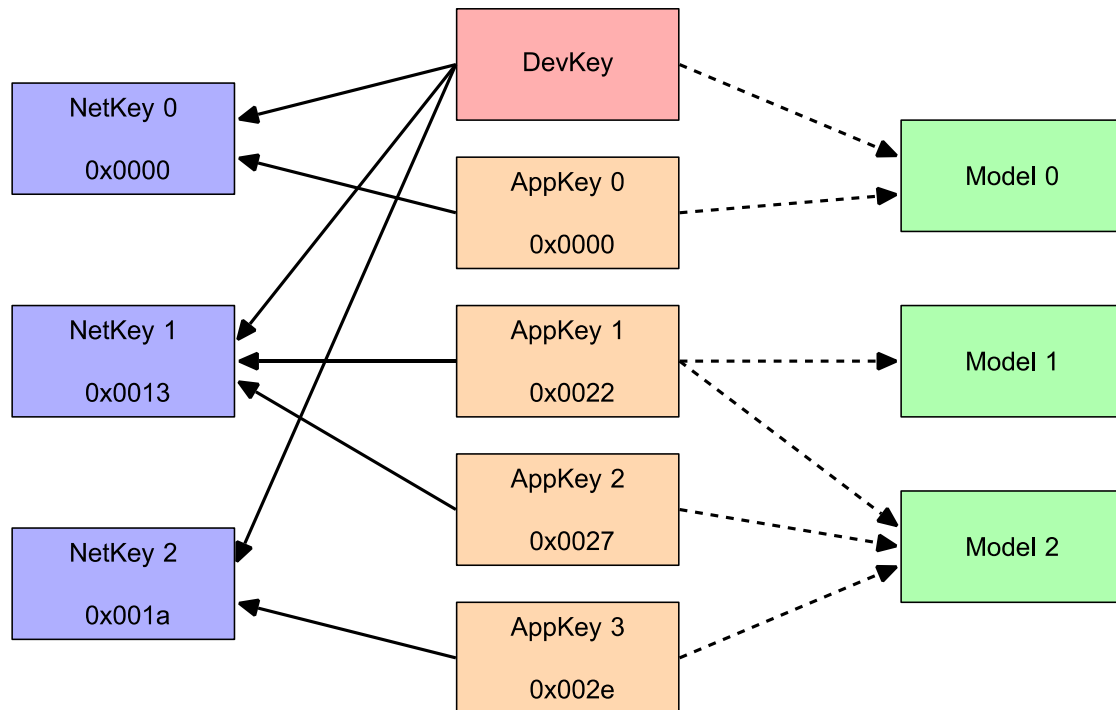


Figure 3.51: Application key binding example

3.9.6.1 Device key

The device key (DevKey) is an access layer key known only to the node, the Provisioner and a Configuration Manager. The device key shall be bound to every network key known to the node. Those bindings cannot be changed. An illustration of the device key derivation is shown in [Figure 3.52](#).

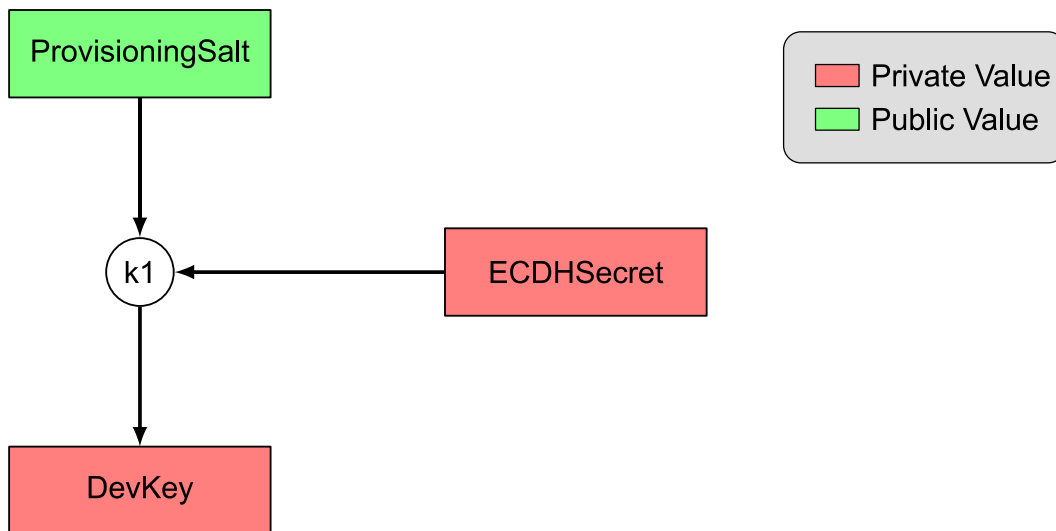


Figure 3.52: Device key derivation

The DevKey shall be derived from the ECDHSecret and ProvisioningSalt as described by the formula below:

$$DevKey = k1(ECDHSecret, ProvisioningSalt, "prdk")$$

The ProvisioningSalt is defined in Section 5.4.2.5 and the ECDHSecret is defined in Section 5.4.2.3.

3.9.6.2 Application key

The application key (AppKey) shall be generated using a random number generator compatible with the requirements in Volume 2, Part H, Section 2 of the Core Specification [1].

The AID is used to identify the application key. An illustration of the AID derivation is shown in Figure 3.53.

$$AID = k4(AppKey)$$

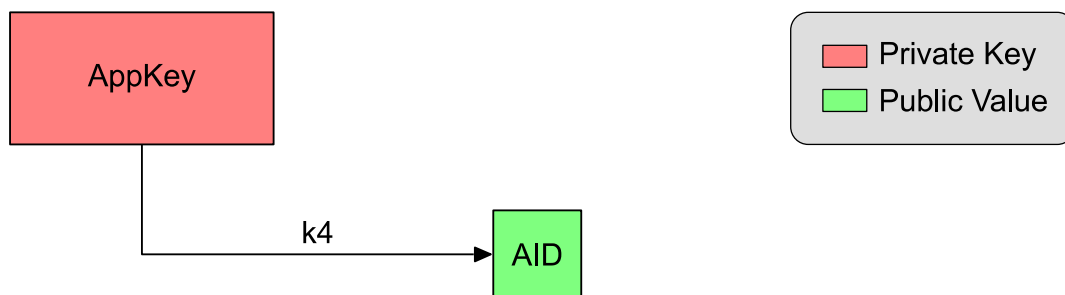


Figure 3.53: AID derivation

3.9.6.3 Network key

The network key (NetKey) shall be generated using a random number generator compatible with the requirements in Volume 2, Part H, Section 2 of the Core Specification [1]. An illustration of the network key hierarchy is shown in Figure 3.54.

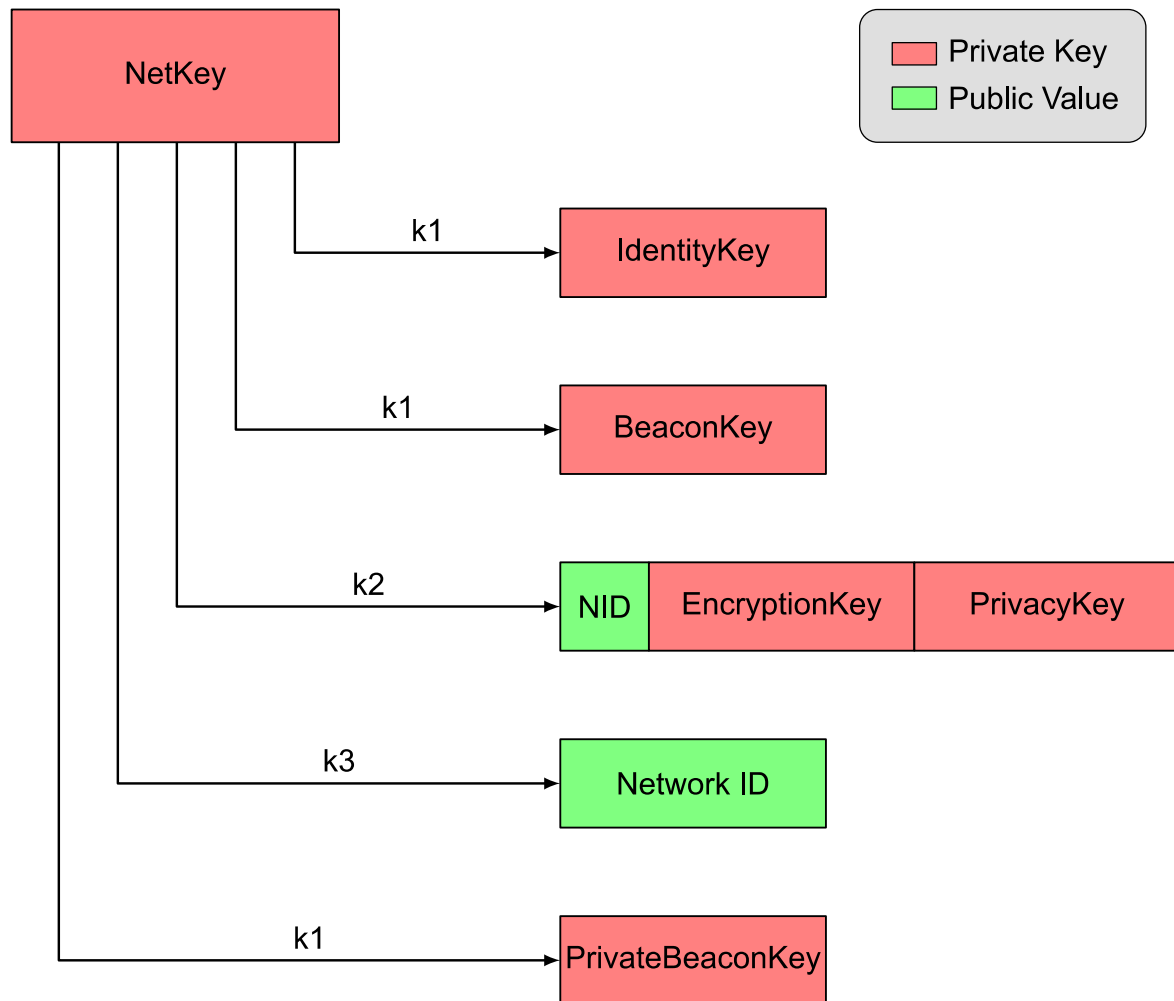


Figure 3.54: Network key hierarchy

3.9.6.3.1 NID, EncryptionKey, and PrivacyKey

Each Network PDU is secured using security material that is composed of the NID, the EncryptionKey, and the PrivacyKey.

The NID is a 7-bit value that identifies the security material that is used to secure this Network PDU.

Note: There are up to 2^{121} possible keys for each NID; therefore, the NID value can only provide an indication of the security material that has been used to secure this Network PDU.

The NID, EncryptionKey, and PrivacyKey are derived using the k2 function with security credentials as inputs.

The managed flooding security material is derived from the managed flooding security credentials using the following formula:

$$NID \parallel EncryptionKey \parallel PrivacyKey = k2(NetKey, 0x00)$$

The friendship security material is derived from the friendship security credentials using the following formula:

$$NID \parallel EncryptionKey \parallel PrivacyKey = k2(NetKey, 0x01 \parallel LPNAddress \parallel FriendAddress \parallel LPNCounter \parallel FriendCounter)$$

Where:

The LPNAddress value is the unicast address set as source address in the Friend Request message that set up the friendship.

The FriendAddress value is the unicast address set as source address in the Friend Offer message that set up the friendship.

The LPNCounter value is the value from the LPNCounter field of the Friend Request message that set up the friendship.

The FriendCounter is the value from the FriendCounter field of the Friend Offer message that set up the friendship.

For Network PDUs that are sent between a Low Power node and Friend node that have a friendship relationship, the friendship security material is used.

The directed security material is derived from the directed security credentials using the following formula:

$$NID \parallel EncryptionKey \parallel PrivacyKey = k2(NetKey, 0x02)$$

For Network PDUs that are transmitted according to directed forwarding functionality, the directed security material is used.

For all other Network PDUs, the managed flooding security material is used.

3.9.6.3.2 Network ID

The Network ID is derived from the network key such that each network key generates one Network ID. This identifier becomes public information.

$$Network\ ID = k3(NetKey)$$

3.9.6.3.3 IdentityKey

The IdentityKey is derived from the network key such that each network key generates one IdentityKey.

$$salt = s1("nkik")$$

$$P = "id128" \parallel 0x01$$

$$IdentityKey = k1(NetKey, salt, P)$$



3.9.6.3.4 BeaconKey

The BeaconKey is derived from the network key such that each network key generates one BeaconKey.

$$\text{salt} = s1("nkbk")$$

$$P = "id128" \parallel 0x01$$

$$\text{BeaconKey} = k1(\text{NetKey}, \text{salt}, P)$$

3.9.6.3.5 PrivateBeaconKey

The PrivateBeaconKey is derived from the network key such that each network key generates a unique PrivateBeaconKey.

$$\text{salt} = s1("nkp k")$$

$$P = "id128" \parallel 0x01$$

$$\text{PrivateBeaconKey} = k1(\text{NetKey}, \text{salt}, P)$$

3.9.6.4 Global key indexes

Network and application keys are organized within the mesh network into two lists, maintained by a Configuration Manager: a list of network keys and a list of application keys. Each list is a shared mesh network resource and can accommodate up to 4096 keys. Keys are referenced using global key indexes: the NetKey Index and the AppKey Index. The key indexes are 12-bit values ranging from 0x000 to 0xFFF inclusive. A network key at index 0x000 is called the primary NetKey.

3.9.7 Message security

Messages are secured using AES-CCM at two different layers. Messages are encrypted and authenticated at the network layer and at the upper transport layer. Each message is also obfuscated to hide possible identifying information from the packets. This is illustrated in [Figure 3.55](#).

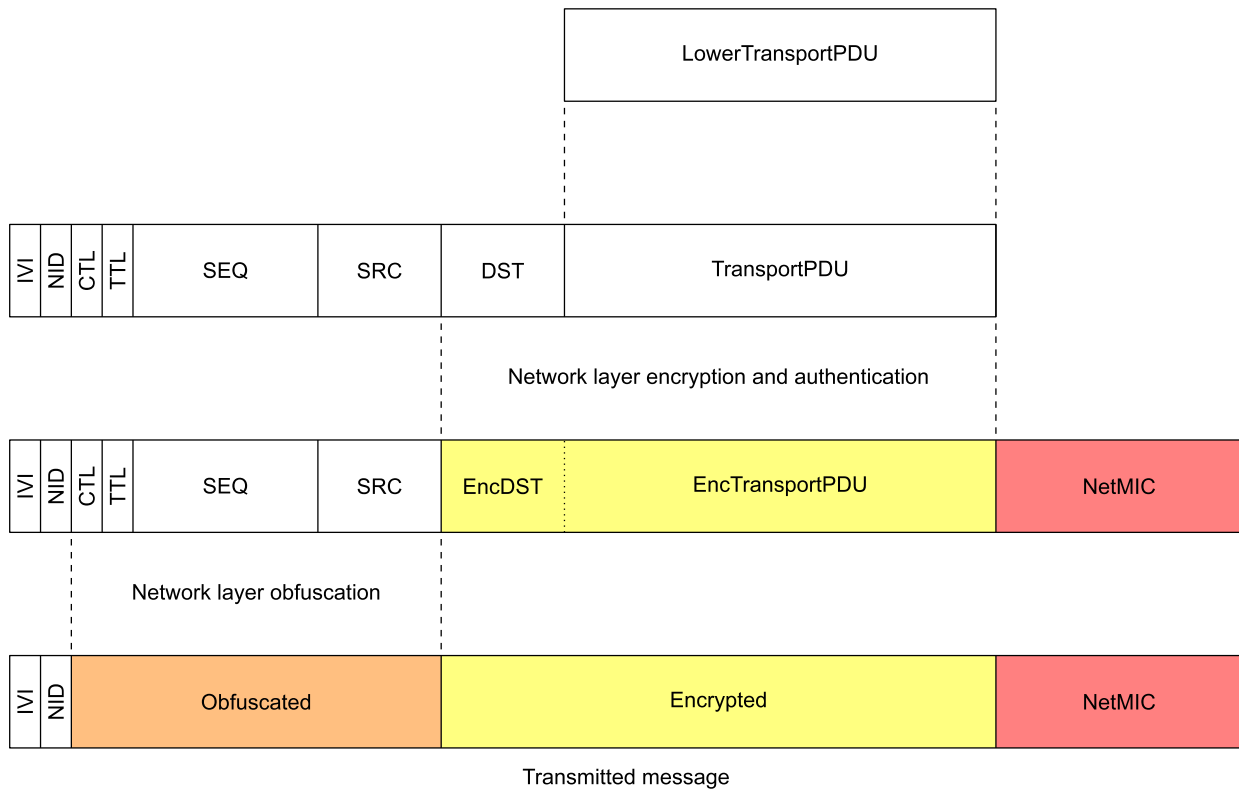


Figure 3.55: Example of network layer encryption, authentication, and obfuscation

Every message has a minimum of 64 bits of authentication information associated with it. This authentication information may be split between the network layer and upper transport layer.

Some messages, known as Transport Control messages, are not authenticated at the upper transport layer and therefore have a 64-bit NetMIC field. Access messages are authenticated at the upper transport layer and therefore have a 32-bit NetMIC field. Access messages that are sent in a single unsegmented message have a 32-bit TransMIC field. Access messages that are segmented over multiple Network PDUs can have either a 32-bit or 64-bit TransMIC field. This allows a higher layer to determine the level of authentication required to securely deliver the Access message and therefore apply the appropriate size for the TransMIC field.

3.9.7.1 Upper transport layer authentication and encryption

Authentication and encryption of the Access message is performed by the upper transport layer.

The Access message is encrypted and authenticated using AES-CCM. This is identical to the way that Bluetooth low energy encryption and authentication works. An illustration of the upper transport layer encryption is shown in [Figure 3.56](#).

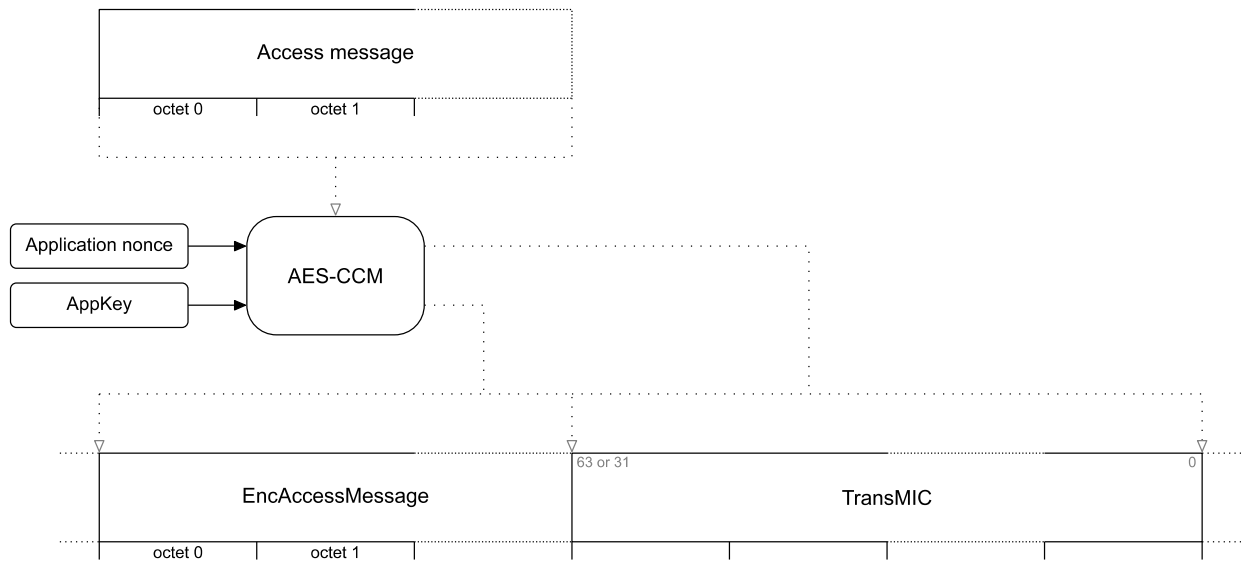


Figure 3.56: Upper Transport layer encryption

If the Access message is secured using the application key, then the Access message is encrypted using the application nonce and the application key.

If the Access message is secured using the device key, then the Access message is encrypted using the device nonce and the device key.

The nonce uses the sequence number and the source address, ensuring that two different nodes cannot use the same nonce. The IV Index is used to provide significantly more nonce values than the sequence number can provide for a given node. Management of the IV Index is described in Section 3.11.5.

Note: The authentication and encryption of the Access message is not dependent on the TTL field value, meaning that as the Access message is relayed through a mesh network, the Access message does not need to be re-encrypted at each hop.

When using an application key and the destination address is a virtual address:

$$EncAccessMessage, TransMIC = AES-CCM_{AppKey} (application\ nonce, Access\ message, Label\ UUID)$$

When using an application key and the destination address is a unicast address or a group address:

$$EncAccessMessage, TransMIC = AES-CCM_{AppKey} (application\ nonce, Access\ message)$$

When using a device key and the destination address is a unicast address:

$$EncAccessMessage, TransMIC = AES-CCM_{DevKey} (device\ nonce, Access\ message)$$

The concatenation of the encrypted Access message and the TransMIC is called the Upper Transport Access PDU:

$$Upper\ Transport\ Access\ PDU = EncAccessMessage \parallel TransMIC$$

3.9.7.2 Network layer authentication and encryption

The DST and the TransportPDU fields are encrypted and authenticated using AES-CCM. This is identical to the way that Bluetooth low energy encryption and authentication works.

All Network PDUs are encrypted using an EncryptionKey that is derived from a network key (see Section 3.9.6.3.1).

An illustration of the network layer encryption is shown in Figure 3.57.

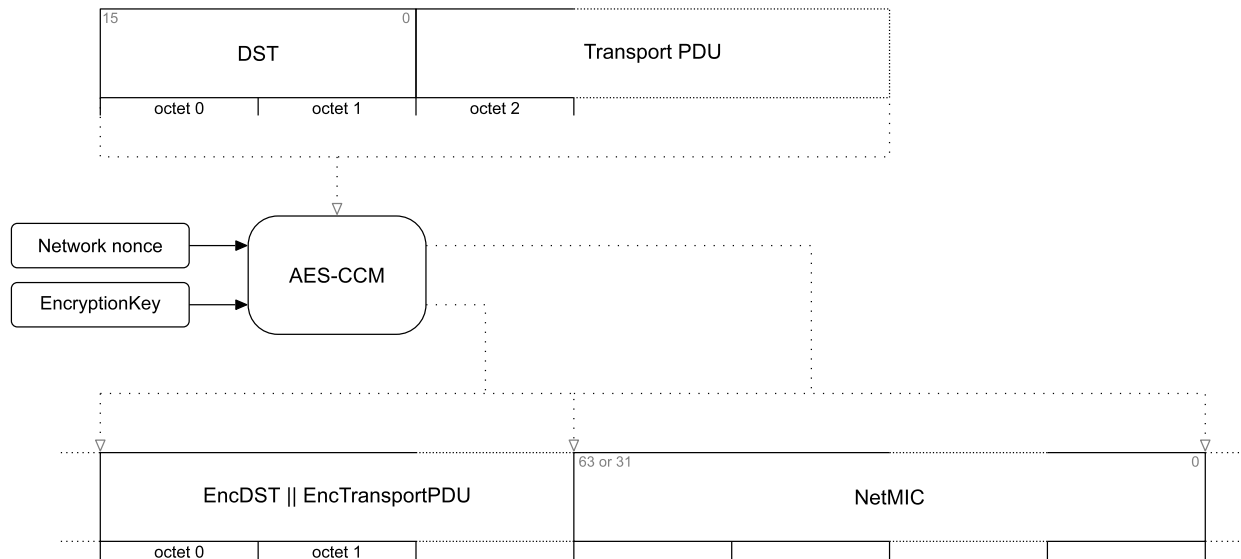


Figure 3.57: Network layer encryption

The following defines how this is performed:

$$EncDST || EncTransportPDU, NetMIC = AES-CCM_{EncryptionKey}(network\ nonce, DST || TransportPDU)$$

3.9.7.3 Network layer obfuscation

In order to obfuscate the Network Header (CTL, TTL, SEQ, and SRC fields), these field values shall be combined with a result of a single encryption function e , designed to prevent a passive eavesdropper from determining the identity of a node by listening to Network PDUs.

The obfuscation occurs after the Message Integrity Check value for Network (NetMIC) has been calculated. The obfuscation is calculated using information available from within the Network PDU. This obfuscation is designed only to help prevent a simple passive eavesdropper from tracking nodes. A determined attacker could still discover patterns within this obfuscation that can lead to the revealing of the source address or sequence number of a node. Critically, obfuscation does not enforce that inputs to the encryption function are unique.

Obfuscation does not protect the PrivacyKey from compromise, and given the above design considerations for protection against only passive eavesdroppers, it is considered that the PrivacyKey could be compromised with sufficient time. The design of obfuscation includes the IV Index, such that when the IV Index changes, any obfuscation attacks would have to start again.

To obfuscate the Network PDU, the first seven octets of the Network PDU that have already been encrypted are combined with the IV Index and a PrivacyKey.

These first seven octets of the Network PDU that have been encrypted include both the EncDST and five octets of either the EncTransportPDU or the EncTransportPDU concatenated with the NetMIC field. These octets are known as the PrivacyRandom value.

The PrivacyKey is derived using a key derivation function from the network key (see Section 3.9.6.3.1) to protect the network key even if the PrivacyKey is compromised.

The IV Index is concatenated with the PrivacyRandom value and used along with the PrivacyKey as inputs to the encryption function e . The output of this is known as the PECB value.

The first six octets of the PECB value are then exclusive-ORed with the CTL, the TTL, the SEQ, and the SRC fields, and then become the ObfuscatedData.

The Network PDU is transmitted as the concatenation of the NID, the IVI, the ObfuscatedData, the EncDST, the EncTransportPDU, and the NetMIC fields.

An illustration of the network layer obfuscation is shown in Figure 3.58.

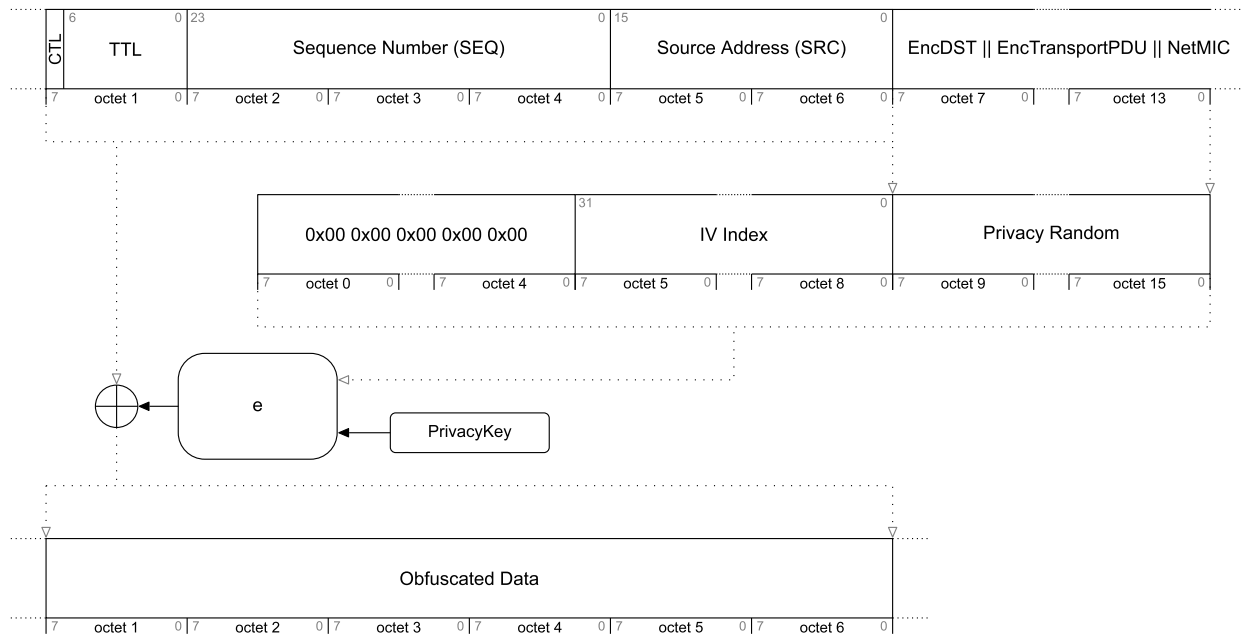


Figure 3.58: Network layer obfuscation

$$\text{Privacy Random} = (\text{EncDST} \parallel \text{EncTransportPDU} \parallel \text{NetMIC})[0-6]$$

$$\text{Privacy Plaintext} = 0x0000000000 \parallel \text{IV Index} \parallel \text{Privacy Random}$$

$$\text{PECB} = e(\text{PrivacyKey}, \text{Privacy Plaintext})$$

$$\text{ObfuscatedData} = (\text{CTL} \parallel \text{TTL} \parallel \text{SEQ} \parallel \text{SRC}) \oplus \text{PECB}[0-5]$$

When reversing this, the following operations are performed:

$$\text{Privacy Random} = (\text{EncDST} \parallel \text{EncTransportPDU} \parallel \text{NetMIC})[0-6]$$

$$\text{Privacy Plaintext} = 0x0000000000 \parallel \text{IV Index} \parallel \text{Privacy Random}$$

$$\text{PECB} = e(\text{PrivacyKey}, \text{Privacy Plaintext})$$

$$(\text{CTL} \parallel \text{TTL} \parallel \text{SEQ} \parallel \text{SRC}) = \text{ObfuscatedData} \oplus \text{PECB}[0-5]$$

3.9.8 Message replay protection

A message sent by a legitimate originating element can be passively received by an attacker and then replayed later without modification. This is called a replay attack.

Since the originating element has encrypted and authenticated the message using the correct keys, the receiver cannot determine whether it is under a replay attack solely by performing the message integrity checks (i.e., on the NetMIC and, if applicable, on the TransMIC).

The IVISeq value is composed of the IV Index and the sequence number of the message. The size of the IVISeq value is 7 octets, where the IV Index is the four most significant octets and the sequence number is the three least significant octets.

To increase protection against replay attacks, each element increases the IVISeq value for each new message that it sends. If a valid message has been received from an originating element with a specific IVISeq value, any future messages from the same originating element that contain an IVISeq value that is lower than or equal to the last valid IVISeq value are very likely replayed messages and shall be discarded. Therefore, messages are delivered to the access layer in ascending IVISeq value order.

If a message encrypted and authenticated with a lower IV Index value from the same originating element has been received, the message shall be discarded.

A node shall implement replay protection for all Access and Transport Control messages that are received from other elements and processed, as well as for proxy configuration messages, if applicable. A node shall be able to determine if a certain message is being replayed. If the node is not able to determine if a certain message is being replayed, the node shall ignore the message.

The check for replay protection involves checking whether the received message from an originating source address with the IVISeq value is a numerically higher number than the last valid IVISeq value from that source address. The check for replay protection is done above the network layer. If the check for replay protection reveals that a message is being replayed, the message shall be discarded; otherwise the message shall be processed.

Replay protection is organized based on a list, known as the replay protection list. This list consists of multiple entries and each entry has two fields:

- A source address, which is the SRC field of an incoming message
- An IVISeq value, which is composed as defined by a row in [Table 3.74](#)